
THE WORK A VERIFIER NEEDS: CERTIFIED INT8 OUTPUT HEADS THAT MATCH A PRODUCTION KERNEL’S ROUNDING

Samuel Mausberg

ABSTRACT

A greedy verifier in speculative decoding, like greedy decoding, needs only the LM head’s winning token, yet the engine scores the whole vocabulary. We find it from an int8 copy of the head: bound each score’s error, recompute exactly the few that could still win, and fall back to the engine’s head when bounds cannot decide. The token must match the engine’s head at the same batch size, after BF16 rounding and its tie rule, not exact arithmetic: their winners differ at 0.5% of greedy positions, and at one position past end-of-text, found in a certified run, stock batch-1 BF16 decoding, unlike Hugging Face’s, ranks first a token 8.9 nats below FP32’s top. We prove when bounds on the FP32 sums fix the token, assuming a tested bound on the closed-source kernel’s rounding error, and extend this to seeded sampling. For Qwen3.5-4B, whose head is 10.3% of a batch-1 plain decode step in SGLang on a GH200, the certified head recomputes about 1.8 entries per decision and leaves 0.28–1.46% of 60,000 replayed positions to the fallback under two error models; every token equals the engine’s, also at all 592,433 positions certified while serving with the stock head run beside it. Served in three sessions against tuned stock configurations, it raises throughput at one request by 2.5% for plain decoding, 4.6% for MTP and 1.2% for DFlash, the fastest there, with identical tokens; the fastest at each of 2 to 8 requests loses 1.0–3.0% (calls of up to 30–64 positions, most falling back), and 0.6% at 128 (gated off). Attention, recurrent-state and admission changes gain more. Bounds carried over from the drafters’ heads skip under 0.7% of the vocabulary in every variant tested: draft and target head inputs differ by a median 92–95% of the target’s norm, where about 1% is needed.

1 INTRODUCTION

In speculative decoding a cheap drafter proposes several tokens and the target model checks them all in one forward pass (Leviathan et al., 2023; Chen et al., 2023). At each checked position the target multiplies its final hidden state, the *head input*, by the output head, which gives a score (logit) for every vocabulary entry. A greedy verifier uses one fact from those scores, which entry is largest, and accepts a drafted token if and only if it is that entry. Plain greedy decoding makes the same single use of the head. A shortcut through the head must return exactly that token. This paper asks how little of the head’s work that requires, and what “exactly” has to mean inside a production engine.

The speed at stake is modest. Qwen3.5-4B’s head has 248,320 entries; in BF16 it is 15% of the weights a

plain decode step reads (derived from the checkpoint; Appendix E.1). On a GH200 at batch 1 the head takes 10.3% of a plain decode step [E43], so even a free head would speed up such a step by only about 11%. With the model’s one-layer multi-token-prediction drafter, MTP-4B, the target’s verification head takes 4.4% of a profiled cycle at batch 1 [E42] and the drafter’s own projections through the same head a further 12.9% (Figure 1). A drafter’s head need not be exact, but a shortcut that returns the engine’s token leaves the drafts, and so the acceptance rate, unchanged.

Most of the head’s computation goes to entries that cannot win, and it can be skipped by bounding instead of computing (Figure 2). Approximate every score cheaply, here from an int8 copy of the head, which reads half the bytes; bound each error, so that every entry gets an interval containing its exact score; discard every entry whose interval lies wholly below another’s and recompute the few that remain exactly; and when the bounds still cannot single out the winner, fall back to the full computation. This pays when the intervals are narrow compared with the gap between the two best scores, as they are on this model (Section 4.2). Certified greedy screening of an LM head has public implementations (sparkpipe contributors, 2026; Hawkins, 2026; Layr Labs, 2026; Chamberlain, 2026), and exact sampling from bounded scores is older (Maddison et al., 2014; Musmann

Code, exact-arithmetic references, committed evidence, research notes and the source of this paper: <https://github.com/SamMausberg/verified-progress>. Small bracketed tags cite the evidence register (Appendix F.1): [En] names the committed files behind a result. A number computed from measured ones is marked “(derived)”.

AI (Claude Opus 5.5, GPT-6 Astra Pro) was used in developing the paper and the code. The author directed the work and is responsible for its claims.

Preprint, 3 October 2026; not peer reviewed.

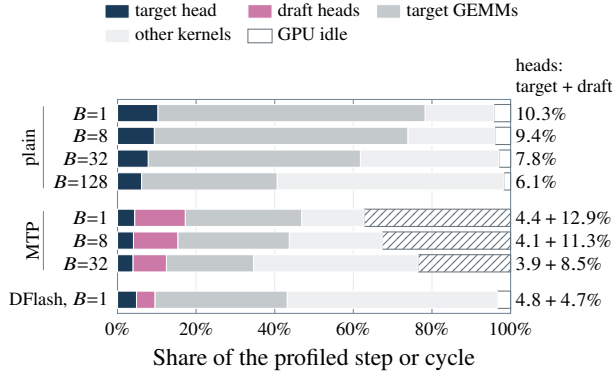


Figure 1. Where the GPU time of a stock decode step goes (Nsight Systems; no certified head), with B requests per decode step (in plain decoding, head calls of $M = B$ positions) [E42]. The target head comprises its GEMM, the FP32 copy of its logits and the argmax. Tracing inflates the MTP cycle’s idle time (hatched) but barely changes kernel durations, so the MTP heads’ shares of an untraced cycle are larger than shown (Appendix E.2). DFlash: the tuned configuration with 16-token blocks, GPU-bound [E46].

et al., 2017); Section 6 compares them.

The hard part is deciding what the cheap computation must agree with, because the engine does not compute exact scores. It sums each logit in FP32, in an order its kernel chooses, rounds the sum to BF16, and returns the first index among equal maxima. BF16 keeps eight significant bits, so logits between 16 and 32 are 0.125 apart. In a hypothetical case, logits of 20.02 and 20.05 both round to 20.0. The engine then returns whichever entry has the lower index, even if its exact score is the smaller. On the captured greedy positions of this model, the engine’s two largest logits tie in BF16 at 1.0% of positions, and the exact-arithmetic winner differs from the engine’s token at about 0.5% [E1].

We therefore take the engine’s own head as the reference: a certified head must return the token the unmodified (stock) head returns for the same head input at the same batch shape, the number of positions in one head call. That makes its correctness testable by bit-equality against the unmodified engine in the same configuration (its serving costs: Section 4.5). We prove when intervals around the stock kernel’s FP32 sums, before rounding, fix that token (Section 2). The proof needs a bound on the kernel’s rounding error, which no vendor documents, so we assume one under two named error models and test it. Positions the intervals cannot decide run the stock head, so a failed certificate costs time, not agreement; a certified token can be wrong only if an assumption of Table 3 fails.

A second source of bounds would reuse the drafter’s work. A drafter that projects through the same head, such as MTP-4B or DFlash-4B, the public block drafter for this model,

computes every logit at its own head input. For each tile of 64 entries it could also record the largest score and, for sampling, the sum of exponentiated scores at little extra cost. A Cauchy–Schwarz bound carries these tile summaries from the drafter’s head input to the target’s; we call this *transport*. Its bounds are sound, but they are narrower than the int8 copy’s only if the *drift*, the distance between draft and target head inputs relative to the target’s, is below about 1% (Section 5.2). The median drift is 92% for DFlash-4B and 95% for MTP-4B, so in every variant tested transport skips under 0.7% of the vocabulary.

1.1 Contributions

What is new here is a certificate for the token of a closed-source production kernel: a public Metal implementation recomputes its candidates with a replica of its engine’s kernel (Layr Labs, 2026), but cuBLAS cannot be replicated, so we bound its error with a model.

1. *A contract against the engine’s own head* (Section 2): proved conditions under which intervals around the stock kernel’s FP32 sums fix its token at the same batch shape, with the stock head as fallback. Exact arithmetic is the wrong reference: its winner is not the engine’s token at 0.5% of greedy positions (Section 4.1), and at one position the engine’s batch-1 BF16 decoding puts first a token 8.9 nats below FP32’s top (Section 4.5). The error model is assumed and tested, not proved.
2. *An error bound for tensor-core accumulation, and an extension to seeded sampling* (Section 3): the int8 copy’s error bound covers truncating tensor-core adders that keep at least 23 fractional bits, and the same decision certifies the token of SGLang’s seeded sampler.
3. *Measurements offline, inside the engine and served* (Section 4): recomputation, fallback and token agreement on the engine’s own states; served in three sessions against tuned stock configurations, the head gains 1.2–4.6% at one request, short of its head-only prediction, and the fastest at each of 2 to 8 requests loses 1.0–3.0%.
4. *A negative result* (Section 5): for both public drafters and every bound variant and tiling tested, certified transport skips under 0.7% of the vocabulary.

Appendix G defines every term; the repository’s research notes record the wider programme, failures included.

2 WHAT THE ENGINE COMPUTES AND WHAT A CERTIFICATE MUST MATCH

2.1 What the Engine Computes

The head is a matrix $W \in \mathbb{B}^{V \times D}$ of $V = 248,320$ vocabulary entries of width $D = 2,560$, where $\mathbb{B}$ denotes the BF16 numbers; row w_i belongs to entry i . We call one head input,

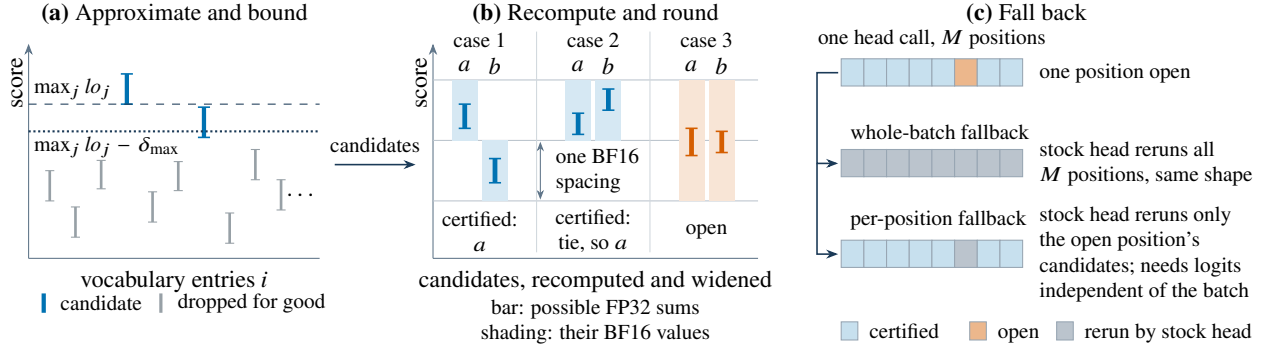


Figure 2. How the certified head chooses a token (schematic; values illustrative). (a) The int8 copy gives every vocabulary entry an interval $[lo_i, hi_i]$ that contains its exact score; an entry with $hi_i < \max_j lo_j - \delta_{\max}$, where $\delta_{\max}$ is a margin for the stock kernel’s modelled error and rounding (Section 2.2), cannot win and is dropped. (b) Three alternative candidate pairs a and b , recomputed from their BF16 weights and widened to the FP32 sums the stock kernel could form (bars; their possible BF16 values shaded). (c) An open position sends the call to the stock head, for the whole batch at the same shape or, with batch-independent logits, for its candidates only (Section 2.3).

with the token decided from it, a *position*. A decode step or a verification pass sends M positions through the head in one call, and M is the *batch shape*. SGLang computes the head as `torch.matmul` of the BF16 head inputs with W^T : a cuBLAS kernel, chosen by the batch shape, sums each logit’s D products in an FP32 *accumulator* and rounds the result to BF16. The logits are widened exactly to FP32, and greedy decoding and verification take `torch.argmax`, which returns the smallest index among equal maxima. We call this unmodified computation the *stock head*, and its GEMM the *stock kernel*.

2.2 References for One Decision

The three references differ in where rounding enters. Let $h \in \mathbb{B}^D$ be a position’s head input and $z_i = \langle w_i, h \rangle$ the *real logit* of entry i , in exact arithmetic over the stored operands. The stock kernel forms an FP32 accumulator $\hat{y}_i$ and outputs $L_i = \text{rn}_B(\hat{y}_i)$, where rn_B rounds to the nearest BF16 value (ties to even); $\text{ulp}_B(x)$ denotes the BF16 spacing at $|x|$.

Definition 2.1 (References for one decision). R-real returns $\min\{i : z_i = \max_j z_j\}$; R-bf16 returns $\min\{i : \text{rn}_B(z_i) = \max_j \text{rn}_B(z_j)\}$; and R-stock returns $\min\{i : L_i = \max_j L_j\}$.

R-stock is what an unmodified server returns. R-bf16, which rounds each real logit once, does not give the engine’s result: the engine rounds its own FP32 sum, which can lie on the other side of a rounding boundary. An exact emulation of a sequential FP32 kernel gives an example with $D = 3$ on which the two choose different tokens (Appendix D.1) [E28]. A certified head therefore targets R-stock.

Lemma 2.2 (BF16 separation). *For real y_a, y_b , if $y_a - y_b > \text{ulp}_B(\max(|y_a|, |y_b|))$ then $\text{rn}_B(y_a) > \text{rn}_B(y_b)$.*

Theorem 2.3 (Gap condition; proof in Appendix A.2). *Let $|\hat{y}_i - z_i| \leq G_i$ for every i , and let a be the R-real token. If*

for all $b \neq a$

$$z_a - z_b > \delta_{ab} := G_a + G_b + \text{ulp}_B(\max(|z_a|, |z_b|) + \max(G_a, G_b)),$$

then R-stock returns a .

The stock kernel’s error Theorem 2.3 needs a bound G_i on how far the stock kernel’s FP32 accumulator can lie from the real logit; for a closed-source kernel it can only be assumed. We write it as $G_i = \gamma_{\text{stock}} \sum_j |w_{ij} h_j| + \eta$, where the floor $\eta = 2^{-113}$ covers underflow in products and partial sums. For an entry never recomputed, Cauchy–Schwarz bounds $\sum_j |w_{ij} h_j|$ by $\|w_i\|_2 \|h\|_2$ from stored row norms.

The constant γ_{stock} comes from one of two named *error models*. The *conservative model* charges $2D$ roundings of relative size 2^{-23} in any order, $\gamma_{\text{stock}} = 6.11 \times 10^{-4}$. It covers any summation order, split- K (a sum divided among thread blocks) with FP32 partial sums, and truncating adders that keep at least 23 fractional bits per block of 8 to 32 addends. The *Hopper model*, $\gamma_{\text{stock}} = 1.193 \times 10^{-4}$, starts from Khattak and Mikaitis’s measurement-based model of BF16 tensor-core arithmetic on Hopper, the GH200’s GPU architecture (Khattak & Mikaitis, 2026), and adds an allowance for FP32 split- K . Khattak and Mikaitis measured it through warp-level `mma` instructions; the two stock head kernels we inspected ($M = 1$ and 32) issue `warpgroup wgmma` instead, and on crafted inputs all fourteen kernels cuBLAS selects for $M = 1$ to 256 accumulate as the Hopper model assumes: blocks of 16 products, 25 fractional bits, and truncation of the dropped bits and of the sum [E13]. That is a measurement on one GPU, and neither model is a vendor contract. The largest error observed in the stock GEMM’s FP32-output variant, a separate kernel whose accumulator can be read, is about 1/58 of the Hopper model’s bound [E12]. Neither

model covers split- K partial sums stored in BF16, which PyTorch permits by default (PyTorch contributors, 2026); on this build, forbidding them changes neither the selected kernel nor any output bit [E12].

A screen needs a margin A screen sees only an interval $lo_i \leq z_i \leq hi_i$ for each entry, and it must not drop any entry that could still be the stock token, including entries it never recomputes. An entry that loses in real arithmetic can still tie or win after the stock kernel’s error and rounding. Let $G_{\max} = \max_i G_i$ and let $\bar{z} = \max_i \max(|lo_i|, |hi_i|)$ be the largest magnitude any interval reaches. Then $\delta_{\max} = 2G_{\max} + \text{ulp}_B(\bar{z} + G_{\max})$ is at least δ_{ab} of Theorem 2.3 for every pair, so, given valid intervals, an entry with $hi_j < \max_i lo_i - \delta_{\max}$ can be dropped for good (Figure 2a).

2.3 The Interval Condition and the Fallback

The gap condition demands a margin of more than one BF16 spacing even when G is negligible, which a few percent of positions lack (Table 6). The rounding boundaries can be used exactly instead. Rounding is monotone, so an interval $[\hat{y}_i^-, \hat{y}_i^+]$ that contains entry i ’s accumulator puts L_i in $[L_i^-, L_i^+] = [\text{rn}_B(\hat{y}_i^-), \text{rn}_B(\hat{y}_i^+)]$ (Figure 2b).

Proposition 2.4 (Interval condition). *Let C be a set of entries such that every $j \notin C$ has $L_j^+ < \max_{i \in C} L_i^-$, and let $k = \min\{i \in C : L_i^- = \max_{j \in C} L_j^-\}$. If $L_j^+ < L_k^-$ for every $j \in C$ with $j < k$, and $L_j^+ \leq L_k^-$ for every $j \in C$ with $j > k$, then R-stock returns k .*

Proof. For $j < k$ in C , $L_j \leq L_j^+ < L_k^- \leq L_k$; for $j > k$ in C , $L_j \leq L_j^+ \leq L_k^- \leq L_k$, so a later index can at most tie and the first-index rule returns k ; entries outside C satisfy $L_j < L_k^- \leq L_k$. $\square$

A BF16 tie holding under every accumulation the error model admits is certified, and the lower index wins; only a tie the accumulation error could make or unmake stays open.

The contract and the fallback A certified head promises R-stock at the batch shape it serves. It certifies a position only when Proposition 2.4 fixes that token for every accumulator value the named error model admits, and sends every other position to a fallback that runs the stock head. The *whole-batch fallback* reruns the same `torch.matmul` on the whole batch at the same M , so cuBLAS selects the same kernel. It needs no error model, but one undecided position makes the whole batch pay. The *per-position fallback* finishes one undecided position with a stock `matmul` over its gathered candidate entries; it applies only to a position left open by a near tie whose candidate list is complete and has at most 64 entries (Section 3.4). It is exact only if a position’s BF16 logits stay the same when the rest of the

batch, or the set of entries computed, changes. In principle cuBLAS may choose another kernel, and so another summation order, for each batch shape; on this build a position’s logits were bitwise the same at 20 batch sizes from 1 to 256, with all entries or only gathered ones computed, and a start-up self-test re-checks the gathered case at the deployed batch sizes [E14]. Nothing guarantees that for other shapes or builds, so the contract still fixes M . Either fallback is a conditional node of the recorded CUDA graph, paid only when it runs. Table 3 in Appendix A lists the assumptions behind every agreement claim; the offline replay approximates real arithmetic in FP64.

3 THE CERTIFIED HEAD

The certified head finds the R-stock token from an int8 copy of W evaluated at the target’s own head input.

3.1 An Error Bound for the Int8 Copy

The deployed copy stores each row as int8 codes q_i with one scale s_i , so $\bar{w}_i = s_i q_i$ approximates w_i with a known error $e_i = w_i - \bar{w}_i$. The pass that evaluates the copy multiplies the codes by the BF16 head input and sums in FP32 on tensor cores. Their adders align several addends to the largest, truncate them and round once (Fasi et al., 2021), so the standard bound for IEEE additions (Higham, 2002) does not apply. We bound the sum’s error by $\gamma_{\text{copy}} \sum_j |\bar{w}_{ij} h_j|$ with the conservative constant, $\gamma_{\text{copy}} = 6.11 \times 10^{-4}$, whichever model is assumed for the stock kernel. Lemma A.1 and the adder model of Appendix A.3 show that it covers the accumulations listed for the conservative model in Section 2.2.

Theorem 3.1 (Error bound of the copy). *If every product is exact in FP32 and the pass’s accumulation error is at most $\gamma_{\text{copy}} \sum_j |\bar{w}_{ij} h_j|$ plus an underflow floor η' , its value $\tilde{z}_i$ satisfies $|z_i - \tilde{z}_i| \leq \beta_i = \|e_i\|_2 \|h\|_2 + \gamma_{\text{copy}} \|\bar{w}_i\|_2 \|h\|_2 + \eta'$.*

We call β_i the copy’s *error bound*: under the theorem’s assumptions, entry i ’s real logit lies in the interval $\tilde{z}_i \mp \beta_i$. The kernel rounds the norms of h up and the interval ends outward, so every comparison it computes is sound, provided it evaluates the analysed operations (Table 3). The screen thus also rests on a hardware assumption, always the conservative one; the Hopper model enters only through the stock kernel’s G_i . The bound assumes the worst alignment of e_i and h , so it is loose: its median on captured states is 1.00 logit, about 75 times the median realized error [E5]. Appendix A proves a general form (Theorem A.2) and gives η' .

3.2 Refinement Against the Stock Kernel

The certified head decides each position in five steps.

1. Evaluate the int8 copy for every entry and form the

intervals $[lo_i, hi_i] = \tilde{z}_i \mp \beta_i$ (Theorem 3.1).

2. Keep as *candidates* the entries with $hi_i \geq \max_j lo_j - \delta_{\max}$; given valid intervals, no other entry can be the stock token (Section 2.2).
3. Recompute each candidate’s real logit from its BF16 row in FP64 (whose tiny rounding error is added) and widen it by G_i to an interval for the stock kernel’s accumulator.
4. Round these intervals to BF16 and apply Proposition 2.4; if it fixes a token, emit it.
5. Otherwise send the position to a fallback (Section 2.3).

The screen in step 2 changes which entries are recomputed, never which decisions stay open, because an entry it drops loses to the stock token under every admissible accumulation. The same steps, with $\delta_{\max}$ removed, decide the R-real token; Appendix A.7 states the general refinement and what of it is machine-checked.

3.3 Seeded Sampling and Fixed-Noise Verification

With a seed and no top- k or top- p truncation, SGLang samples a token as the winner of a race: it adds Gumbel noise, computed from a 32-bit hash of (seed, token position in the sequence, token), to the log-probabilities and takes the argmax (SGLang contributors, 2026). Because the noise is fixed before any score is known, the same five steps certify the race’s winner, with the bounds applied to the noisy scores (Appendix B). The stock seeded sampler exists only in SGLang’s deterministic-inference mode, whose head kernel is DeepGEMM’s, not cuBLAS; the conservative model covers it, and the Hopper model is unchecked for it. With its fallback, the certified decision returned the sampler’s token at all 60,000 kernel-replay positions at $T = 0.7$ and 1, on the same logits [E19], and at all 21,016 verification positions of the engine check at $T = 0.7$ [E20]. We also implemented *fixed-noise verification* (Daliri et al., 2025), which accepts a drafted token exactly when it wins the target’s race at its position. Against the rejection sampling of SGLang’s speculative sampler, which it replaces, it accepts deterministic drafts (SGLang’s default) as often and sampled drafts as little as half as often (Appendix B.2); it follows the target’s distribution only for idealized continuous noise and prefix-only logits, which the engine does not guarantee (Appendix B.1), and its acceptance on real drafts was not measured.

3.4 The Kernel

The certified head runs as a short pipeline of GPU launches whose grids depend only on M , recorded with its fallback as one CUDA graph per batch size. The int8 pass multiplies the int8 weights by the BF16 head inputs (W8A16), accumulates in FP32 on tensor cores and computes each interval in its epilogue with directed rounding. Instead of writing the $M \times V$ logits, each GPU program keeps, per position

and chunk of vocabulary entries, the four largest upper ends and the largest among the rest; a selection step then keeps every stored entry that reaches the screen’s threshold, and the whole chunk if the rest’s largest does, so no candidate is lost. A position whose candidates overflow the list’s fixed capacity goes to the whole-batch fallback. A kernel inherits the error bound only if it computes the modelled arithmetic, which is checked, not proved: each compiled variant must pass a self-test before it certifies, and every call checks eight exactly computed vocabulary entries per position [E9]. The checks caught two faults, among them wrong products from Triton 3.7.1’s int8 tiles loaded through a 64-byte TMA box; both are refused (Appendix A.9) [E10,E11]. The int8 copy takes 0.64 GB beside the resident BF16 head (derived).

4 MEASUREMENTS

This section measures the certified head on states recorded from the engine, then the served engine with and without engine changes. Its numbers come from six sources, each named with its decision rule where it is used (Table 24 in Appendix F.1): the *held-out replay* in FP64, the *kernel replay* at the captured batch shapes, the *engine check* inside SGLang, a head-only *microbenchmark*, the *speculative captures* and the served sweeps (Section 4.5). Numbers labelled baseline are the stock engine’s.

4.1 Setup

System One NVIDIA GH200 with SGLang (Zheng et al., 2024) at commit bd66ce343e4f, PyTorch 2.13 for CUDA 13.0 and, unless a configuration names another, the Flash-Infer attention backend (Appendix F.3 lists every version). The target is Qwen3.5-4B at revision 851bf6e806ef (Qwen, 2026a), whose decoder layers are, three in four, Gated DeltaNet (GDN) layers: linear attention with a recurrent state per request (Yang et al., 2025). The drafters are MTP-4B, the model’s native MTP layer, run with top- k 1 and four draft steps in the captures (three in Figure 1 and served), and DFlash-4B, the public block drafter for this target (Chen et al., 2026; z-lab, 2026), which drafts blocks of 16 tokens and projects them through the target’s head.

Capture A capture-only engine patch records each position’s head input as BF16 bit patterns, with the token the engine chose; under speculation it also records the drafter’s head inputs and pairs each drafted token with the target head input that verifies it. The capture servers ran without CUDA graphs or the overlap scheduler, with at most 16 running requests in plain decoding (Appendix C.4). The 320 *capture prompts* come from eight public datasets in four domains and were decoded greedily for 384 tokens. They were split by prompt into an analysis half, on which the data-dependent choices were made, and a held-out half.

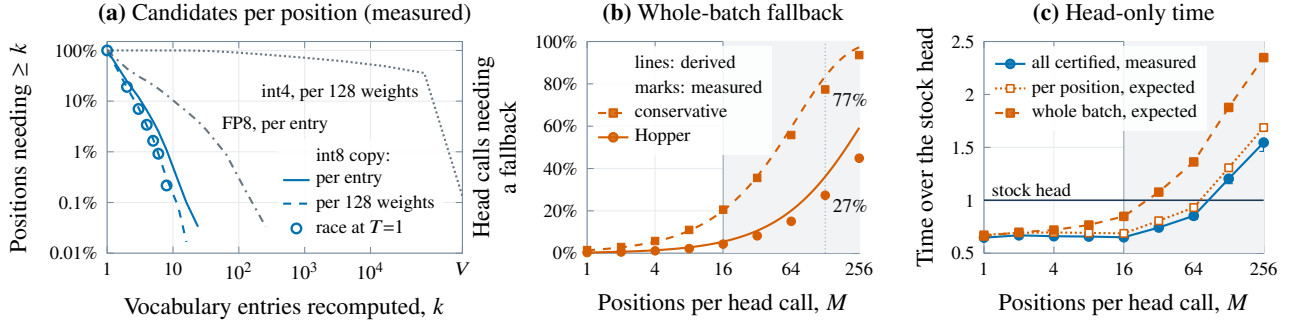


Figure 3. What the certified head costs. **(a)** Held-out replay, R-real decision: share of positions that need at least k candidates, for four copies of the head (greedy) and for the per-128-weights int8 copy in a seeded race at $T = 1$ [E4]. **(b)** Share of head calls of M positions that fall back to the stock head: lines, independent fallback at the held-out shares of Table 6 [E6]; markers, the kernel replay [E17]. **(c)** Time per call over the stock head’s, head alone: measured with every position certified (medians; p10–p90 within the markers below 256), and expected with each fallback under the conservative model from measured times and the rates of (b) [E17]. Shaded: above 16 positions the calls are consecutive replay positions, not engine batches.

The deployed copy and its bound depend on the weights alone, so the kernel replay draws its positions from both halves. In plain decoding the FP64 argmax of the captured head input equals the engine’s token at 99.51% of positions, and every disagreement lies within one BF16 spacing of the FP64 winner [E1].

A fixed configuration Bit-equality is a useful test only within one configuration, because the stock engine’s own output varies across configurations. On a server that runs at most 16 requests at once and queues the rest, plain greedy decoding differs within 256 tokens on 52% of 320 separate prompts between a client that keeps 1 request in flight and one that keeps 32 (Appendix D.6) [E34]. A bit-exact trace places the first differences upstream of the head, in kernels whose results depend on the batch or the memory pools, never in the head GEMM or the tie rule [E33]. Every comparison therefore fixes the serving configuration and compares speculative runs with stock speculative verification.

4.2 How Many Entries Are Recomputed

Kernel replay, R-stock On the first 60,000 captured positions, the certified head recomputes about 1.8 entries per decision on average and one at the median, depending on the stock-kernel model through the margin $\delta_{\max}$ (Table 6 in Appendix A.8) [E15].

Held-out replay, R-real This replay compares copies of the head on 6,005 held-out plain-decoding positions, drawn as whole decode steps, with the simpler R-real decision, whose screen needs no margin. No error bound was violated, and the exact winner was always a candidate [E2,E3]. Int8 with one scale per entry needs 1.55 candidates on average; FP8 needs about three times as many, and int4 leaves tens of thousands (Figure 3a; Table 4). Int8 is enough be-

cause the margins are wide: its intervals are about two logits wide, while the median gap between the two largest logits is 7.3 logits [E5], so a tighter bound would help little (Appendix A.8).

4.3 Fallback and Agreement

Kernel replay, R-stock, interval condition The certified head leaves 0.28% of positions to the stock head under the Hopper model and 1.46% under the conservative model, and with that fallback every token equals the engine’s [E15]. The held-out replay gives nearly the same shares, 0.35% and 1.40%; under the gap condition, which needs a margin of more than one BF16 spacing, its share never falls below about 2% (Table 6). This agreement is consistent with each model, not a proof of any: even an IEEE FP32 summation tree, with a γ_{stock} about 360 times smaller than the conservative one, certifies every held-out position and gets every token right [E6].

Fallback grows with the batch, because one undecided position sends the whole batch to the stock head. If positions fell back independently with probability p , a call of M positions would contain an undecided one with probability $1 - (1 - p)^M$, which matches the measured share of decode steps with a fallback (Table 6). On calls of 128 consecutive positions the kernel replay measures 27% and 77% (Hopper, conservative), where the held-out rates predict 36% and 84% (Figure 3b; Appendix A.8) [E17]. With the whole-batch fallback most large batches would rerun the stock head, so at serving batch sizes the certified head depends on the per-position fallback.

Engine check, R-stock, interval condition The engine check ran the kernel inside SGLang in *check mode*, which also runs the stock head and counts the positions whose

token differs, untimed and under the conservative model, on plain decoding, greedy and fixed-noise verification and the drafters’ projections. In concurrent runs (up to 16 requests for plain decoding, 8 for speculation) none of 196,734 certified positions differed from the stock head at the same batch shape [E20]. Runs with one request at a time gave identical certified and stock outputs on all 64 prompts for plain decoding and MTP-4B; DFlash-4B’s were compared only when served (Section 4.5). Every path used the whole-batch fallback except one plain-decoding run, which used the per-position fallback. Plain decoding fell back at 1.38% of positions (kernel replay: 1.46%) and MTP-4B verification at 1.74%. DFlash-4B verification sends up to 128 positions per call and left only 3.99% of them open, yet ran the whole-batch fallback in 90% of its verify head calls.

4.4 Head-Only Runtime

The certified head pays off only if

$$T_{\text{pass}} + T_{\text{select}} + T_{\text{rescore}} + P_{\text{fb}}(M) T_{\text{fallback}}(M) < T_{\text{head}}(M), \quad (1)$$

where the left side adds the int8 pass, the selection of candidates, their recomputation and the fallback’s time weighted by the share $P_{\text{fb}}(M)$ of calls that fall back, and T_{head} is the stock head (GEMM, FP32 copy and argmax).

In the microbenchmark (the head alone at $M = 1$ to 256, CUDA graphs, warm L2 cache; medians of 30 graph replays) a fully certified batch at $M = 1$ takes 239.6 μs , against 370.5 μs for the stock head. Adding each fallback’s measured cost at its measured rate gives an expected time of about 0.7 of the stock head’s up to 16 positions, with the per-position fallback under the conservative model (Figure 3c) [E16,E17]. From 128 positions every mode is slower, because the int8 pass alone is slower than the stock head there; with the whole-batch fallback, 77% of calls also pay for the stock head. The pass is not bound by memory bandwidth: at one position its directed-rounding epilogue keeps SM throughput at 79% of peak while it draws 62–63% of DRAM bandwidth, so its time grows with M faster than the stock GEMM’s [E18]. At one position the expected time under the conservative model is 0.66–0.67 of the stock head’s with either fallback [E17], so with the head at 10.3% of a plain decode step at batch 1, the step would shorten by at most about 3.4–3.5% (derived; served, 2.5%, Section 4.5).

4.5 Served Decoding

The stock baseline Each serving configuration, or *arm*, runs with CUDA graphs and the overlap scheduler (checked at launch) and without the prefix cache, which the workload does not reuse. A *session* launches each compared arm afresh, once, at full capacity, and sweeps client concurrency c , the requests a closed-loop client keeps in flight, on 1,152 held-out public prompts (chat, code, mathematics) with 512-

token outputs [E48]. Figure 4 plots the per-request output rate x , including the time to first token, against throughput y (Appendix E.3). Over three sessions speculation serves more than plain decoding up to $c = 32$ and, with SGLang’s default admission, less from 48, for every drafter [E50]. At $c = 1$, DFlash-4B with 16-token blocks, a public drafter whose serving configuration we tuned and validated, gives 3.06 times plain decoding’s throughput, both with Triton attention. Every arm is stock or *exact up to rounding*, a class fixed after the first comparisons had been seen: each first divergence from a matched stock reference is an exact tie, one BF16 step or a near tie within two, and no output differs in length [E51]. On GSM8K the four arms tested (two MTP, DFlash block 8, replayed plain decoding) are within plain decoding’s noise, which excludes losses above about 3.3 points (derived), not smaller ones [E52].

Engine changes The *host-gap patches* remove blocking GPU reads from FlashInfer’s attention planning, host work that idles the MTP cycle (Figure 1), and raise MTP’s throughput 9.4–9.6% at $c = 1$ –8 [E53], still short of stock MTP with Triton attention, which plans nothing on the host [E50]. *Backbone GEMM routing* to the kernels a microbenchmark found fastest gains 3.4–3.5% for plain decoding at $c = 1$ but not for MTP [E55]. Removing per-request state traffic pays as the batch grows: *replayed GDN decoding*, which

Table 1. Engine changes (Section 4.5) against the tuned stock arm each modifies (group rows), flags otherwise identical; one session each, except the rows marked^c. Ratio: throughput y , or per-request rate x where marked, with over without the change; brackets: run-pair range, or 95% t interval^c. Outputs: greedy outputs against stock, same configuration; exactness is not established where not compared.

Change	c	Ratio (pairs)	Outputs
<i>MTP, three steps, FlashInfer attention</i> [E53,E58]			
Host-gap patches	1–8	1.094–1.096 (1.090–1.101)	token-identical ^a
Prefill delay, cap ^{c,e}	32–128	1.19–1.48 (1.17–1.50)	exact up to rounding
<i>Plain decoding</i> [E55,E50,E51]			
Backbone GEMM routing	1	1.035 (1.034–1.035)	exact up to rounding
Replayed GDN decoding	128	1.081 (one pair)	exact up to rounding
<i>DFlash, 16-token blocks</i> [E54,E57]			
Fold verify	1	0.968 (0.966–0.970)	bitwise ^b
	8–32	1.061–1.120 (1.053–1.134)	
Fold, FA4 ^{c,f}	1–4	x 1.084–1.093 (1.056–1.114)	exact up to rounding
on 8-token blocks	8–32	1.124–1.137 (1.108–1.166)	
<i>Certified head, per arm</i> ^c [E59]			
Plain decoding	1	1.025 (1.022–1.028)	identical ^d
MTP, Triton attention	1	1.046 (1.045–1.046)	identical ^d
DFlash, 16-token	1	1.012 (1.007–1.018)	identical ^d
	2–4	0.970–0.990 (0.958–0.996)	
DFlash, 8-token	4–8	0.975–0.988 (0.967–0.999)	check mode ^d

^aTokens, verify-step counts and draft histograms on 192 of 192 requests; log-probabilities not compared. ^bIdentical tokens and top-five log-probabilities, with matched pools and fixed batch composition, in a separate check. ^cThree sessions; geometric mean of the session ratios. ^dToken ids at $c = 1$ and every certified position in check mode (Appendix E.5). ^eAgainst MTP without it (best non-speculative arm: 1.18–1.32 at $c = 48$ –128); brackets: session range; outputs classed at $c = 64$ and 128. ^fNarrow verify tiles, FA4 target and drafter attention; from $c = 8$ the 8-token arm; x declared at $c \leq 4$; outputs classed at $c = 1$.

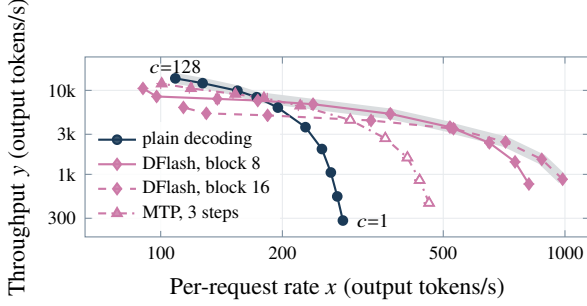


Figure 4. Stock serving frontier (measured; baseline, no certified head): four of nine tuned arms, client concurrency 1 (lower right) to 128 (64 for 16-token DFlash blocks). Points average three or four sessions (hollow: one); one standard deviation, below 2% of each mean, hides behind the markers [E50]. Grey: envelope of all nine arms’ points with three or more sessions.

writes the full recurrent state only every 16 tokens, gains 8.1% at $c = 128$ (one run pair) [E50], and *fold verify*, which verifies DFlash blocks without per-position state snapshots (SGLang’s fold-every-commit protocol), costs 3.2% at $c = 1$ and gains 6.1% at $c = 8$ [E54]; with routing and the head it fell below its declared range at $c = 2$ –8 (Appendix E.4). Up to $c = 32$ the fold with narrow verify tiles and FA4 attention, given a backported paged-KV fix, gain most together: on the envelope’s DFlash arms they raise the per-request rate 8.4–9.3% at $c = 1$ –4 and throughput 12.4–13.7% at 8–32, exact up to rounding as classed at $c = 1$ [E57]. From $c = 48$ speculation loses on admissions, since its requests finish and are prefilled one or two at a time; delaying prefills and capping each prefill batch at 16 requests lets MTP serve 1.18–1.32 times the best non-speculative arm at $c = 48$ –128, exact up to rounding (classed at $c = 64$ and 128), for 2.6–3.1 times plain decoding’s 99th-percentile time to first token [E58]. Two declared approximations miss the quality budget set in advance (GSM8K within 1 point; a logit probe) [E60]: FP16 recurrent state, which serves 1.16–1.17 times the best non-lossy arm timed with it at $c = 64$ –256 (three sessions) for 1.0–1.4 GSM8K points below plain decoding’s reference runs, within the exact arms’ own spread, and an int4 target with its int4 drafter (Kim et al., 2026), which fails the probe and is slower above $c = 1$.

The certified head, served Each tuned arm was served with and without the certified head (per-position fallback, conservative model) in three sessions; the head certifies calls of at most 64 positions; in a larger call the same CUDA graph runs the stock head, the head *gated off*. At $c = 1$ it raises throughput by 2.5% for plain decoding, 4.6% for MTP and 1.2% for 16-token DFlash, 28–71% of the saving the head-only times predict, and keeps gaining up to $c = 32$ for plain decoding and $c = 8$ for MTP (Table 1; Appendix E.5) [E59]. Both DFlash arms lose 1.0–3.0% above $c = 1$ where

the head is active (calls of up to 28–64 positions; 54–82% of verify calls fall back in check mode) and MTP 3.3% at 64 (64-position draft calls, 76% falling back), though plain decoding at $c = 32$ and 64 and MTP at $c = 8$ –32 run such calls without a detectable loss. The best arm at each concurrency (Figure 4’s envelope) gains only at $c = 1$ and loses 1.0–3.0% at $c = 2$ –8 (calls of up to 30–64 positions) and 0.6% at 128, gated off. Deploying it costs more: the certified graphs take 3.8–25.7 GB more memory, so both DFlash comparisons pinned the attention cache at 60,000 tokens; gated off, it still costs 0.6–1.2% (8-token DFlash’s intervals include 1); and a batch that requests log-probabilities, penalties or a grammar gets the stock verify. Its tokens equal stock’s at $c = 1$, in check mode at the tuned flags (0 of 592,433 certified positions differ, besides the engine check’s 196,734) and, for MTP, under identical batch evolution; closed-loop runs of plain decoding, MTP and 8-token DFlash diverge from stock at 1.03–1.21 times the rate between stock runs (only MTP’s 95% interval, 1.0002–1.22, excludes 1), and 16-token DFlash’s runs show 9 divergences against none (at $c = 8$, gated off). The one large divergence lies where stock BF16 decoding at batch 1, unlike Hugging Face’s, puts first a token 8.9 nats below FP32’s top, at one position past the model’s end of text [E41].

5 TRANSPORT AND WHY IT FAILS

Transport carries a drafter’s tile summaries to the target’s head input, bounding how far each score can move, so the verifier projects only the tiles whose bounds leave its decision open.

5.1 Transported Tile Certificates

Let h^d and h^t be aligned draft and target head inputs (each the final norm’s output) behind the same verified draft token, $\Delta = h^t - h^d$, $z_i^d = \langle w_i, h^d \rangle$ and $z_i^t = \langle w_i, h^t \rangle$. A partition of the vocabulary into tiles g carries, as checkpoint metadata, a centre μ_g and a radius $r_g \geq \max_{i \in g} \|w_i - \mu_g\|_2$. The draft pass emits $m_g^d = \max_{i \in g} z_i^d$ and $Z_g^d = \sum_{i \in g} e^{z_i^d}$.

Theorem 5.1 (Transported certificates). *In real arithmetic, with $a_g = \langle \mu_g, \Delta \rangle$ and $\epsilon_g = r_g \|\Delta\|_2$, every $i \in g$ satisfies $|z_i^t - z_i^d - a_g| \leq \epsilon_g$. Hence $\max_{i \in g} z_i^t \leq m_g^d + a_g + \epsilon_g =: U_g$ and $e^{a_g - \epsilon_g} Z_g^d \leq Z_g^t \leq e^{a_g + \epsilon_g} Z_g^d$.*

Proof. $z_i^t - z_i^d - a_g = \langle w_i - \mu_g, \Delta \rangle$, which Cauchy–Schwarz bounds by ϵ_g . Maximizing over $i \in g$ gives the first bound; exponentiating and summing gives the second. $\square$

A greedy verifier may skip tile g when $U_g < \tau$ for an exact target score τ , such as the draft token’s. *Bound families* replace the radius by Hölder, coordinate, grouped or low-rank radii, trading metadata for width, and *tilings* group the entries contiguously, at random or by k -means (Appendix C).

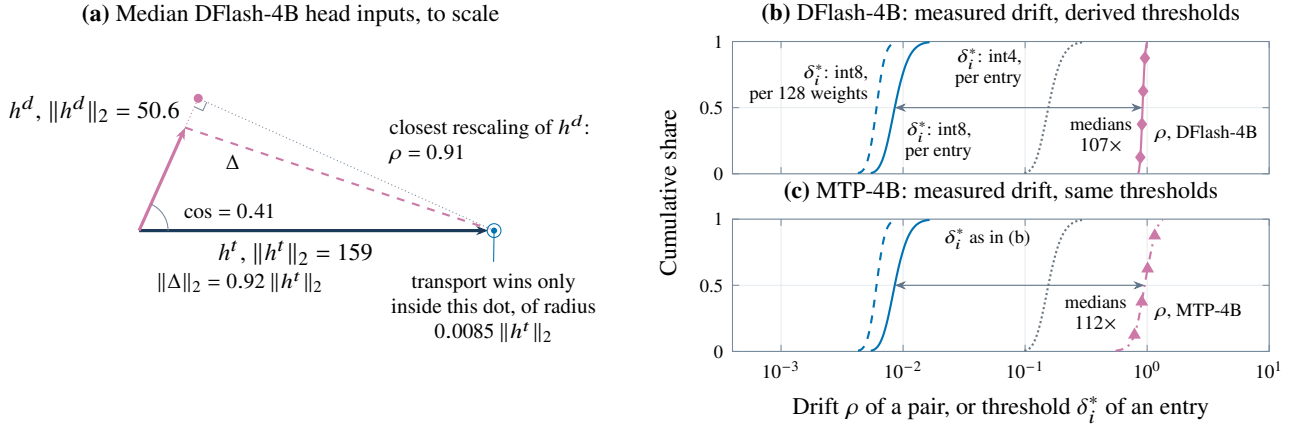


Figure 5. Why transport fails. (a) Median DFlash-4B head inputs, to scale: norms over the 4,020 whole-step pairs, cosine over 20,000 random aligned pairs. Transport’s bound beats the int8 copy’s only if h^d lies in the dot at the tip of h^t , of radius $\delta^*\|h^t\|_2$ for the median δ^* (Proposition 5.2). (b, c) Cumulative distributions of the measured drift ρ over 40,000 aligned held-out pairs of DFlash-4B (b) and MTP-4B (c), against the thresholds δ_i^* derived from the weights for every vocabulary entry; the arrows join the median threshold for int8 with one scale per entry to each drafter’s median drift (107 and 112 times apart) [E21,E22,E23,E25,E26].

5.2 What Transport Must Beat

The int8 copy’s error for entry i is $\langle e_i, h^t \rangle$, bounded by $\|e_i\|_2 \|h^t\|_2$; transport’s is $\langle w_i - \mu_g, \Delta \rangle$, bounded by $r_g \|\Delta\|_2$. Each bound is attained by some admissible vector, so neither can be tightened without more information (Proposition C.1). Both are a weight-side constant times an input-side norm, so the comparison reduces to one ratio, the *drift*

$$\rho = \|h^t - h^d\|_2 / \|h^t\|_2. \quad (2)$$

Proposition 5.2 (Crossover). *For every tile and centre, $r_g \geq \frac{1}{2} \text{diam}_2(g)$. Transport’s bound for entry $i \in g$ is narrower than the int8 copy’s, $r_g \|\Delta\|_2 < \|e_i\|_2 \|h^t\|_2$, if and only if $\rho < \delta_i^* := \|e_i\|_2 / r_g$.*

The weights alone fix δ^* . On this head, with 64-entry tiles of contiguous token indices and tile-mean centres, its median over all entries is 0.85% against int8 with one scale per entry [E21]. Draft and target head inputs must therefore agree to about one part in a hundred, which a drafter trained to predict tokens need not achieve. Transport must also beat the *static screen*, the tile bound $\langle \mu_g, h \rangle + r_g \|h\|_2$ at the target’s head input with no draft, and once the drift reaches 1 it is no tighter (Proposition C.4).

5.3 The Measured Drift

The drift is measured on 40,000 aligned pairs per drafter, drawn at random from the held-out prompts; certification (below) runs on the pairs from whole held-out capture steps (4,020 for DFlash-4B, 16,016 for MTP-4B). Its median is 0.917 for DFlash-4B [E22] and 0.954 for MTP-4B [E25], two orders of magnitude above the median δ^* (Figure 5b, c).

Why the drift is large Two properties of the DFlash drafter set its ρ (Figure 5a) [E26]. Its final RMSNorm has unit gain, so its head input always has norm $\sqrt{D} \approx 50.6$, about a third of the target’s median of 159, which alone gives $\rho \geq 0.68$. The angle is large as well: even the best rescaling of h^d leaves ρ near 0.91, since the drafter is trained to rank the right token first under Wh^d , not to reproduce h^t , and the head’s own metric does not hide the drift (Appendix C.5) [E22]. MTP-4B’s head input is slightly longer than the target’s, so its drift is mainly in direction [E24].

What certification achieves Over every bound family and tiling tested, and their elementwise minimum (Table 12), the best certified bound lets a greedy verifier skip at most 0.59% of the vocabulary for DFlash-4B and 0.64% for MTP-4B (0.65% bounding each entry from its own draft score, Appendix C.6), about as much as the static screen, which ignores the draft [E23,E24], while the int8 copy needs about 1.6 candidates per decision on the same DFlash-4B positions (Appendix A.8). A sampled draft fares no better: transported bounds alone leave its acceptance test undecided with mean probability $P_?$ of 0.92–1.00 (Appendix C.7).

Why certification gains so little Two measured facts explain it. First, the drift is far above the threshold, which is low because the int8 copy’s error is about 1% of a row’s norm while a tile’s radius exceeds that norm (Table 8). Second, a verifier sharing weight reads across a batch must read every tile any of its positions needs: over real verify batches, at least 99.4% of the vocabulary for every certified family. Even *oracle radii*, the realized per-tile deviations no certified bound can undercut, would skip most of the vocabulary for a single pair (Table 12) but need 97% for

random groups of 16 DFlash-4B pairs (Appendix C.7) [E23].

6 RELATED WORK

Certified low-precision heads Sparkpipe screens with an E4M3 copy whose per-group certificate covers CUDA-core FMA accumulation and recomputes the survivors exactly, for a model with the same vocabulary (sparkpipe contributors, 2026), and dgpp bounds logits from the high bit planes of an int8 head, with the quantized head as reference (Hawkins, 2026). A Metal implementation recomputes its candidates with a replica of its engine’s kernel (Layr Labs, 2026), and knlp reports argmax matches of 0.998–1.000 (Chamberlain, 2026). Du et al. show why the reference must be named: BF16 and FP16 greedy decoding diverge on 49–100% of prompts (Du et al., 2026).

Screening and sampling Bound-and-refine search is old in similarity search (Weber et al., 1998; Li et al., 2017; Teflioudi et al., 2015; Ram & Gray, 2012), where dense multiplication often wins end to end (Abuzaid et al., 2019). SVD-Softmax and HiRE recompute a fixed top set, uncertified (Shim et al., 2017; Samaga B L et al., 2024). CSV-Decode certifies top- k with cluster bounds (Liu et al., 2025), the static screen of Section 5.2; transport differs only in starting at the draft’s head input. FlashSampling fuses Gumbel-max sampling, exact in distribution, into the head over every tile (Ruiz et al., 2026), with tiling-dependent noise in its public kernel (FlashSampling contributors, 2026). Batch-invariant kernels make outputs batch-independent (He & Thinking Machines Lab, 2025; The SGLang Team, 2025) but change the kernel a certificate must match.

Speculative decoding Drafter-invariant decoding couples draft and target by shared Gumbel noise (Daliri et al., 2025; Rowan et al., 2025), which Section 3.3 uses to make sampled verification a race, free of the admission bias DSpark and D-cut describe (Cheng et al., 2026; Liu et al., 2026; Appendix B.5). EAGLE, native MTP and DFlash drafters project through a head at each draft step (Li et al., 2024; Gloeckle et al., 2024; Chen et al., 2026), and draft-vocabulary reduction trades acceptance for draft cost (Zhao et al., 2025; Goel et al., 2025; Williams et al., 2026). A certified head keeps the full vocabulary and serves the target.

What is and is not new Norm or quantization bounds, bound-then-refine search, certified greedy screening of an LM head, sampling from bounded scores and drafter-invariant verification are prior art. What is new is the contract with a closed-source kernel’s rounding (Section 1.1); transport of draft-side tile summaries is the one mechanism without a precedent we found, and it is a negative result.

7 LIMITATIONS

The error model is an assumption Agreement with R-stock rests on an assumed model of a closed-source kernel’s accumulation; neither of the two models covers BF16 split- K partial sums (Section 2.2). The crafted-input probes behind the Hopper model used products with one exact operand and reached neither subnormals nor other GPUs, so neither model is established for all inputs. The replayed positions cannot tell the two models apart (Section 4.3). The engine’s BF16-output kernel hides its accumulator, so the error measured on its FP32-output variant is indirect evidence [E12]. The int8 kernel is checked per compiled variant, not proved to compute the modelled arithmetic (Section 3.4).

Replay and serving Recomputation and fallback figures come from plain-decoding replay at batches of up to 16 positions, then consecutive ones, and the engine check from 64 prompts under the conservative model only. The served runs, on one greedy workload, did not time the in-graph fallbacks, so attributing the shortfall to them is a hypothesis.

Scope Transport was refuted for two drafters and the bound families and tilings of Table 12, in a replay that favours it by treating draft logits as exact. A drafter trained to reproduce the target’s head input is untested. The measurements concern one model, engine commit and GPU, with text-only prompts. The contract and proofs carry over to any head that rounds an FP32 accumulation to BF16 and breaks ties by index; the error model and the fallback shares must be re-established for each kernel and GPU.

8 CONCLUSION

A verifier does not need every logit, but the token it needs is the engine’s own, rounding and tie rule included, not the exact-arithmetic winner. On this engine an int8 copy of the head finds it with one or two exact recomputations per decision, under a named bound on the kernel’s error; served, it gains 1.2–4.6% throughput at one request with identical tokens and loses on DFlash above one request and on MTP at 64, where fallbacks become common, and 0.6–1.2% gated off. Bounds carried over from the drafters’ heads fail in every variant tested: their head inputs are far from the target’s. The design that makes the certified head safe, a fast path held to the engine’s output with its kernel as fallback, is not specific to the head. On this hybrid model the head is a small target (6% of a plain step at batch 128, the state kernel 42%; Appendix E.2); other changes gain more against stock (three sessions): the fold with FA4 attention adds 8.4–9.3% to DFlash’s per-request rate at $c = 1$ –4 and 12.4–13.7% to throughput at 8–32, and delayed, 16-capped prefill batches give MTP 1.18–1.32 times the best non-speculative arm’s throughput at 48–128, at a later first token.

REFERENCES

How each reference was checked is recorded in the committed citation audit [E61].

Abdelfattah, A., Dongarra, J., Fasi, M., Mikaitis, M., and Tisseur, F. Analysis of floating-point matrix multiplication computed via integer arithmetic. *arXiv:2506.11277v3*, 2025. URL <https://arxiv.org/abs/2506.11277v3>.

Abuzaid, F., Sethi, G., Bailis, P., and Zaharia, M. To index or not to index: Optimizing exact maximum inner product search. In *2019 IEEE 35th International Conference on Data Engineering (ICDE)*, pp. 1250–1261, 2019. doi: 10.1109/ICDE.2019.00114.

Ben Shoham, O. Balancing coverage and draft latency in vocabulary trimming for faster speculative decoding. *arXiv:2603.05210v1*, 2026. URL <https://arxiv.org/abs/2603.05210v1>.

Chamberlain, L. Certified LM-head decode: read a quarter of the output head, losslessly. *mcgrof/knlp, docs/lm-head-decode.md*, commit 1dd92463092d, June 2026. URL <https://github.com/mcgrof/knlp/blob/1dd92463092d8959ba5d00cc7cf95bb8e34904f0/docs/lm-head-decode.md>.

Chen, C., Borgeaud, S., Irving, G., Lespiau, J.-B., Sifre, L., and Jumper, J. Accelerating large language model decoding with speculative sampling. *arXiv:2302.01318v1*, 2023. URL <https://arxiv.org/abs/2302.01318v1>.

Chen, J., Liang, Y., and Liu, Z. DFlash: Block diffusion for flash speculative decoding. In *International Conference on Machine Learning (ICML 2026)*, 2026. URL <https://arxiv.org/abs/2602.06036v2>.

Cheng, X., Yu, X., Shao, C., Li, J., Xiong, Y., Qian, Y., Zhu, J., Ma, S., Zhang, X., Ye, J., Chen, Q., Deng, C., Yu, J., Dai, D., Zhang, Z., Wei, Y., Tan, Y., Yang, W., Xu, R., Wu, Y., Xu, Z., Wang, X., Chen, M., Tian, R., Bi, X., Hao, Z., Chen, S., Cao, H., Zhang, W., Xu, A., Zhang, H., Zhao, D., and Liang, W. DSpark: Confidence-scheduled speculative decoding with semi-autoregressive generation. *arXiv:2607.05147v1*, 2026. URL <https://arxiv.org/abs/2607.05147v1>.

Daliri, M., Musco, C., and Suresh, A. T. Coupling without communication and drafter-invariant speculative decoding. In *2025 IEEE International Symposium on Information Theory (ISIT)*, pp. 1–6, 2025. doi: 10.1109/ISIT63088.2025.11195663.

DeepSeek-AI. DeepSeek-V3 technical report. *arXiv:2412.19437v2*, 2024. URL <https://arxiv.org/abs/2412.19437v2>.

Du, G., Khan, A. N., Zhou, R., Liu, X., Chakrabarti, D., Sua, F., and Li, X. Greedy decoding is not precision-invariant: Cross-precision output divergence in LLM inference. *Transactions on Machine Learning Research*, 2026. URL <https://openreview.net/forum?id=QDOKyg7a5e>.

Fasi, M., Higham, N. J., Mikaitis, M., and Pranesh, S. Numerical behavior of NVIDIA tensor cores. *PeerJ Computer Science*, 7: e330, 2021. doi: 10.7717/peerj-cs.330.

FlashSampling contributors. Fused matrix multiplication and Gumbel-max sampling kernel (*src/fused_mm_sampling/core.py*). Source code, commit 6e376af5184d9d3a572755b5260c9c4368562828, 2026. URL https://github.com/FlashSampling/FlashSampling/blob/6e376af5184d9d3a572755b5260c9c4368562828/src/fused_mm_sampling/core.py.

Frieder, O., Mele, I., Muntean, C. I., Nardini, F. M., Perego, R., and Tonello, N. Caching historical embeddings in conversational search. *ACM Transactions on the Web*, 18(4): 1–19, 2024. doi: 10.1145/3578519.

Gloeckle, F., Idrissi, B. Y., Rozière, B., Lopez-Paz, D., and Synnaeve, G. Better & faster large language models via multi-token prediction. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pp. 15706–15734. PMLR, 2024. URL <https://proceedings.mlr.press/v235/gloeckle24a.html>.

Goel, R., Agrawal, S., Gagrani, M., Park, J., Zao, Y., Zhang, H., Liu, T., Yang, Y., Yuan, X., Lu, J., Lott, C., and Lee, M. VOCABTRIM: Vocabulary pruning for efficient speculative decoding in LLMs. *arXiv:2506.22694v2*, 2025. URL <https://arxiv.org/abs/2506.22694v2>. ICML 2025 Workshop on Efficient Systems for Foundation Models.

Hawkins, S. Head bit-plane exact argmax: read 25–33 % fewer head bytes on every draft/greedy head read, all families. GitHub issue 69, HawkBearPig/dgpp; implementation in *src/kernels/packq_head.cu* at commit a23d8b1, September 2026. URL <https://github.com/HawkBearPig/dgpp/issues/69>.

He, H. and Thinking Machines Lab. Defeating nondeterminism in LLM inference. Thinking Machines Lab: Connectionism, September 2025. URL <https://thinkingmachines.ai/blog/defeating-nondeterminism-in-llm-inference/>.

Higham, N. J. *Accuracy and Stability of Numerical Algorithms*. Society for Industrial and Applied Mathematics, Philadelphia, PA, second edition, 2002. doi: 10.1137/1.9780898718027.

Khattak, F. A. and Mikaitis, M. Accurate models of NVIDIA tensor cores. *ACM Transactions on Architecture and Code Optimization*, 23(3):1–25, 2026. doi: 10.1145/3830409.

Kim, J., Lee, J., and Kim, B.-K. Quantize the target, quantize the drafter: Efficient inference with Qwen3.5-4B. *arXiv:2607.04244v2*, 2026. URL <https://arxiv.org/abs/2607.04244v2>. Submission to the Efficient Qwen Competition, ICML 2026 Workshop on AdaptFM.

Kocabay, Ş. T., Akkuş, T. R., and Yuksel, K. A. ARCHead: Activation-metric residual correction for large language model output heads. *arXiv:2608.02703v1*, 2026. URL <https://arxiv.org/abs/2608.02703v1>.

Kool, W., van Hoof, H., and Welling, M. Stochastic beams and where to find them: The Gumbel-Top- k trick for sampling sequences without replacement. In *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, pp. 3499–3508. PMLR, 2019. URL <https://proceedings.mlr.press/v97/kool19a.html>.

- Layr Labs. Laguna LM-head prune (Sources/MLXFastModel/LagunaLMHeadPrune.swift). Source code, Layr-Labs/mlxfast-challenge, commit 4f4f68959cfe, July 2026. URL <https://github.com/Layr-Labs/mlxfast-challenge/blob/4f4f68959cfe94e31ef6cc3b22c9df468120b51/Sources/MLXFastModel/LagunaLMHeadPrune.swift>.
- Leviathan, Y., Kalman, M., and Matias, Y. Fast inference from transformers via speculative decoding. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pp. 19274–19286. PMLR, 2023. URL <https://proceedings.mlr.press/v202/leviathan23a.html>.
- Li, H., Chan, T. N., Yiu, M. L., and Mamoulis, N. FEXIPRO: Fast and exact inner product retrieval in recommender systems. In *Proceedings of the 2017 ACM International Conference on Management of Data (SIGMOD)*, pp. 835–850, 2017. doi: 10.1145/3035918.3064009.
- Li, Y., Wei, F., Zhang, C., and Zhang, H. EAGLE: Speculative sampling requires rethinking feature uncertainty. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pp. 28935–28948. PMLR, 2024. URL <https://proceedings.mlr.press/v235/li24bt.html>.
- Liu, D., Wang, S., Yu, Y., Wang, H., and Lengerich, B. CSV-Decode: Certifiable sub-vocabulary decoding for efficient large language model inference. arXiv:2511.21702v2, 2025. URL <https://arxiv.org/abs/2511.21702v2>.
- Liu, T., Shen, Y., Cen, R., Shi, J., Zhang, J., Qin, G., Liu, H., Liu, S., Yu, G., and Zhu, J. D-cut: Adaptive verification depth pruning for batched speculative decoding. arXiv:2607.14647v1, 2026. URL <https://arxiv.org/abs/2607.14647v1>.
- Maddison, C. J., Tarlow, D., and Minka, T. A* sampling. In *Advances in Neural Information Processing Systems 27 (NIPS 2014)*, pp. 3086–3094, 2014. URL https://proceedings.neurips.cc/paper_files/paper/2014/hash/937debc749f041eb5700df7211ac795c-Abstract.html.
- Musmann, S. and Ermon, S. Learning and inference via maximum inner product search. In *Proceedings of the 33rd International Conference on Machine Learning*, volume 48 of *Proceedings of Machine Learning Research*, pp. 2587–2596. PMLR, 2016. URL <https://proceedings.mlr.press/v48/musmann16.html>.
- Musmann, S., Levy, D., and Ermon, S. Fast amortized inference and learning in log-linear models with randomly perturbed nearest neighbor search. In *Proceedings of the Thirty-Third Conference on Uncertainty in Artificial Intelligence (UAI 2017)*, 2017. URL <https://arxiv.org/abs/1707.03372>.
- NVIDIA. Nsight Systems user guide. Nsight Systems 2025.3 documentation, 2025. URL <https://docs.nvidia.com/nsight-systems/2025.3/UserGuide/index.html>.
- NVIDIA. Floating point and IEEE 754. CUDA 13.4 documentation, 2026. URL <https://docs.nvidia.com/cuda/floating-point/index.html>. Accessed 30 September 2026.
- Papandreou, G. and Yuille, A. L. Perturb-and-MAP random fields: Using discrete optimization to learn and sample from energy models. In *2011 International Conference on Computer Vision (ICCV)*, pp. 193–200, 2011. doi: 10.1109/ICCV.2011.6126242.
- Ponnusamy, P., Sahni, S., Wang, J., and Dao, T. SonicSampler: Unified tile-aware kernels for LLM sampling and speculative verification. arXiv:2607.20475v1, 2026. URL <https://arxiv.org/abs/2607.20475v1>.
- PyTorch contributors. Numerical accuracy. PyTorch 2.14 documentation, 2026. URL https://docs.pytorch.org/docs/2.14/notes/numerical_accuracy.html.
- Qwen. Qwen3.5-4B model card. Hugging Face, revision 851bf6e806efd8d0a36b00ddf55e13ccb7b8cd0a, 2026a. URL <https://huggingface.co/Qwen/Qwen3.5-4B/tree/851bf6e806efd8d0a36b00ddf55e13ccb7b8cd0a>.
- Qwen. Qwen3.5-4B model configuration (config.json). Hugging Face, revision 851bf6e806efd8d0a36b00ddf55e13ccb7b8cd0a, 2026b. URL <https://huggingface.co/Qwen/Qwen3.5-4B/blob/851bf6e806efd8d0a36b00ddf55e13ccb7b8cd0a/config.json>.
- qwen38-challenge_senpai contributors. The verify readout streams 715 MB per round to answer a two-element question. Prove most of the vocabulary cannot win, and skip it. GitHub pull request 142 (experiment E142), morganmcg1/qwen38-challenge_senpai, August 2026. URL https://github.com/morganmcg1/qwen38-challenge_senpai/pull/142.
- Ram, P. and Gray, A. G. Maximum inner-product search using cone trees. In *Proceedings of the 18th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 931–939, 2012. doi: 10.1145/2339530.2339677.
- Rowan, J., Phan, B., and Khisti, A. List-level distribution coupling with applications to speculative decoding and lossy compression. In *Advances in Neural Information Processing Systems 38 (NeurIPS 2025)*, 2025. URL <https://neurips.cc/virtual/2025/poster/118977>.
- Ruiz, T., Qin, Z., Zhang, Y., Shen, X., Zhong, Y., and Wang, M. FlashSampling: Fast and memory-efficient exact sampling. arXiv:2603.15854v3, 2026. URL <https://arxiv.org/abs/2603.15854v3>.
- Rump, S. M. Verification methods: Rigorous results using floating-point arithmetic. *Acta Numerica*, 19:287–449, 2010. doi: 10.1017/S096249291000005X.
- Samaga B L, Y., Yerram, V., You, C., Bhojanapalli, S., Kumar, S., Jain, P., and Netrapalli, P. HiRE: High recall approximate top-k estimation for efficient LLM inference. arXiv:2402.09360v1, 2024. URL <https://arxiv.org/abs/2402.09360v1>.
- SGLang contributors. Sampler (python/sglang/srt/layers/sampler.py). Source code, commit bd66ce343e4f6e2f2b75d7e820fe4d0718a8d824, 2026. URL <https://github.com/sgl-project/sglang/blob/bd66ce343e4f6e2f2b75d7e820fe4d0718a8d824/python/sglang/srt/layers/sampler.py>.
- Shim, K., Lee, M., Choi, I., Boo, Y., and Sung, W. SVD-Softmax: Fast softmax approximation on large vocabulary neural networks. In *Advances in Neural Information Processing Systems 30 (NIPS 2017)*, 2017. URL https://proceedings.neurips.cc/paper_files/paper/2017/hash/4e2a6330465c8ffcaa696a5a16639176-Abstract.html.

- sparkpipe contributors. Certified screened LM head (SparkLmHeadCertifiedFp8QuantizeKernel, SparkLmHeadCertifiedFp8ScoreKernel). Source code, model-families/common/include/sparkpipe/spark_lm_kernels.cuh, commit 84efd5b63d8d; pull request 744, August 2026. URL <https://github.com/sparkpipe/sparkpipe/pull/744>.
- Sun, Z., Suresh, A. T., Ro, J. H., Beirami, A., Jain, H., and Yu, F. SpecTr: Fast speculative decoding via optimal transport. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*, pp. 30222–30242, 2023. URL https://proceedings.neurips.cc/paper_files/paper/2023/hash/6034a661584af6c28fd97a6f23e56c0a-Abstract-Conference.html.
- Sun, Z., Mendlovic, U., Leviathan, Y., Aharoni, A., Ro, J. H., Beirami, A., and Suresh, A. T. Block verification accelerates speculative decoding. In *International Conference on Learning Representations (ICLR 2025)*, 2025. URL <https://arxiv.org/abs/2403.10444v3>.
- Tao, C., Liu, Q., Dou, L., Muennighoff, N., Wan, Z., Luo, P., Lin, M., and Wong, N. Scaling laws with vocabulary: Larger models deserve larger vocabularies. In *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*, pp. 114147–114179, 2024. URL https://proceedings.neurips.cc/paper_files/paper/2024/hash/cf5a019ae9c11b4be88213ce3f85d85c-Abstract-Conference.html.
- Teflioudi, C., Gemulla, R., and Mykytiuk, O. LEMP: Fast retrieval of large entries in a matrix product. In *Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data*, pp. 107–122, 2015. doi: 10.1145/2723372.2747647.
- The SGLang Team. Towards deterministic inference in SGLang and reproducible RL training. LMSYS Org blog, September 2025. URL <https://www.lmsys.org/blog/2025-09-22-sglang-deterministic/>.
- Wang, J., Wonka, P., and Ye, J. Lasso screening rules via dual polytope projection. *Journal of Machine Learning Research*, 16:1063–1101, 2015. URL <https://jmlr.org/papers/v16/wang15a.html>.
- Weber, R., Schek, H.-J., and Blott, S. A quantitative analysis and performance study for similarity-search methods in high-dimensional spaces. In *Proceedings of the 24th International Conference on Very Large Data Bases (VLDB 1998)*, pp. 194–205, 1998. URL <https://www.vldb.org/conf/1998/p194.pdf>.
- Williams, M., Kwon, Y. D., Li, R., Kouris, A., and Venieris, S. I. Speculative decoding with a speculative vocabulary. In *Findings of the Association for Computational Linguistics: ACL 2026*, pp. 40240–40254, 2026. doi: 10.18653/v1/2026.findings-acl.2000.
- Xing, S., Ye, Z., Hou, B., Ceze, L., and Chen, T. Sorting-free GPU kernels for LLM sampling. FlashInfer blog, March 2025. URL <https://flashinfer.ai/2025/03/10/sampling.html>.
- Yang, S., Kautz, J., and Hatamizadeh, A. Gated delta networks: Improving Mamba2 with delta rule. In *International Conference on Learning Representations (ICLR 2025)*, 2025. URL <https://openreview.net/forum?id=r8H7xhYPwz>.
- Ye, Z., Chen, L., Lai, R., Lin, W., Zhang, Y., Wang, S., Chen, T., Kasikci, B., Grover, V., Krishnamurthy, A., and Ceze, L. FlashInfer: Efficient and customizable attention engine for LLM inference serving. In *Proceedings of the Eighth Conference on Machine Learning and Systems (MLSys 2025)*, 2025. URL https://proceedings.mlsys.org/paper_files/paper/2025/hash/dbf02b21d77409a2db30e56866a8ab3a-Abstract-Conference.html.
- Yuan, J., Li, H., Ding, X., Xie, W., Li, Y.-J., Zhao, W., Wan, K., Shi, J., Hu, X., and Liu, Z. Understanding and mitigating numerical sources of nondeterminism in LLM inference. In *Advances in Neural Information Processing Systems 38 (NeurIPS 2025)*, 2025. URL <https://neurips.cc/virtual/2025/poster/118169>. Version 1 of the arXiv preprint was titled “Give Me FP32 or Give Me Death? Challenges and Solutions for Reproducible Reasoning”.
- z-lab. Qwen3.5-4B-DFlash model card. Hugging Face, revision 9a1996ccf887b79ab3af4fcbf8c1d1f4b5658bcf, 2026. URL <https://huggingface.co/z-lab/Qwen3.5-4B-DFlash/tree/9a1996ccf887b79ab3af4fcbf8c1d1f4b5658bcf>.
- Zhang, C. Exact adaptive hybrid retrieval without fixed Top-L cutoffs. arXiv:2608.07152v1, 2026. URL <https://arxiv.org/abs/2608.07152v1>.
- Zhao, W., Pan, T., Han, X., Zhang, Y., Sun, A., Huang, Y., Zhang, K., Zhao, W., Li, Y., Zhou, J., Zhou, H., Wang, J., Liu, Z., and Sun, M. FR-Spec: Accelerating large-vocabulary language models via frequency-ranked speculative sampling. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 3909–3921, 2025. doi: 10.18653/v1/2025.acl-long.198.
- Zheng, L., Yin, L., Xie, Z., Sun, C., Huang, J., Yu, C. H., Cao, S., Kozyrakis, C., Stoica, I., Gonzalez, J. E., Barrett, C., and Sheng, Y. SGLang: Efficient execution of structured language model programs. In *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*, pp. 62557–62583, 2024. URL https://proceedings.neurips.cc/paper_files/paper/2024/hash/724be4472168f31ba1c9ac630f15dec8-Abstract-Conference.html.

A THE CERTIFIED HEAD: ASSUMPTIONS, PROOFS AND CHECKS

This appendix collects what the main text’s contract and certified head rest on. It lists the numerical assumptions in full (Appendix A.1) and proves the results of Sections 2.2 and 3 whose proofs are not in the main text. It also states and proves the general form of the copy’s error bound (Appendix A.4) and the refinement ladder (Appendix A.7), with the secondary results they need: the constants of the two error models, the integer pass, and the computed norms and outward rounding that turn a real error bound into an interval of computed values. It then bounds and measures how many vocabulary entries a decision re-scores (Appendix A.8), says how the GPU kernel is checked (Appendix A.9), and separates what is machine-checked from what is tested and what is proved only on paper (Appendix A.10).

A.1 Numerical assumptions in full

Table 2 lists every rounding, accumulation, tie-breaking and error-model assumption with its status and the results that depend on it; Table 3 summarizes it.

A.2 The BF16 separation lemma and the gap condition

Proof of Lemma 2.2. Rounding is nondecreasing, so if $\text{rn}_B(y_a) \leq \text{rn}_B(y_b)$ while $y_a > y_b$, both values round to the same v and lie in its preimage. When $|v|$ is not a power of two, the preimage has length $\text{ulp}_B(v)$ and both values lie in the binade of v . When $|v| = 2^e$ the preimage is $[v - \text{ulp}_B(v)/4, v + \text{ulp}_B(v)/2]$ for $v > 0$ (symmetrically for $v < 0$): two values on one side of v are within half the spacing of their binade, and two values on both sides are within $\frac{3}{4} \text{ulp}_B(v) \leq \text{ulp}_B(y_a)$. At $v = 0$ the preimage has the subnormal spacing. In every case $y_a - y_b \leq \text{ulp}_B(\max(|y_a|, |y_b|))$, contradicting the hypothesis. $\square$

Proof of Theorem 2.3. For every $b \neq a$, $\hat{y}_a - \hat{y}_b \geq z_a - z_b - G_a - G_b > \text{ulp}_B(\max(|z_a|, |z_b|) + \max(G_a, G_b)) \geq \text{ulp}_B(\max(|\hat{y}_a|, |\hat{y}_b|))$, because $|\hat{y}_i| \leq |z_i| + G_i$ and ulp_B is nondecreasing in the magnitude. Lemma 2.2 gives $L_a > L_b$, so a is the only maximal index and R-stock returns a . $\square$

A.3 Accumulation trees and the constants of the error models

The bound below covers every accumulation order the error models need; Section 3.1 applies it to the low-precision pass.

Lemma A.1 (Accumulation trees). *Let $s = \sum_\ell p_\ell$ be evaluated by a tree whose internal nodes v return $\hat{s}_v$ with $|\hat{s}_v - \sum_c \hat{s}_c| \leq u_v \sum_c |\hat{s}_c| + \eta_v$ over their children c . With d the largest number of internal nodes on a leaf-to-root path, N the number of internal nodes, $u \geq \max u_v$, $\eta \geq \max \eta_v$ and $du < 1$: $|\hat{s} - s| \leq \gamma_d \sum_\ell |p_\ell| + N\eta(1 + \gamma_d)$, where $\gamma_d = (1 + u)^d - 1 \leq du/(1 - du)$.*

Proof of Lemma A.1. For a node v and a leaf ℓ below it, let $P_{v\ell} = \prod (1 + u_x)$ over the internal nodes x on the path from ℓ up to v , and for an internal node w below v let P'_{vw} be the same product over the nodes strictly above w up to v , with $P'_{vv} = 1$. By induction on height,

$$|\hat{s}_v| \leq \sum_\ell |p_\ell| P_{v\ell} + \eta \sum_w P'_{vw}, \quad |\hat{s}_v - s_v| \leq \sum_\ell |p_\ell| (P_{v\ell} - 1) + \eta \sum_w P'_{vw}.$$

Leaves satisfy both with equality. For an internal node, $|\hat{s}_v| \leq (1 + u_v) \sum_c |\hat{s}_c| + \eta$ and $|\hat{s}_v - s_v| \leq \sum_c |\hat{s}_c - s_c| + u_v \sum_c |\hat{s}_c| + \eta$; substituting the hypotheses for the children multiplies every P and P' by $(1 + u_v)$ and adds $P'_{vv} = 1$. Finally $P_{v\ell} \leq (1 + u)^d$, $P'_{vw} \leq (1 + u)^{d-1}$, and there are N internal nodes. $\square$

Depths and node units The tree’s shape is all that distinguishes CUDA-core, tensor-core and split- K accumulation. An IEEE rounded addition has $u_v = u$, since $|a + b| \leq |a| + |b|$ (Higham, 2002). Sequential accumulation from zero has depth $d = n$, pairwise summation $\lceil \log_2 n \rceil$, blocks of B summed as a tree and then accumulated $\lceil n/B \rceil + \lceil \log_2 B \rceil$, and split- K adds the number of partials. A multi-term adder that aligns $k + 1$ addends to the largest, truncates each to F fractional bits below its leading bit, adds exactly and rounds once has $u_v = (k + 1)2^{-F} + 2^{1-F}$ with truncation, and the accumulation path has $\lceil n/k \rceil$ such nodes. Tensor cores behave this way rather than as IEEE adders (Fasi et al., 2021). A node that multiplies its only child by a constant and rounds adds one level of depth.

Table 2. Numerical assumptions and the results that depend on them. “Code reading”: read in SGLang’s source at the pinned commit bd66ce343e4f (Appendix F.3) with PyTorch 2.13; $D = 2,560$.

Assumption	Status	Results that depend on it
Head weights and inputs are BF16 and are normal or zero. Every product $w_{ij}h_j$, and every product of an int8 code with a BF16 input, is exact in FP32 unless it underflows; underflow and flush to zero of products and partial sums cost less than 2^{-126} each and are charged to the absolute floor η .	Exact in the normal range; normal operands assumed.	Thms. 2.3, 3.1, A.2; Prop. 2.4
The stock head rounds its FP32 accumulator to BF16 (nearest, ties to even) and widens it exactly; <code>torch.argmax</code> returns the smallest maximal index in greedy decoding and verification.	Code reading.	Def. 2.1; Lem. 2.2; Prop. 2.4
Stock accumulation error $ \hat{y}_i - z_i \leq \gamma_{\text{stock}} \sum_j w_{ij}h_j + \eta$ at every served shape: $\gamma_{\text{stock}} = 6.11 \times 10^{-4}$ (conservative model: $2D$ roundings at 2^{-23} , any order, which covers multi-term adders that keep at least 23 fractional bits per block of 8 to 32 terms) or $\gamma_{\text{stock}} = 1.193 \times 10^{-4}$ (Hopper model: Khattak and Mikaitis’s model of Hopper’s BF16 tensor cores, measured on the warp-level <code>mma</code> path, plus FP32 split- K); $\eta = 2^{-113}$; no split- K partials stored in BF16; crafted-input probes of the stock kernels match the Hopper model (Appendix A.3) [E13]. The replay omitted η , which can change a decision only if an endpoint lies within 2^{-112} of a BF16 rounding boundary or a gap within 2^{-112} of its threshold.	Assumed; consistent with all replayed positions and with the FP32-output kernel’s observed error, 58 times inside the Hopper model [E12]; implemented floor 2^{-100} [E9].	Certification against R-stock; Table 5
A position’s stock BF16 logits do not depend on the other positions of the batch or on which vocabulary entries are present.	Measured on this build at 20 batch sizes; the part about which entries are present is re-checked at start-up, at the deployed batch sizes [E14].	Per-position fallback only
Comparisons are made at the same batch shape and serving configuration.	Required (Section 4.1).	Every agreement claim
The low-precision pass accumulates in FP32 on tensor cores within $\gamma_{\text{copy}} = 6.11 \times 10^{-4}$, the conservative model whichever model is assumed for the stock kernel, which holds if its adders keep at least 23 fractional bits per block of 8 to 32 terms, or exactly in int32 for int8 codes times int8 activation codes.	Model assumed; integer pass exact.	Thms. 3.1, A.2; Table 4
The low-precision kernel computes the modelled arithmetic: the scaled products of codes and inputs, accumulated in FP32 within γ_{copy} or exactly in int32, and the error-bound epilogue as analysed.	Required, not proved; checked per compiled variant at start-up and by runtime probes [E9]. Int8 TMA tiles with a 64-byte box gave wrong products under Triton 3.7.1 and are refused [E10].	Tokens certified by a kernel, not by the replay; its runtime
Re-scoring sums the exact BF16 products of each candidate in FP64 on CUDA cores under IEEE rounding, with error at most $\gamma_D(2^{-53}) \sum_j w_{ij}h_j < 3 \times 10^{-13} \sum_j w_{ij}h_j $, which widens its accumulator enclosure.	Implemented and tested [E9].	Decision by Prop. 2.4
Norms of h are rounded up (ℓ_1 reduction depth at most 2^{13}); interval endpoints are rounded outward; offline constants are exact and stored rounded up.	Proved (Appendix A).	Every computed enclosure
Untruncated seeded noise is a function of (seed, token position in the sequence, token) through a 32-bit hash, evaluated in FP64 after an FP32 softmax and logarithm; statements in law idealize it as i.i.d. continuous Gumbel. The seeded sampler exists only in deterministic-inference mode, whose head is SGLang’s batch-invariant <code>mm</code> , here DeepGEMM’s deterministic BF16 GEMM (one FP32 accumulator, no split- K , round-to-nearest output): the conservative model covers it; the Hopper model is unchecked for it. Undecided sampled positions run the stock sampler.	Code reading; the chain’s rounding bounded by CUDA’s documented <code>expf</code> and <code>logf</code> accuracy; matched on 60,000 replayed positions [E19].	Thms. B.2, B.3; sampling contract
Replay: real logits are evaluated in FP64 from the stored operands (error below $D 2^{-52} \sum_j w_{ij}h_j $); draft logits are exact, ignoring the BF16 rounding a served draft pass adds, which favours transport.	Replay convention.	Tables 4–12; Figs. 3, 5
Transport is in real arithmetic with one head for both passes; δ^* uses Euclidean error bounds, 64-entry tiles of contiguous token indices and tile-mean centres.	Stated; other geometries in Table 12.	Thm. 5.1; Prop. 5.2; Fig. 5

Table 3. Numerical assumptions, in short (in full: Table 2). “Code”: read from SGLang’s and PyTorch’s code at the versions of Section 4.1.

Assumption	Status
BF16 operands are normal or zero, so products are exact in FP32 (underflow is charged to η)	Exact for normal operands
The stock head rounds its FP32 accumulator to BF16 (nearest, ties to even); <code>torch.argmax</code> returns the first maximal index	Code
The stock kernel’s error is at most $\gamma_{\text{stock}} \sum_j w_{ij} h_j + \eta$ at every served shape, under the conservative model or the Hopper model; no split- K partial sums in BF16	Assumed; consistent with every replayed position, the FP32-output variant and crafted-input probes (Section 2.2) [E12,E13]
The int8 pass’s own accumulation error is at most $\gamma_{\text{copy}} = 6.11 \times 10^{-4}$ times $\sum_j \tilde{w}_{ij} h_j $, and the kernel computes the modelled arithmetic	Assumed; checked per compiled variant at start-up and by runtime probes, not proved (Section 3.4) [E9,E10]
Agreement is checked at the same batch shape and configuration	Required (Section 4.1)
Per-position fallback only: a position’s stock logits do not depend on the rest of the batch or on which entries are present	Measured at 20 batch sizes; entry subsets re-checked at start-up [E14]
Seeded sampling (Section 3.3): noise is a function of (seed, token position in the sequence, token); no top- k or top- p ; the deterministic mode’s head kernel is covered by the conservative model only	Code; matched on replayed positions [E19]
The offline replay evaluates real logits in FP64; transport is analysed in real arithmetic	How the replay computes

The constants used in this paper The stock kernel’s constant is γ_{stock} , set by one of the two models below; the int8 pass’s own accumulation is bounded by γ_{copy} , the conservative constant, whichever model is assumed for the stock kernel. Both models of the stock kernel add the absolute floor $\eta = 2^{-113}$, which covers underflow and flush to zero of products and partial sums, at most $2D \cdot 2^{-126}(1 + \gamma_{\text{stock}})$. Each model below except the IEEE one is an assumption about hardware that no vendor documents; Table 2 lists where each is used. The values are for $D = 2560$.

- *The conservative model*, $\gamma_{\text{stock}} = \gamma_{\text{copy}} = 6.11 \times 10^{-4}$. At most $2D$ roundings, each of relative size 2^{-23} of the magnitudes it combines, in any order: $\gamma_{2D}(2^{-23}) = 6.107 \times 10^{-4}$. It covers any reduction tree, split- K with FP32 partials, and multi-term adders over blocks of 8 to 32 terms that keep at least 23 fractional bits and round to nearest or truncate; with the adder unit u_v above, 22 fractional bits are not covered. The committed replay analyses use 6.11×10^{-4} , the largest value over the exact reference’s fused-adder models with blocks of 2 to 64 products, rounding to nearest or toward zero, plus one scale multiply [E2,E6]; the two agree to the precision the paper quotes.
- *The Hopper model*, $\gamma_{\text{stock}} = 1.193 \times 10^{-4}$. Khattak & Mikaitis (2026) model Hopper’s BF16 tensor cores with FP32 output, from measurements of the warp-level mma path (SASS HMMMA), as a $k = 16$ adder with $F = 25$ and truncation, so $u_v = 17 \cdot 2^{-25} + 2^{-23} = 6.26 \times 10^{-7}$ over $D/16 = 160$ nodes, and $(1 + u_v)^{160} - 1 \approx 1.001 \times 10^{-4}$, which the conservative model covers about sixfold for any reduction order. The two stock head kernels whose SASS could be read ($M = 1$ and 32) issue the warpgroup `wgmma` instead (only `HGMMMA`); the other twelve could not be read statically. On crafted inputs whose exact sums are known, Triton’s `wgmma` and all fourteen stock head kernels accumulate the same way: blocks of 16 products with the FP32 accumulator inside the aligned sum, 25 fractional bits below the largest addend, dropped bits truncated toward zero, and the sum truncated to FP32. Inputs off the rounding ties rule out rounding to nearest, and the stock kernels’ BF16 epilogue rounds to nearest even [E13]. The operands were powers of two and short sums of them, with one operand of each product equal to 1, on one GH200 (driver 570.195.03, CUDA 13.0, PyTorch 2.13’s cuBLAS); full-significand products, subnormals, offsets beyond 128 terms and other GPUs or libraries were not probed, so this is a measurement on probes, not a vendor contract. An allowance of up to 160 further truncating FP32 additions for split- K partials gives $(1 + u_v)^{160}(1 + 2^{-23})^{160} - 1 = 1.1922 \times 10^{-4}$, which the paper quotes rounded up as 1.193×10^{-4} . The replays of the head and of the stock divergences compute this value in exact rational arithmetic and round it up to the next double [E6,E33].
- *FP8 tensor cores*. The same published model gives FP8 `wgmma` only 13 fractional bits in 32-term blocks, consistent with the accumulation limits reported for FP8 training (DeepSeek-AI, 2024); an FP8 tensor-core pass needs periodic promotion to FP32 and its own analysis, and none is used here.
- *IEEE FP32 on CUDA cores*. A balanced tree over blocks of 128 products, a sequential sum across the 20 blocks and one scale multiply give depth 28 with $u = 2^{-24}$, so $\gamma = 1.67 \times 10^{-6}$. This model does not describe the stock tensor-core

GEMM; the paper uses it only as a sensitivity value of γ_{stock} (Table 5).

Since the Hopper model is a published measurement-based model and not a documented contract, it rests on those measurements and on the crafted-input probes above [E13]; the exact reference of Appendix A.10 checks the bounds and decisions against exact arithmetic only under the conservative model’s adders [E28]. On 2,048 captured head inputs, the FP32-output form of the stock head GEMM (the call `--enable-fp32-1m-head` makes) showed a largest error of 2.07×10^{-6} of $\sum_j |w_{ij}h_j|$, 295 times inside the conservative model and 58 times inside the Hopper model; the BF16-output kernel that SGLang runs does not expose its FP32 accumulator, so for it the model remains an assumption [E12].

A.4 The error bound of the low-precision copy

Theorem 3.1 states the error bound for the deployed copy, int8 codes with one scale per entry. The general form below also covers group scales, columns kept exact, blockwise bounds on the quantization error and any summation tree of Lemma A.1; the copies with column groups in Table 4 use it.

Theorem A.2 (Error bound of the copy, general form). *Let row i of the copy have group scales s_{ig} , integer codes and a set of columns kept exact, with dequantized row $\bar{w}_i$ and exact error $e_i = w_i - \bar{w}_i$. If every product is exact in the accumulator format and the pass’s accumulation obeys Lemma A.1, with relative bound γ_d , N internal nodes and absolute floor η , its value $\tilde{z}_i$ satisfies $|z_i - \tilde{z}_i| \leq \beta_i = Q_i + \gamma_d \|\bar{w}_i\|_2 \|h\|_2 + N\eta(1 + \gamma_d) \max(1, \max_g s_{ig})$, where Q_i is $\|e_i\|_2 \|h\|_2$ or any blockwise Cauchy–Schwarz or Hölder form.*

Proof of Theorems 3.1 and A.2. $z_i - \tilde{z}_i = \langle e_i, h \rangle + (\langle \bar{w}_i, h \rangle - \tilde{z}_i)$. The first term is bounded blockwise by Cauchy–Schwarz or Hölder, which gives each of the four forms of Q_i . The second is the rounding error of evaluating $\langle \bar{w}_i, h \rangle$ by a tree whose leaves are the exact products $s_{ig}q_{ij}h_j$ and $w_{ij}h_j$; the group-scale multiplications are nodes with one child, which Lemma A.1 admits after multiplying the leaves below them by s_{ig} , so the underflow term carries the factor $\max(1, \max_g s_{ig})$. Lemma A.1 bounds this term by $\gamma_d \sum_j |\bar{w}_{ij}h_j| + N\eta(1 + \gamma_d) \max(1, \max_g s_{ig})$, and $\sum_j |\bar{w}_{ij}h_j| \leq \|\bar{w}_i\|_2 \|h\|_2$ completes the general bound. Theorem 3.1 is the case of one scale s_i per entry, no exact columns and $Q_i = \|e_i\|_2 \|h\|_2$, with the pass’s accumulation error bounded by $\gamma_{\text{copy}} \sum_j |\bar{w}_{ij}h_j| + \eta'$. For a summation tree that the conservative model covers, $\gamma_d \leq \gamma_{\text{copy}}$ (Appendix A.3), and Lemma A.1 gives the floor $\eta' = N\eta(1 + \gamma_{\text{copy}}) \max(1, s_i)$, with N the internal nodes of the tree and η its per-node floor. $\square$

The componentwise midpoint–radius form $|z - \tilde{z}| \leq |e|^\top |h|$ is tighter than any of the four relaxations and costs the same to evaluate when $|e|$ is stored per group (Rump, 2010); the norm forms are used because they need one constant per vocabulary entry or block.

A.5 The integer pass

Quantizing the activation as well as the weights removes the floating-point accumulation from the pass altogether, because int8 products accumulated in int32 are exact for this shape (Abdelfattah et al., 2025). The price is an activation-quantization term. In retrieval, EAHR bounds int8 inner products by $\|e_q\| \|v\| + \|\hat{q}\| \|e_v\|$ with a floating-point guard and rechecks ambiguous items in FP32 (Zhang, 2026); the theorem below is the corresponding statement for a head with exact int32 accumulation.

Theorem A.3 (Integer pass). *Let $s_h > 0$ be any activation scale in the output format, q_h integer codes, and $e_h = h - s_h q_h$ the exact residual. Let $I_i = \sum_j q_{ij}q_{h,j}$ be accumulated exactly in 32-bit integers ($D q_{\max}^w q_{\max}^h < 2^{31}$; $2560 \cdot 127^2$ qualifies) and $\tilde{z}_i = \text{fl}(\text{fl}(s_i \text{fl}(I_i)) s_h)$ without overflow or underflow. Then*

$$|z_i - \tilde{z}_i| \leq Q_i + \|s_i q_i\|_2 \|e_h\|_2 + \gamma_k \|s_i q_i\|_2 \|s_h q_h\|_2,$$

with $k = 3$ in general and $k = 2$ when s_h is a power of two.

Proof. $z_i = s_i s_h I_i + s_i \langle q_i, e_h \rangle + \langle e_i, h \rangle$ holds in exact arithmetic for any s_h and any codes, because e_h is defined as the residual. The computed value applies at most three roundings to the product chain (the conversion of I_i and the two multiplications), so $|\tilde{z}_i - s_i s_h I_i| \leq \gamma_3 s_i s_h |I_i|$; when s_h is a power of two the last multiplication is exact and γ_2 suffices. Apply Cauchy–Schwarz to $I_i = \langle q_i, q_h \rangle$ and to $\langle q_i, e_h \rangle$, and bound $\langle e_i, h \rangle$ by Q_i as in Theorem A.2. $\square$

The bound holds however the scale and codes of h were chosen, so an implementation must bound $\|e_h\|_2$ from above for the codes it actually produced; $\sqrt{D} s_h/2$ is valid only if every code is the nearest integer to h_j/s_h and none is clipped.

A.6 Computed norms and outward rounding

The error bound β_i of Theorem 3.1 is a real number, and the kernel evaluates it in floating point. The two lemmas below make the evaluated interval an enclosure: the first computes norms of h that are never below the exact norms, and the second rounds the endpoints outward so that every comparison between computed endpoints is sound. Offline constants ($\|e_i\|_2$, $\|\bar{w}_i\|_2$, $\|w_i\|_2$) are computed exactly and stored rounded up.

Lemma A.4 (Computed norms). *In FP32 with round to nearest and $D \leq 2^{14}$, let $\hat{\sigma}$ be any computed sum of the rounded squares $\text{fl}(h_j^2)$ with depth $d \leq 2^{14}$, $\hat{r} \geq (1 - 2^{-16})\sqrt{\hat{\sigma}}$ any computed square root, and $N_2 = \text{fl}(\text{fl}(\hat{r}(1 + 2^{-10})) + 2^{-50})$. Then $N_2 \geq \|h\|_2$. Likewise, if $\hat{\sigma}_1$ is any computed sum of the $|h_j|$ with depth $d \leq 2^{13}$, then $N_1 = \text{fl}(\text{fl}(\hat{\sigma}_1(1 + 2^{-10})) + 2^{-50}) \geq \|h\|_1$.*

Proof. A rounded nonnegative value, and a rounded addition of nonnegative values, returns at least $(1 - u)$ times its exact value less μ (which also covers flush-to-zero). For N_2 the squares add one rounding to the d levels of the sum, so $\|h\|_2^2 \leq (1 - u)^{-(d+1)}\hat{\sigma} + 3D\mu$ and $\|h\|_2 \leq (1 - u)^{-(d+1)/2}(1 - 2^{-16})^{-1}\hat{r} + 2^{-54}$ for $D \leq 2^{14}$. The computed $N_2 \geq (1 - u)^2(1 + 2^{-10})\hat{r} + 2^{-51}$, and $(1 - u)^2(1 + 2^{-10}) \geq (1 - u)^{-(d+1)/2}(1 - 2^{-16})^{-1}$ because $(d + 1)u/2 \leq 2^{-11} + 2^{-25}$. For N_1 the terms $|h_j|$ are exact, so $\|h\|_1 \leq (1 - u)^{-d}(\hat{\sigma}_1 + D\mu)$, and $(1 - u)^2(1 + 2^{-10}) \geq (1 - u)^{-d}$ holds for $d \leq 2^{13}$ (then $du \leq 2^{-11}$) but fails at $d = 2^{14}$, where the margin is about -6×10^{-7} ; there N_1 must be inflated by $1 + 2^{-9}$ instead. $\square$

Lemma A.5 (Outward rounding of the computed interval). *In a format with $p \geq 12$ and round to nearest, let $\tilde{z}$ be representable, let β^* be a real bound with $|z - \tilde{z}| \leq \beta^*$, and let b be computed from representable upper bounds by at most $k \leq 59$ rounded operations $(+, \times, \sqrt{})$ on nonnegative values along any path of the expression, so that $b \geq (1 - u)^k\beta^* - k\mu$. Set*

$$\hat{b} = \text{fl}(\text{fl}(\text{fl}(b(1 + 2^{6-p})) + \text{fl}(2^{4-p}|\tilde{z}|)) + 2^{e_{\min}+26}), \quad lo = \text{fl}(\tilde{z} - \hat{b}), \quad hi = \text{fl}(\tilde{z} + \hat{b}).$$

Then $lo \leq z \leq hi$.

Proof. Each rounding of a nonnegative value loses at most a factor $(1 - u)$ and μ , so $\hat{b}(1 - u) \geq (1 - u)^4(1 + 64u)b + 16u(1 - u)^3|\tilde{z}| + (1 - u)^2 2^{e_{\min}+26} - 4\mu$. Also $lo \leq \tilde{z} - \hat{b}(1 - u) + u|\tilde{z}| + \mu$. It suffices that $\hat{b}(1 - u) \geq \beta^* + u|\tilde{z}| + \mu$: the $|\tilde{z}|$ terms satisfy $16u(1 - u)^3 \geq u$, the b terms satisfy $(1 - u)^{4+k}(1 + 64u) \geq (1 - 63u)(1 + 64u) \geq 1$ for $u \leq 2^{-12}$, and $2^{26}\mu$ absorbs the absolute terms. The upper endpoint is symmetric. $\square$

For FP32 the constants are $1 + 2^{-18}$, $2^{-20}|\tilde{z}|$ and 2^{-100} . The exact reference contains witnesses that each ingredient is necessary: dropping the accumulation term of the error bound, the outward rounding or the norm inflation produces an interval that misses the exact logit (575 cases for the accumulation term, 58 of 300 checks for the norm inflation, and a constructed witness for the outward rounding) [E28].

A.7 The refinement ladder

The certified head's screen and the certified race (Theorem B.2) are instances of one refinement procedure over a sequence of enclosures.

Theorem A.6 (Refinement ladder). *Let levels $\ell = 0, \dots, L$ supply enclosures $lo_i^\ell \leq z_i \leq hi_i^\ell$, exact at level L , and let each f_i be nondecreasing. Starting from entries S , at each level let $\tau = \max_{i \in S} f_i(lo_i^\ell)$ and keep $\{i \in S : f_i(hi_i^\ell) \geq \tau\}$; stop when one entry remains, every kept interval is a point, or $\ell = L$, and return the smallest kept index. The result is $\min \arg \max_{i \in S} f_i(z_i)$, whatever the precision or reduction order of each level; $f_i(x) = x$ gives R -real and $f_i(x) = x/T + g_i$ gives S -race (Definition B.1).*

The Lean proof assistant checks the theorem's decision logic and its filter (it keeps every maximizer and drops only beaten entries) over scaled integers, not under IEEE rounding [E29].

Proof of Theorem A.6. $[f_i(lo_i), f_i(hi_i)]$ encloses $f_i(z_i)$ because f_i is nondecreasing. A maximizer m is never dropped: for the entry j that attains τ , $f_m(hi_m) \geq f_m(z_m) \geq f_j(z_j) \geq f_j(lo_j) = \tau$. A dropped entry is strictly below the entry attaining τ , so it is not a maximizer. If one entry remains, it is the maximizer. If every kept interval is a point, each kept value is at least τ and at most τ , so every kept entry is a maximizer, and every maximizer is kept. The exact level produces points. The returned index is a function of the maximizer set alone, so it does not depend on the precision or reduction order of any level. $\square$

A.8 How many entries a decision re-scores

The ladder says which vocabulary entries survive; the proposition below bounds how many do, which is what decides whether re-scoring is cheap.

Proposition A.7 (Candidate counts). *Let the level-0 intervals have widths $\omega_i = h_i - l_i \leq \omega$ (with outward rounding, $\omega_i \approx 2\beta_i$) and let C be the entries kept at level 0.*

1. *Greedy:* $C \subseteq \{i : z_i \geq z_{\max} - \omega_i - \omega_a\}$ for any maximizer a , hence $|C| \leq e^{2\omega/T'} / p_{\max}^{(T')}$ for every $T' > 0$, where $p^{(T')} = \text{softmax}(z/T')$.
2. *Race:* if the g_i are i.i.d. standard Gumbel and $p = \text{softmax}(z/T)$, then with $t = 2\omega/T$,

$$\mathbb{E}|C| \leq \sum_i \frac{p_i e^t}{1 + p_i(e^t - 1)} \leq \min(V, e^t).$$

Proof. (1) $h_i \leq z_i + \omega_i$ and $l_a \geq z_a - \omega_a$, so a kept entry satisfies $z_i + \omega_i \geq \tau \geq z_a - \omega_a$. Each such entry has $p_i^{(T')} \geq p_{\max}^{(T')} e^{-2\omega/T'}$, and the probabilities sum to at most one. (2) In score space the widths are ω_i/T , so $C \subseteq \{i : y_i \geq M - t\}$ with $y_i = z_i/T + g_i$ and $M = \max_j y_j$. Entry i belongs to that set if and only if $y_i + t \geq \max_{j \neq i} y_j$. The maximum of independent Gumbels with locations z_j/T is Gumbel with location $\log \sum_{j \neq i} e^{z_j/T}$, and two independent Gumbels X_a and X_b with locations a and b satisfy $P(X_a \geq X_b) = e^a / (e^a + e^b)$. Hence the probability is $p_i e^t / (p_i e^t + 1 - p_i)$; sum over i . $\square$

At $T = 1$ and error bound β (so $t \approx 4\beta$), the bound on the expected re-scoring set of an exact race is 55 entries for $\beta = 1$, 2,981 for $\beta = 2$ and up to 1.6×10^5 for $\beta = 3$. Exact sampling through the cheap pass is therefore guaranteed to be cheap only if the bound is about two logits or less. Greedy selection has no distribution-free bound, since a flat distribution (small $p_{\max}$) keeps many entries. In logits the bound is $\beta_i \approx \|e_i\|_2 \|h\|_2$; with the head’s int8 constants for one scale per entry (median $\|e_i\|_2 / \|w_i\|_2 = 0.97\%$ and $\|w_i\|_2 = 0.652$) that is about $0.0063 \|h\|_2$, so the norm of the head input decides whether β is a fraction of a logit or several. On the captured plain-decode states that norm has a median of 159 (1st percentile 132, 99th 169), which puts the int8 error bound at about one logit [E2,E21].

Measured on captured states, the bound’s median is 1.00 logit, 75 times the median realized error of 0.013, because the bound assumes the worst alignment of e_i and h [E5]. The measured candidate sets are far below the bound because most top-two margins are several logits: the gap between the two largest logits has a median of 7.3 logits and a 10th percentile of 0.68 [E5]. Tables 4 and 5 give the candidate sets per copy and the positions and steps the contract leaves to the stock kernel, which Section 4.2 summarizes.

Table 4. Candidate tokens per greedy decision on 6,005 held-out plain-decode positions from 160 prompts; conservative model for the copy’s accumulation; Cauchy–Schwarz error bound per entry for int8 with one scale per entry, blockwise ℓ_2 otherwise. “Bytes”: the copy relative to the BF16 head; “1 entry”: decided without re-scoring. Measured on replayed states [E3].

Copy	Bytes	Candidate tokens			Decided with	
		mean	p99	max	1 entry	≤ 8
int8, per entry	0.50	1.55	8	30	75.4%	99.3%
int8, 128 cols	0.52	1.34	5	23	81.4%	99.8%
int8, 32 cols	0.54	1.26	5	20	84.0%	99.9%
FP8 E4M3, per entry	0.51	4.89	60	605	56.2%	88.4%
int4, 128 cols	0.27	72,847	248,295	248,320	0%	0%

Fallback by call size On calls of 128 consecutive positions the kernel replay measures 27% and 77% of calls with a fallback (Hopper, conservative; Section 4.3) [E17]. Independence at the kernel replay’s own rates would give 30% and 85% (derived) [E15], so the measured shares are consistent with undecided positions clustering in fewer calls than independence predicts.

Table 5. Share of the 6,005 held-out positions, and of the capture’s 750 decode steps (counting each step’s held-out positions, 8.0 on average), that the contract leaves to the stock kernel, by condition and by the model assumed for the stock kernel’s accumulation. The last two rows are sensitivity rows that do not model the stock tensor-core GEMM. Measured on replayed states [E6].

Stock model	γ_{stock}	Gap (Thm. 2.3)		Interval (Prop. 2.4)	
		positions	steps	positions	steps
Conservative	6.11×10^{-4}	3.03%	21.6%	1.40%	10.9%
Hopper	1.193×10^{-4}	2.18%	16.0%	0.35%	2.8%
Sensitivity	10^{-5}	1.97%	14.9%	0.017%	0.13%
IEEE FP32 tree	1.7×10^{-6}	1.95%	14.8%	0%	0%

Table 6. Recomputation and fallback by the model assumed for the stock kernel: mean candidates and the share of positions left to the stock head. Held-out replay columns: the gap and interval conditions, and the share of decode steps (8.0 held-out positions each) with a position the interval condition leaves. The last row is a sensitivity check, not a model of the stock kernel. By condition and step in full: Table 5.

Model (γ_{stock})	Kernel replay [E15]		Held-out replay [E6]		
	Cand.	Interval	Gap	Interval	Steps with a fallback
Conservative (6.11×10^{-4})	1.84	1.46%	3.03%	1.40%	10.9%
Hopper (1.193×10^{-4})	1.78	0.28%	2.18%	0.35%	2.8%
IEEE FP32 tree (1.7×10^{-6})	–	–	1.95%	0%	0%

Other copies on plain decoding The same replay covers heads the main text summarizes in a sentence [E2]. Int4 alone is useless as a certificate: with one scale per 32 weights a decision needs 20,751 candidate tokens on average (p99 230,238, maximum 248,192) and 0.02% of positions are decided by one entry, and with one scale per 128 weights the median set is 13,991 entries. A two-plane cascade, an int4 plane for every entry and an int4 residual plane only for entries the first cannot exclude, reads 0.32 of the BF16 head’s bytes for one position and 0.41–0.42 over capture steps with 5 to 16 held-out positions, at the price of an irregular second gather. Seeded races at $T = 1$ and 0.7 behave like greedy selection, with a mean of 1.33 candidates and a p99 of 5 for the copy with one scale per 128 weights [E3].

Re-scoring candidates with IEEE FP32 accumulation on CUDA cores leaves no position undecided; with the tensor-core error model applied to the re-scoring as well, 2.1% of positions still overlap and need FP64. Re-scoring also reads little: on capture steps with 5 to 16 held-out positions the candidates of the copy with one scale per 128 weights cover 9.5–12.5 distinct vocabulary entries on average [E2], so the gather of their BF16 rows reads tens of kilobytes beside a 636 MB int8 pass (derived). Tightening the error bound helps little: 16 to 64 exact outlier columns or a 64-dimensional PCA rotation of h lower the mean count by 3–7% for 1–7% more pass bytes, and centring W raises it by 10% [E3,E5]. Centring fails because the mean entry points along the bulk of the vocabulary (median cosine 0.39) but not along the entries that compete for the argmax (median cosine -0.11 over each position’s top eight), so it lowers the median error norm over all entries by 14% and raises it by 20% on the entries that matter [E5]. For comparison, an int8 copy of the Qwen3-8B head with one scale per entry agrees with the BF16 argmax on 97.56% of positions (Kocabay et al., 2026), so an exact re-score must catch the rest.

Verification and draft positions Table 7 gives the candidate sets and the gap condition’s fallback on the two speculative captures, with plain decoding for comparison [E3,E7,E8,E6]. The counts use the elementwise minimum of the rigorous per-entry error bounds of the int8 copy with one scale per entry, under the conservative model; it is the only bound for which the replay records whether the verifier needs a position. The positions a greedy verifier needs behave as on plain decoding; for MTP-4B, 76.1% of them are decided by one candidate. Verification positions after the first rejection are conditioned on a wrong prefix and have flatter distributions, so they need more candidates. Unlike DFlash’s, the MTP draft head needs more candidates than the verifier, because its input keeps the target’s scale and the error bound grows with $\|h\|$. Of the two R-stock conditions, only the gap condition was evaluated on the speculative captures. On DFlash-4B it leaves at least one position to the stock kernel in most verify steps, so a whole-batch fallback would run in most verify passes unless the interval condition does much better there. Offline it was not evaluated on the captures; in the engine check it left 3.99% of DFlash-4B verification positions open, and the whole-batch fallback ran in 90% of verify head calls (Section 4.3) [E20]. On

both captures no bound was violated and no winner was missed.

Table 7. Candidates and the gap condition on replayed speculative captures, with plain decoding for comparison: the int8 copy with one scale per entry (FP8 where named) under the elementwise minimum of the rigorous per-entry error bounds and the conservative model for the copy. “Candidates”: mean and 99th percentile. “Left to stock”: share of positions the gap condition (Theorem 2.3) leaves to the stock kernel; “Steps”: share of the capture’s steps with at least one such position; each under the Hopper model and under the conservative model of the stock kernel. Measured [E3,E7,E8,E6].

Capture	Positions	Count	Candidates		Left to stock		Steps	
			mean	p99	Hopper	conservative	Hopper	conservative
Plain decoding	held-out	6,005	1.54	7	2.18%	3.03%	16.0%	21.6%
DFlash-4B, block 16	all verification	4,000			5.68%	8.60%	78.2%	84.2%
	verification, needed by the verifier	1,162	1.57	8				
	verification, after the first rejection	2,838	4.03	20				
	all verification, FP8 copy	4,000	28.91	240				
	draft head, reached	1,138 of 4,020	1.28	4				
	draft head, not reached	2,882 of 4,020	3.36	11				
MTP-4B, four steps	all verification	4,010			2.84%	4.04%	37.5%	48.0%
	verification, needed by the verifier	2,901	1.51	7				
	verification, after the first rejection	1,109	2.44	11				
	draft head, reached	2,997 of 4,024	2.44	15				
	all draft positions	4,024			4.20%	6.44%	44.0%	60.0%

A.9 The kernel and its checks

The certified path of Section 3.4 is sound only for a kernel that computes the modelled arithmetic (Table 2), a property that is checked, not proved. Each compiled variant must pass a self-test before it certifies, and every call checks eight exactly computed vocabulary entries for each position of the batch [E9]. The package’s 145 tests passed on the GPU, where all 148 kernel variants compiled for sm_90 and each default tile configuration had no misses in about a million checks; the host-side refusals added after that run have CPU tests [E9]. One violation of the modelled arithmetic was measured. Under Triton 3.7.1, an int8 tile of the W8A16 pass loaded through a TMA box with a 64-byte inner dimension gave wrong raw products against FP64 in every such configuration tested, at some or all of $M = 1, 16, 64, 128$ and 256, while pointer loads and 128-byte boxes were exact; three tiles with block_m 128 missed the error bound in some runs, and both families are refused [E10]. Of seven Triton and ptas builds tested, the four without both Triton $\geq 3.8.0$ and ptas ≥ 13.3 gave wrong products and the three with both did not, a version pattern that matches the closed issue triton-lang/triton#9433 [E11].

Expected time under the Hopper model Section 4.4 gives the head’s expected time with each fallback under the conservative model. Under the Hopper model, with its lower fallback rates, both expected times lie between those and the all-certified time at every measured M [E17].

A.10 What is machine-checked, what is tested, and what is proved on paper

Machine-checked Lean 4.19.0 elaborates two files of this paper’s lemmas with no sorry, no admit and no added axiom [E29]. Three further files, on the drafting proposals of Section 5.8 of the research notes, elaborate the same way; nothing in this paper rests on them. The paper’s two files work over integers, which stand for scores after clearing a common positive denominator. formal/CertifiedArgmax.lean proves that a quantization bound and an accumulation bound add and that outward widening keeps an enclosure valid (envelope_compose, widen); that disjoint enclosures decide comparisons, that a certified winner is the unique maximizer and that any two certificates from valid enclosures name the same token (separation, certified_argmax, certificate_unique), which is the decision logic of Theorem A.6; that the filter $hi_i \geq \max_j lo_j$ keeps every maximizer and drops only beaten entries (maximizer_survives, dropped_row_loses); that adding a fixed noise value or scaling by a positive temperature preserves an enclosure (shift_encloses, scale_encloses); the acceptance guards of Theorem B.5 with an enclosed numerator and their exclusivity (accept_guard, reject_guard, guards_exclusive); the residual-weight enclosure

of Theorem B.7 (`residual_interval`); the ℓ_∞/ℓ_1 Hölder bound behind the Hölder form of transport (Appendix C.1) and the Hölder form of the quantization error bound (`holder`); and the sequential case of Lemma A.1 (`sequential_sum_bound`). `formal/DecisionGuards.lean` holds nine declarations: scaled-integer acceptance and rejection guards and their stability under refinement, associativity of chronological composition, deterministic replay of records from equal starts (the logical skeleton of the replay proposition for the recurrent state in Section 6.2 of the research notes), and guards that stop a stale epoch or prefix from publishing state. A deliberately false variant of `sequential_sum_bound` is rejected, so the check is not vacuous.

Outside the checker are IEEE rounding and the bridge from floating-point values to these integers; the tree, pairwise, blocked and fused forms of the accumulation bound; Cauchy–Schwarz; real exponentials and the transported mass bounds; the Gumbel-max law and every probability statement; the tensor-core adder model; and compiler lowering, race freedom on the GPU and the engine’s state restoration.

Tested against exact arithmetic An exact floating-point reference (`src/precision_reference.py`) emulates every BF16, FP32 and FP64 rounding on rationals and checks each bound and decision of Sections 3 and 3.3 against real arithmetic [E28]. Its 21 test methods run 40,015 generated checks: 5,600 accumulation-bound checks over eight summation models (sequential, pairwise, blocked, split- K , round toward zero, flush to zero and the fused multi-term adder), where the largest observed error was 17–83% of the bound depending on the model; 1,767 approximate-pass entries and 2,767 integer-pass entries enclosed; 420 certified argmax cases, 72 of them exact ties, of which 183 ended at the approximate pass, 191 after FP32 re-scoring, 24 in FP64 and 22 in exact arithmetic; 360 races under a fixed noise field (62 with a forced tie at the top), 200 top- k sets, 300 acceptance decisions (148 with the uniform placed next to the threshold), 300 residual races and 300 verifications in SGLang’s sibling form; 900 emulations of the stock head’s rounding on adversarial near-tie heads with $D \leq 8$; 160 heads with subnormal weights and inputs; and the witnesses cited in the text for R-bf16 (Section 2.2), for BF16 split- K partials and for a noise stream consumed in evaluation order. The heads are synthetic with small V and D , and the fused adder is a model, not hardware.

A second exact reference, with bounded integer heads and base-2 masses represented as rationals (so that no floating-point exponential is trusted), tests the decision and race algorithms [E30]. Its 18 decision and 3 race test methods pass on the instances listed in Section 8 of the research notes. The race tests use fixed positive rational priorities, so they establish pathwise equivalence of bounded and dense algorithms, not properties of continuous Gumbel noise. The exact witnesses that separate the contracts are in Appendix D.1; those for the recurrent state are in Section 6.2 of the research notes.

Proved on paper only Every other statement of the paper, including the floating-point lemmas above, the transport theorem and the crossover proposition, the sampling results of Appendix B and the propositions of the research notes, is proved in the text and checked by the tests above only where a test is named. Tests are not proofs, and neither is a GPU measurement.

B SAMPLED DECISIONS: FURTHER ANALYSIS

The engine path of Section 3.3 verifies sampled drafts with a fixed noise field, which turns every sampled decision into a race. This appendix states and proves the two theorems behind that choice, the certified race and fixed-noise verification (Appendix B.1), and develops what the choice avoids: the standard acceptance rule evaluated with enclosures, residual races and adaptive verification depth. Every statement takes enclosures of the logits as input and does not depend on the mechanism that produced them, so it applies to transport as well as to the int8 copy. None of these alternatives is implemented on the GPU. The research notes treat sampling before the scores are known, an approximate mode, a demand frontier over a fixed protocol and a CPU experiment that shows why the alternatives were not built (their Sections 7.4 to 7.7).

B.1 The certified race and fixed-noise verification

A sampled decision has a reference of its own beside the three of Definition 2.1.

Definition B.1 (Sampled reference). For $T > 0$ and a fixed noise vector $g \in \mathbb{R}^V$, S-race returns $\min\{i : z_i/T + g_i = \max_j(z_j/T + g_j)\}$.

Theorem B.2 (Certified race). For a fixed noise vector g and $T > 0$, the ladder with $f_i(x) = x/T + g_i$ returns the token of a dense race with the same g and tie rule. If the g_i are i.i.d. standard Gumbel and independent of z , that token has law $\text{softmax}(z/T)$.

Proof. Each f_i is nondecreasing, so Theorem A.6 returns $\min \arg \max_i (z_i/T + g_i)$, which is S-race. The law is the Gumbel-max identity (Papandreou & Yuille, 2011; Maddison et al., 2014); ties have probability zero. $\square$

Bounding the perturbed scores so that most of them need not be computed is the principle of A* Sampling and of lazy Gumbel sampling (Maddison et al., 2014; Musmann et al., 2017); a keyed field adds pathwise equality with the dense race.

Theorem B.3 (Fixed-noise verification). *Let $g_{t,i}$ be keyed by (seed, token position t in the sequence, token i) and i.i.d. standard Gumbel, and let $Y_t = \min \arg \max_i (z_{t,i}(Y_{<t})/T + g_{t,i})$. A speculative decoder that accepts a drafted token if and only if it equals $\arg \max_i (z_{t,i}(x_{<t})/T + g_{t,i})$ at its drafted prefix, and emits that argmax at the first mismatch, outputs Y for every drafter and every rule for how many tokens to draft, and Y has the target’s autoregressive law.*

This is the drafter-invariant construction of Daliri et al. (2025); it turns sampled verification into an argmax that Theorem B.2 certifies.

Proof. By induction on positions, every accepted token equals the argmax at a prefix equal to $Y_{<t}$, so it is Y_t ; at the first mismatch the prefix is $Y_{<t}$ and the emitted token is Y_t . The drafter changes only how many positions one target pass verifies, so the output is the same for every drafter and depth policy. Given $Y_{<t}$, the field $g_{t,\cdot}$ is independent of $z_t(Y_{<t})$, so $Y_t \mid Y_{<t} \sim \text{softmax}(z_t(Y_{<t})/T)$ by the Gumbel-max identity. $\square$

The theorem treats z_t as a function of the prefix alone, which the engine does not guarantee (Section 4.1 and Appendix D.4); the contract of Section 3.3 is scoped accordingly.

A keyed field is what gives pathwise equality with the dense race: in the exact rational emulation, feeding the candidates noise from a stream consumed in evaluation order changed the winner in 247 of 300 cases [E28].

Two further conditions complete the random-number contract of Section 3.3. Equality in distribution, as opposed to seeded equality, needs i.i.d. noise with the correct marginal that is independent of the proposal draw, the acceptance uniform and the scores; a residual race (Appendix B.4) also needs independence from the acceptance event. And a hash with 2^{32} levels makes the law exact only for the idealized continuous field; the stock seeded sampler shares that approximation.

Truncated sampling The Gumbel-max identity has a top- k form that samples k tokens without replacement (Kool et al., 2019). Top- k sampling applies the ladder to membership (drop i when hi_i is below the k -th largest lo) and races over the certified set. With noise keyed by token, as in the full-vocabulary race, this gives equality in distribution with top- k sampling from the reference scores and, once the engine’s tie semantics at the k -th position are matched, pathwise equality with a token-keyed dense race over the same set. It does not give seeded equality with SGLang’s sampler at the pinned SGLang commit (Appendix F.3).

That sampler’s seeded top- k and top- p path sorts the probabilities, keys each token’s Gumbel noise by the token’s rank among the sorted probabilities rather than by its index, and maps the winner back to a token through the sort’s permutation, so the noise a token receives depends on its rank (SGLang contributors, 2026). Seeded equality with that path therefore needs, beyond membership, the complete order within the certified set: enclosures of the FP32 probabilities the sampler sorts that are strictly separated for every pair of members whose ranks could otherwise differ, and `torch.sort`’s order where probabilities are equal. The pinned sampler also refuses min- p with a seed. Its full-vocabulary seeded path keys the noise by token index, and that is the case the certified race of Section 3.3 covers.

Other truncations differ in what they need. Min- p admits i when $z_i \geq z_{\max} + T \log(\min-p)$, which intervals decide without a partition function. Top- p admission depends on the mass above each token relative to the partition function Z , so it needs partition bounds and is usually cheaper to hand to the ordinary path. Fused samplers take the other route and make the dense path cheap: FlashInfer samples top- k , top- p and min- p without sorting and implements chain speculative sampling (Ye et al., 2025; Xing et al., 2025), and SonicSampler fuses the post-logit sampling and verification kernels over a bounded top-128 pool and describes top- p under that pool as theoretically lossy (Ponnusamy et al., 2026). The closest precedent for sampling from bounded scores, Musmann, Levy and Ermon’s lazy Gumbel sampling, draws noise only for a retrieved top set and, with an approximate top set, widens the cutoff by the approximation error (Musmann et al., 2017; Musmann & Ermon, 2016); the certified race differs in bounding every vocabulary entry with a dense cheap pass and in keying the noise by token, which gives pathwise equality with a dense race keyed the same way, not with the stock sampler’s rank-keyed truncated path.

B.2 Acceptance under fixed-noise coupling

Fixed-noise verification of a sampled draft accepts less often than the standard rule. The proposition says by how much.

Proposition B.4 (Acceptance under fixed-noise coupling). *Suppose that the drafter proposes the token $\arg \max_i (\log q_i + g_i)$ and that the target accepts it exactly when it equals $\arg \max_i (\log p_i + g_i)$. Then*

$$P(\text{accept}) = \sum_i \left(\sum_j \max\left(\frac{p_j}{p_i}, \frac{q_j}{q_i}\right) \right)^{-1},$$

$$\frac{1}{2} \sum_i \min(p_i, q_i) \leq \sum_i \frac{p_i q_i}{p_i + q_i} \leq P(\text{accept}) \leq \sum_i \min(p_i, q_i).$$

Proof. Both choose i if and only if $g_i > \max_{j \neq i} (g_j + m_j)$ with $e^{m_j} = \max(p_j/p_i, q_j/q_i)$, an event of probability $(1 + \sum_{j \neq i} e^{m_j})^{-1}$. The bounds follow from $\max(x, y) \leq x + y$, from $\max(x, y) \geq x$ and $\max(x, y) \geq y$ summed over j , and from $p + q \leq 2 \max(p, q)$. $\square$

Per position, fixed-noise acceptance of a sampled draft is therefore at most the standard rule's rate $1 - \text{TV}(p, q)$ and at least half of it, and the two agree when $p = q$. For SGLang's default point-mass drafts the question does not arise: the stock rule accepts a draft d with probability $p(d)$, and so does the keyed race. Whether a cheaper certificate pays for any lost acceptance with sampled drafts is an empirical question, and the acceptance under fixed-noise coupling against standard verification on real drafts was not measured.

B.3 The standard acceptance rule with enclosures

At the pinned commit, SGLang's sampled speculative verification uses point-mass drafts by default: a drafted token is accepted when a uniform coin falls below the target probability mass of the siblings tried so far, and a rejection samples the target restricted to the untried tokens by inverse CDF (code reading). In general, let p be the target law at the reached prefix and q the *actual* proposal law, conditional on the information used to produce the proposal. A softmax buffer from a different selector is not q , and a deterministic proposal is a point mass. The standard one-step correction accepts $X \sim q$ with probability $\min(1, p_X/q_X)$ and otherwise samples $(p - q)_+$ normalized; the accepted mass at i is $\min(p_i, q_i)$ and the residual contributes $(p_i - q_i)_+$, which sum to p_i (Leviathan et al., 2023; Chen et al., 2023). SpecTr and block verification extend the rule to several drafts and to whole blocks (Sun et al., 2023; 2025). A certificate may change how this comparison is evaluated, never the law used in it.

For one reached proposal $X = x$ write $w_x = e^{z_x}$ and $Z = \sum_i e^{z_i}$, and draw $U \in [0, 1)$ with the reference convention. For $q_x > 0$, acceptance is equivalent to

$$U q_x Z < w_x, \quad (3)$$

and the $\min(1, \cdot)$ need not be evaluated because $U < 1$ already handles $p_x \geq q_x$.

Theorem B.5 (Acceptance guards with enclosures). *Let $w_x \in [N^-, N^+]$ and $Z \in [Z^-, Z^+]$ with $N^- > 0$, let $0 < q_x \leq 1$ be the actual proposal mass and $U \in [0, 1)$. Then $U q_x Z^+ < N^-$ implies acceptance, $U q_x Z^- \geq N^+$ implies rejection, the two never both hold, and replacing an enclosure by a tighter one cannot reverse a certified decision. For U uniform and independent of the enclosures, neither guard fires with probability*

$$P_\gamma = \min(1, N^+/(q_x Z^-)) - \min(1, N^-/(q_x Z^+)).$$

If $N^- = N^+ = w_x$ and every entry outside a re-scored set R has a mass enclosure whose upper and lower ends differ by a factor at most κ , then with $\pi = \sum_{i \notin R} p_i$,

$$P_\gamma \leq \frac{p_x}{q_x} \cdot \frac{(\kappa - 1)\pi}{1 - \pi(1 - 1/\kappa)}.$$

Proof. The implications are transitivity with (3); both firing would give $N^+ \leq U q_x Z^- \leq U q_x Z^+ < N^-$. A tighter enclosure stays on the same side of the true values. The unresolved uniforms form $[N^-/(q_x Z^+), N^+/(q_x Z^-)) \cap [0, 1)$. For the bound, $P_\gamma \leq (w_x/q_x)(Z^+ - Z^-)/(Z^+ Z^-)$; $Z^+ - Z^- \leq \sum_{i \notin R} e^{z_i}(\kappa - 1)$ because each lower mass is at most e^{z_i} ; and $Z^- \geq Z - \sum_{i \notin R} e^{z_i}(1 - 1/\kappa)$ because each lower mass is at least e^{z_i}/κ . Divide by Z . $\square$

Transported tile masses (Theorem 5.1) and exponentiated intervals from the int8 copy (Theorem 3.1) both supply $Z^\pm$. With the int8 copy $\kappa = e^{2\beta}$ times the outward rounding of the exponential; at $\beta = 1$ ($\kappa \approx 7.4$), keeping P_γ below 10% at $p_x/q_x \approx 1$ requires the unre-scored mass π to be below about 1.5%. After an adaptive refinement chosen using U the probability statement no longer applies, although the guards remain sound pointwise. The measured partition widths on real draft states are in Appendix C.7; for transport they leave P_γ near one.

The guards decide acceptance, and that is where they stop being enough. A rejection still requires a sample from $(p - q)_+$, and knowing that rejection occurred does not supply one. Under a stock-token contract the stock inverse-CDF residual would need every logit of the position after each rejection, so a cheap acceptance decision would still be followed by a pass over the whole head. This is why the engine path uses fixed-noise verification, which needs neither a partition function nor a residual sampler, and why the guards and the residual races below are analysis, not built.

B.4 Residual and bonus races

When all drafted tokens are accepted, a bonus token needs a target sample; after a rejection the residual needs one. Two facts about the engine and one construction reduce the problem. First, SGLang’s default sampled verification uses point-mass proposals (Appendix B.3); with q a point mass at x , the residual is the target restricted to the untried tokens, a race over $V \setminus \{x\}$ that needs no partition function. Second, under fixed-noise verification there is no residual at all. Third, for a proposal with small support there is an exact bounded race.

Let $Q = \{i : q_i > 0\}$. With unnormalized target weights w_i and partition Z , the residual can be sampled from the weights

$$r_i = (w_i - Zq_i)_+, \quad \frac{r_i}{\sum_j r_j} = \frac{(p_i - q_i)_+}{\sum_j (p_j - q_j)_+}, \quad (4)$$

with $r_i = w_i$ outside Q . Given $Z^- \leq Z \leq Z^+$, positive-part monotonicity gives $(w_i - Z^+q_i)_+ \leq r_i \leq (w_i - Z^-q_i)_+$ inside Q .

Theorem B.6 (Bounded residual race). *Assume valid enclosures of the target weights and of Z , the actual proposal law, a nonzero residual, and a priority field $\pi_i = e^{g_i} > 0$ that is fixed and independent of the proposal draw, the acceptance uniform and the rejection event. A refinement procedure that returns only an interval-certified ordered maximizer of $r_i\pi_i$, or completes the dense race, returns the same token as the dense race. With i.i.d. standard Gumbel g_i it samples the residual law (4) exactly.*

Proof. The enclosures of r_i multiplied by $\pi_i > 0$ enclose the priorities. A certified maximizer dominates every possible competitor, including the actual one, and a completed evaluation yields the dense ordered maximizer. The distributional statement is Gumbel-max applied to the nonzero residual weights. $\square$

Theorem B.7 (Residual races from weight enclosures). *Replace the enclosures above by $r_i \in [(N_i^- - Z^+q_i)_+, (N_i^+ - Z^-q_i)_+]$ for $q_i > 0$ and $r_i \in [N_i^-, N_i^+]$ for $q_i = 0$. The conclusions of Theorem B.6 hold. For a point-mass proposal at x , $r_x = 0$ as soon as $Z^- \geq N_x^+$, and the residual is a race over $V \setminus \{x\}$ that needs no partition bound. SGLang’s target-only rule (accept sibling j if $UZ < \sum_{l \leq j} w_{x_l}$; otherwise sample the target restricted to the untried tokens) is certified by the guards with a cumulative numerator and an exclusion race.*

Proof. Positive-part monotonicity gives the enclosures; the rest is the proof of Theorem B.6. For the point mass, $w_x - Zq_x = w_x - Z \leq 0$. $\square$

The engine samples that residual by inverse CDF, so a race matches it only in distribution. For a support chosen after the draft pass, retaining the top $K + 1$ perturbed entries per tile tightens the maximum outside the support after up to K support tokens are excluded; it is not needed for validity, since $r_i \leq w_i$ makes the plain tile maximum an upper bound. A bonus sample is simpler still: a certified race over the target scores with the bonus position’s field, with no partition function.

B.5 When adaptive verification changes the sampling law

Avoiding work whose result is already determined differs from choosing which sampled proposals count. DSpark states that lossless speculation requires admission decisions that do not depend on future candidate tokens, and D-cut gives a counterexample and a repair (Cheng et al., 2026; Liu et al., 2026); the two-token illustration below is theirs in substance. Let

$p = q = (1/2, 1/2)$, draw $X \sim q$, discard it and draw afresh from p if $X = 0$, and admit it to the ordinary verifier if $X = 1$. Since $p = q$ an admitted proposal is always accepted, and the output law is

$$\Pr(Y = 0) = \frac{1}{4}, \quad \Pr(Y = 1) = \frac{3}{4}. \quad (5)$$

The acceptance formula was unchanged; admission selected proposals by their realized value. A mandatory progress floor on the first token does not help, since the same construction applies at the second position. Three settings avoid the issue: greedy decoding; target-only verification with point-mass proposals, where there is no proposal law to bias; and fixed-noise verification, under which every depth policy yields the same output (Theorem B.3). The demand frontier of the research notes (their Section 7.6) changes only the amount and order of arithmetic used to decide a fixed protocol, so it is exact pointwise.

C TRANSPORT: SUPPLEMENTARY ANALYSIS AND RESULTS

Section 5 states the transport bound, the condition under which it can be tighter than the int8 copy’s error bound, and its measured failure on both drafters. This appendix gives the geometry a transport bound can use, the proofs behind the crossover, the full weight-only constants, how the draft and target pairs were captured and checked, and the measurements the main text summarizes: the drift by block position, outcome, domain and context, certification by bound family, sampled acceptance, batch unions, and the drift-scaling extrapolation. Throughout, W is the tied head, h^d and h^t are the draft and target head inputs behind the same verified draft token, $\Delta = h^t - h^d$, and a tile g has centre μ_g and radius r_g as in Theorem 5.1.

C.1 What the bound can use

Theorem 5.1 uses a Euclidean tile radius $r_g \geq \max_{i \in g} \|w_i - \mu_g\|_2$ and charges $\epsilon_g = r_g \|\Delta\|_2$. The other natural pairing is Hölder’s, an ℓ_∞ radius $r_g^\infty \geq \max_{i \in g} \|w_i - \mu_g\|_\infty$ with $\epsilon_g = r_g^\infty \|\Delta\|_1$; its proof is the same with Hölder’s inequality in place of Cauchy–Schwarz. Either way a good draft does not imply a small ϵ_g . The head can ignore directions in which the states differ, and probabilities ignore a common shift of all logits, yet a hidden-space norm charges both. With $W = ((1, 0), (0, 0))$ and $\Delta = (0, L)$ both logits are unchanged for every L , while $r_g^\infty \|\Delta\|_1$ grows with L ; coordinate radii remove that particular failure. The bounds form a hierarchy: a scalar radius; coordinate radii, with $\epsilon_g = \sum_j r_{gj} |\Delta_j|$; grouped norms

$$\epsilon_g = \sum_p r_{gp} \|\Delta_p\|_2, \quad r_{gp} \geq \max_{i \in g} \|(w_i - \mu_g)_p\|_2; \quad (6)$$

and a low-rank split $w_i = \mu_g + Qu_i + e_i$ with the projected part evaluated exactly and a norm bound on e_i . Richer geometry costs metadata and bound arithmetic, and a learned centre or subspace does not certify the residual radius, which must cover the stored weights. In the engine the draft maxima and masses come from BF16-rounded draft logits, so every transported bound must also be widened by the draft kernel’s error before it encloses a value the engine computes; the terms are those of Appendix A.3, and the replay omits them, which favours transport.

Greedy and top- k admission Suppose at least k distinct vocabulary entries have been evaluated at the target and τ is the k -th best evaluated score. A tile may be skipped only when its transported upper bound $U_g = m_g^d + a_g + \epsilon_g$ is below τ ; then at least k evaluated items outrank every skipped one. The strict inequality preserves unresolved ties. This establishes exact selection, not an exact unrestricted softmax, because an omitted logit still contributes to the normalizer even when it cannot win.

Three small falsifiers Each shows a shortcut that the bound cannot take. *Agreement at an anchor proves nothing away from it*: let $h^d = (1, 0)$ and $h^t = (1, 1)$; entries whose weight rows share the first coordinate have identical anchor logits, while their second coordinates make the target logits arbitrarily different, so an anchor maximum or mass alone cannot certify the target, and a geometric bound, more projection or exact completion is necessary. *One anchor mass cannot follow a temperature change*: the unnormalized masses $(1, 3)$ and $(2, 2)$ both sum to four, while their squares sum to ten and eight, so the mass at a new temperature cannot be reconstructed from the old sum. *Transformed logits*: if a transform adds a known per-token correction b_i with tile bounds $b_g^- \leq b_i \leq b_g^+$, then $z_i^d + a_g - \epsilon_g + b_g^- \leq z_i^t + b_i \leq z_i^d + a_g + \epsilon_g + b_g^+$ and the mass bounds multiply by $e^{b_g^+}$, but a mask that deletes unknown members cannot preserve the unmasked lower mass, and a safe lower bound may collapse to zero.

C.2 Proofs for the crossover

The crossover compares two error bounds that are each optimal for the information they use, so no sharper inequality can change its conclusion.

Proposition C.1 (No better bound from the same information). *For fixed h and radii $\epsilon_b \geq 0$ over a partition of the coordinates into blocks b , $\sup\{|\langle e, h \rangle| : \|e_b\|_2 \leq \epsilon_b \forall b\} = \sum_b \epsilon_b \|h_b\|_2$ and $\sup\{|\langle e, h \rangle| : \|e_b\|_\infty \leq \epsilon_b \forall b\} = \sum_b \epsilon_b \|h_b\|_1$. Likewise $\sup\{\langle w - \mu, \Delta \rangle : \|w - \mu\|_\infty \leq r\} = r \|\Delta\|_1$.*

Proof. The inequalities are Cauchy–Schwarz and Hölder. Equality holds at $e_b = \epsilon_b h_b / \|h_b\|_2$, at $e_b = \epsilon_b \text{sign}(h_b)$ and at $w = \mu + r \text{sign}(\Delta)$. $\square$

For errors not aligned with h the realized $|\langle e_i, h \rangle|$ is typically about $\|e_i\|_2 \|h\|_2 / \sqrt{D}$; closing that gap needs more arithmetic, not a better inequality.

Proposition C.2 (Exclusion conditions). *After the winner a is scored exactly, let $m_i = z_a - z_i$. With ϵ_g either transported half-width, entry-level transport excludes i if and only if $m_i > \epsilon_g - \langle w_i - \mu_g, \Delta \rangle$; tile-level transport excludes tile g if and only if $z_a > m_g^d + \langle \mu_g, \Delta \rangle + \epsilon_g$; the int8 copy (in real arithmetic, with error bound β_i) excludes i if and only if $m_i > \beta_i - \langle e_i, h \rangle$. The slack terms lie in $[0, 2\epsilon_g]$ and $[0, 2\beta_i]$.*

Proof. Substitute $z_i^d + \langle \mu_g, \Delta \rangle = z_i - \langle w_i - \mu_g, \Delta \rangle$ and $\tilde{z}_i = z_i - \langle e_i, h \rangle$ into the upper bounds. $\square$

Proof of Proposition 5.2. For entries k and l of the tile, $\|w_k - w_l\|_2 \leq \|w_k - \mu_g\|_2 + \|w_l - \mu_g\|_2 \leq 2r_g$, so $r_g \geq \frac{1}{2} \text{diam}_2(g)$ for every centre. The equivalence is the definition of δ_i^* rearranged: $r_g \|\Delta\|_2 < \|e_i\|_2 \|h^t\|_2$ exactly when $\|\Delta\|_2 / \|h^t\|_2 < \|e_i\|_2 / r_g$. $\square$

The Hölder pairing has a crossover of the same kind, with a sufficient condition that depends on the tile only through one property of its entries.

Proposition C.3 (Crossover in the Hölder family). *With b -bit round-to-nearest codes and one scale per entry, $s_i = \|w_i\|_\infty / (2^{b-1} - 1)$ and no clipping, $|\langle e_i, h \rangle| \leq \frac{s_i}{2} \|h\|_1$, and $\frac{s_i}{2} \|h\|_1 \leq r_g^\infty \|\Delta\|_1$ whenever*

$$\frac{\|\Delta\|_1}{\|h\|_1} \geq \frac{\|w_i\|_\infty}{(2^{b-1} - 1) \text{diam}_\infty(g)}.$$

If some entry of g has the opposite sign, or zero, in the coordinate where $|w_{ij}|$ peaks, then $\text{diam}_\infty(g) \geq \|w_i\|_\infty$ and the condition is implied by $\|\Delta\|_1 / \|h\|_1 \geq 1 / (2^{b-1} - 1)$: 1/127 for int8 and 1/7 for int4.

Proof. Round-to-nearest codes without clipping give $|e_{ij}| \leq s_i/2$, and Hölder’s inequality gives the first bound. As in the Euclidean case $r_g^\infty \geq \frac{1}{2} \text{diam}_\infty(g)$, which gives the second. If entry k has the opposite sign, or zero, where $|w_{ij}|$ peaks, then $|w_{ij} - w_{kj}| \geq |w_{ij}|$, so $\text{diam}_\infty(g) \geq \|w_i\|_\infty$. $\square$

With the tile radius measured rather than bounded by the diameter, the corresponding threshold on $\|\Delta\|_1 / \|h\|_1$ has a median of 0.30% against int8 with one scale per entry (Table 8); the condition $\text{diam}_\infty(g) \geq \|w_i\|_\infty$ holds for 99.9% of entries with 64-entry tiles [E21].

Proposition C.4 (Transport against the static screen). *If μ_g lies in the convex hull of the tile’s weight rows, then $U_g^T - U_g^{\text{static}} \geq r_g (\|\Delta\|_2 - \|h\|_2)$, where $U_g^T = m_g^d + \langle \mu_g, \Delta \rangle + r_g \|\Delta\|_2$ and $U_g^{\text{static}} = \langle \mu_g, h \rangle + r_g \|h\|_2$ are the upper bounds of transport and of a static screen at the target state with the same geometry. For the Hölder pairing, $U_g^T - U_g^{\text{static}} \geq r_g^\infty (\|\Delta\|_1 - \|h\|_1)$ with both bounds in r_g^∞ and ℓ_1 norms. Transport is therefore no tighter than the static screen once $\|\Delta\| \geq \|h\|$ in the norm the bound uses.*

Proof. $U_g^T - U_g^{\text{static}} = (m_g^d - \langle \mu_g, h^d \rangle) + r_g (\|\Delta\|_2 - \|h\|_2)$, and $m_g^d = \max_k \langle w_k, h^d \rangle \geq \langle \mu_g, h^d \rangle$ for a convex combination μ_g . The Hölder case is the same computation. $\square$

On both drafters the drift is close to the size of h in either norm: $\|\Delta\|_2 / \|h^t\|_2$ has a median of 0.92 for DFlash-4B and 0.95 for MTP-4B, and $\|\Delta\|_1 / \|h^t\|_1$ of 0.93 and 0.99 [E22,E24]. The proposition therefore leaves transport at most a few percent of the static screen’s radius term to gain, and Table 11 shows that it gains nothing measurable.

C.3 Weight-only constants

Table 8 gives the constants of the tied head from which the thresholds of Section 5.2 are derived. They use no hidden states, so they decide nothing about certification rates on their own. Two further facts follow from them. First, the static tile screen is weak on this head, because the radius of a 64-entry tile exceeds the norm of an entry’s weight row (median $r_g/\|w_i\|_2 = 1.14$), so $\langle \mu_g, h \rangle + r_g\|h\|_2$ is wider than the plain per-entry bound $\|w_i\|_2\|h\|_2$, and tiles in token order are barely tighter than random ones; a public experiment on a Qwen3.8 head with the same vocabulary also found that centroid-radius bounds prune almost nothing (qwen38-challenge_senpai contributors, 2026). Second, a scalar ℓ_∞ radius paired with $\|\Delta\|_1$ is about four times looser than the Euclidean pairing for a spread-out Δ (derived from the same constants, $r_g^\infty \approx 1.30\|w_i\|_\infty$ and $\|\Delta\|_1 \approx 0.8\sqrt{D}\|\Delta\|_2$). Bounding an inner product under a bounded displacement of the query, which is what transport does, is known from sequential screening rules for the lasso (Wang et al., 2015) and from cached retrieval with drifting queries (Frieder et al., 2024).

Table 8. Weight-only constants of the Qwen3.5-4B head, computed in float64 on the CPU; quantiles over all 248,320 vocabulary entries. $\delta_i^* = \|e_i\|_2/r_g(i)$ is the drift below which transport’s Euclidean error bound for entry i is narrower than that of the named low-precision copy (“per entry”: one scale per entry), with 64-entry tiles of contiguous token indices and μ_g the tile mean; the last row is the analogous threshold for the ℓ_∞/ℓ_1 pairing, $s_i\|h\|_1/2$ against $r_g^\infty\|\Delta\|_1$. Error-bound widths are compared, not certification rates or runtime. Derived [E21].

Quantity (tied head, $248,320 \times 2,560$)	p10	p50	p90
Weight-row norm $\ w_i\ _2$	0.574	0.652	0.746
int8 per entry, $\ e_i\ _2/\ w_i\ _2$	0.81%	0.97%	1.25%
int8 per 128 weights	0.64%	0.68%	0.75%
int4 per entry	14.8%	17.5%	22.6%
64-entry tile $r_g/\ w_i\ _2$, token-index order	0.99	1.14	1.31
random tiles (control)	1.03	1.19	1.37
δ^* against int8 per entry	0.67%	0.85%	1.15%
δ^* against int8 per 128 weights	0.51%	0.61%	0.72%
δ^* against int4 per entry	12.2%	15.5%	20.8%
Hölder threshold on $\ \Delta\ _1/\ h\ _1$ against int8 per entry (Proposition C.3)	0.23%	0.30%	0.40%

C.4 Captures and their alignment

Every capture uses the same 320 public prompts, the *capture prompts*, 40 from each of eight datasets in four domains: MT-Bench and No Robots (chat), HumanEval and MBPP (code), GSM8K and MATH-500 (mathematics), and m-ArenaHard and MGSM (multilingual), 112 of them in thinking mode; Appendix F.3 names their manifest. They are split by prompt into analysis and held-out halves before any fitting. In plain decoding the FP64 argmax of the captured head input equals the engine’s token at 99.51% of 105,269 positions, and all 519 disagreements lie within one BF16 spacing of the FP64 winner [E1].

Two drafters produce the speculative pairs. The model’s own MTP layer drafts one token per step and projects it through the target’s head. The public DFlash drafter for this target (z-lab, 2026) is a six-layer transformer of width 2,560 with no embedding or head of its own: it embeds with the target’s table, conditions on the target’s hidden states after eight of its layers, and drafts a block of 16 positions in one forward pass, of which positions 1 to 15 go through the target’s tied head, their argmaxes being the drafted tokens (read from the checkpoint and from SGLang’s DFlash worker at the pinned SGLang commit; Section 5.1 of the research notes gives the details).

Speculative pairs follow the verify layout. A verify step for one request holds $[x_0, d_1, \dots, d_S]$ with x_0 the last committed token; draft token d_k came from the draft head input h_k^d and is decided by the target head input at verify position $k - 1$. Each pair is labelled accepted, rejected or unreached (after the first rejection). The target input at the last verify position, which produces the bonus token, has no draft anchor at all, so a transport-only verifier still streams the whole head for that position. The captures used the patch and the server configuration of Section 4.1 (no CUDA graphs and no overlap scheduler, because the hooks run in Python between forwards), greedy decoding and 384 new tokens on the capture prompts; the DFlash drafter ran at block size 16 with five concurrent requests, and the MTP layer with four draft steps, top- k 1 and at most eight running requests [E26,E24].

The DFlash-4B capture was checked as the plain-decode capture was [E26]. It holds 24,020 verify blocks and 360,300 draft

slots. The FP64 argmax of the captured draft head input equals the engine’s draft token at 98.2% of slots, and that of the target head input the engine’s target token at 98.9%; every disagreement lies within one BF16 spacing of the FP64 winner, and the exact logits rounded to BF16 with the first-index rule (R-bf16) reproduce the engine at 99.996% (draft) and 99.998% (target). The engine’s accepted lengths match the labels derived from the captured tokens on all 24,020 blocks. Draft agreement falls from 99.3% at block position 1 to 97.0% at position 15, as later draft distributions become flatter and near ties more common. Only 39% of prompts produce output identical to plain decoding; over all prompts, counting identical outputs at their full 384 tokens, the median first diverging output token is 181. This is not a pairing error, since every per-slot check uses the engine’s own decisions. The verify pass runs different kernels, at different batch shapes, from plain decoding, and 75 of the 77 divergences of DFlash outputs from plain decoding measured on 80 further prompts (16 each of chat, code and GSM8K training prompts from the serving workload, and 32 MATH-500 problems) are at near ties [E47]; they were not traced to a kernel.

The MTP-4B capture passes the same checks [E27]. It holds 30,328 verify blocks and 121,312 draft positions. The FP64 argmax of the captured draft head input equals the engine’s draft token at 99.08% of positions, and that of the target head input the engine’s target token at 99.33%; all 1,116 draft and 816 target disagreements lie within one BF16 spacing, R-bf16 reproduces the engine at 99.997% (draft) and 99.998% (target), and the engine’s accepted lengths match the derived labels on all 30,328 blocks. Draft agreement falls from 99.31% at step 1 to 98.90% at step 4, and a reached step is accepted at 84.5%, 80.0%, 79.5% and 80.6% at steps 1 to 4. The engine’s own top two target logits were equal in BF16 at 1.3% of positions. 42% of prompts produce output identical to plain decoding, and the median first diverging output token, counted as for DFlash-4B, is 193.5.

The analyses use random subsets of the held-out pairs, the drift samples drawn pair by pair and the transport samples as whole capture steps: the DFlash drift on 40,000 pairs (9,296 accepted, 2,434 rejected and 28,270 unreached), DFlash transport on 4,020 pairs from 110 prompts, the MTP drift on 40,000 pairs from 160 prompts (24,835 accepted, 5,698 rejected and 9,467 unreached), and MTP transport on 16,016 held-out pairs from 160 prompts (9,969 accepted, 2,298 rejected and 3,749 unreached) [E22,E23,E25,E24].

C.5 The drift in detail

Table 9 breaks the DFlash drift down. It barely depends on anything: accepted drafts are no closer than rejected ones, the median rises from 0.88 at block position 1 to 0.92 from position 6 on, and domain and context length move it by a few hundredths. For every pair, transport’s error bound is narrower than any of the four low-precision bounds (int8 and int4, one scale per entry and one per 128 weights) on 0.08% of entries, and those entries are the three tiles of identical unused-token entries, whose radius is zero [E22].

The main text explains the size of the drift by the draft input’s fixed norm and its angle to the target’s. The drafter’s scores are also softer than the target’s: the median top-token probability is 0.17 against 0.69 [E26]. Further measurements show that the drift is not an artefact of the Euclidean metric [E22,E23]. In the head’s own metric the drift stays of order one: $\|W\Delta\|_2/\|Wh^t\|_2$ has a median of 0.87 before and 0.85 after removing each position’s mean logit, and the median over pairs of the largest logit drift $\max_i |\langle w_i, \Delta \rangle|$ is 13.6 logits (root mean square 2.0). The largest realized transport error in a tile is a median 77 times the largest realized int8 error. Restricted to W ’s leading 64 right singular directions, the relative drift is 0.89, and those directions carry 25% of Δ ’s norm, against 16% for an isotropic vector. In the ℓ_1 norm of the Hölder family the ratio is 0.93.

Table 10 breaks down the drift of the MTP layer. It is wider than DFlash’s and reaches lower, with a median of 0.954 and a smallest drift of 0.385 against 0.917 and 0.823, because the MTP head input is not held at a fixed norm (medians 176 for the draft and 160 for the target) [E24]. It also depends more on the draft: the median rises from 0.86 at step 1 to 1.00 at step 4, rejected drafts lie lower than accepted ones (0.89 against 0.99), and code and mathematics (both 1.02) lie above chat and multilingual prompts (0.92). The other metrics agree [E25]. In the head’s metric the median is 0.83, and 0.88 after removing each position’s mean logit; in W ’s leading 64 right singular directions it is 0.86, and those directions carry 23% of Δ ’s norm; the median ℓ_1 ratio is 0.99; and the median largest logit drift is 10.7 [E24]. None of this changes the verdict. Transport’s error bound is narrower than int8’s only on the 0.08% of entries in the tiles of unused tokens, the realized per-tile transport error is a median 81 times int8’s, and certified ℓ_2 transport and the static screen each skip 0.08% of entries. Even oracle radii skip 99.97% (median; p10 98.6%), where the int8 copy decides with about one candidate [E25].

Table 9. Drift $\rho = \|h^t - h^d\|_2 / \|h^t\|_2$ of DFlash-4B pairs by outcome, domain, block position and context length (tokens before the verified position), on 40,000 held-out pairs. The smallest drift over all pairs is 0.823. Measured [E22].

Group	Pairs	p10	Median	p90	Group	Pairs	p10	Median	p90
All	40,000	0.875	0.917	0.964	Block position 1	2,690	0.852	0.882	0.938
Accepted	9,296	0.865	0.909	0.958	Block position 2	2,634	0.865	0.900	0.957
Rejected	2,434	0.856	0.891	0.952	Block position 4	2,625	0.875	0.915	0.963
Unreached	28,270	0.884	0.920	0.966	Block position 8	2,729	0.880	0.921	0.966
Chat	13,374	0.876	0.915	0.956	Block position 12	2,661	0.886	0.922	0.966
Code	4,880	0.880	0.923	0.961	Block position 15	2,716	0.889	0.924	0.964
Mathematics	6,240	0.875	0.925	0.965	Context 0–255	20,109	0.874	0.915	0.959
Multilingual	15,506	0.872	0.914	0.970	Context 256–511	17,715	0.875	0.918	0.965
					Context 512–1,023	1,909	0.881	0.934	0.981
					Context $\geq 1,024$	267	0.877	0.930	0.966

Table 10. Drift ρ of MTP-4B pairs by outcome, domain, draft step and context length, on 40,000 held-out pairs from 160 prompts. Measured [E25].

Group	Pairs	Smallest	p10	Median	p90
All	40,000	0.385	0.767	0.954	1.174
Accepted	24,835	0.385	0.760	0.986	1.201
Rejected	5,698	0.415	0.728	0.887	1.085
Unreached	9,467	0.597	0.808	0.935	1.109
Chat	11,711	0.410	0.757	0.919	1.119
Code	7,720	0.470	0.804	1.016	1.216
Mathematics	8,929	0.385	0.796	1.025	1.211
Multilingual	11,640	0.418	0.747	0.917	1.131
Draft step 1	10,069	0.385	0.667	0.864	1.092
Draft step 2	10,035	0.520	0.783	0.947	1.157
Draft step 3	9,871	0.611	0.823	0.982	1.190
Draft step 4	10,025	0.607	0.845	1.002	1.214
Context 0–255	19,604	0.385	0.767	0.949	1.165
Context 256–511	18,379	0.410	0.766	0.957	1.182
Context 512–1,023	1,733	0.445	0.781	0.972	1.175
Context $\geq 1,024$	284	0.516	0.782	0.967	1.169

C.6 Certification by bound family

Table 11 gives, per tiling, the best certified family, which is the elementwise minimum of all certified radii, and Table 12 summarizes both drafters over every family and tiling, with the share a real verify batch must still read. Table 13 gives every family on the tiling where transport did best, k -means tiles of 64 entries. The greedy skip hardly depends on the family: every certified family skips between 0.52% and 0.64% of the vocabulary on average, within 0.014 percentage points of the static screen with the same family. A row-level variant, which bounds each entry from its own draft score with its tile’s radius, skips 0.65% for MTP-4B on the same tiles, the largest certified skip measured [E24]. The family does matter for the partition interval of sampled acceptance, which the scalar ℓ_∞ radius leaves 820–901 nats wide and the grouped, low-rank and combined families narrow about sixfold, to 134–169 nats. No certified family brings it below 130 nats, and even oracle radii, at 12–13 nats, leave the acceptance test undecided most of the time (Appendix C.7). On these tiles, entry-level transport, which bounds each entry by its own deviation from the tile centre instead of the tile’s largest, adds at most 0.012 percentage points to the tile-level skip.

C.7 Sampled acceptance and batch unions

For sampled acceptance the relevant quantity is the probability $P_?$ that the guards of Theorem B.5 leave the test undecided. Transported masses leave it near one whichever way it is taken [E23,E24]. For DFlash-4B with the best certified family, its

Table 11. Transport on held-out draft pairs with the best certified bound family per tiling (the elementwise minimum of all certified radii). “Skipped”: mean share of the vocabulary a greedy verifier skips, by transport (T), by the static screen with the same geometry (S) and by oracle radii (O), the realized per-tile deviations that no certified geometry can undercut. “Width”: median partition width $\log(Z^+/Z^-)$ in nats; P_γ : mean probability, over proposals $x \sim q$ at $T = 1$, that the acceptance test stays undecided. Measured [E23,E24].

Tiling	Skipped (greedy)			Sampled	
	T	S	O	Width	P_γ
<i>DFlash-4B, block 16: 4,020 pairs, 110 prompts</i>					
Contiguous 64	0.08%	0.08%	89.3%	133	0.995
Contiguous 256	0.00%	0.00%	83.9%	213	1.000
Contiguous 1,024	0.00%	0.00%	76.9%	244	1.000
Random 256	0.00%	0.00%	76.0%	238	1.000
k -means 64	0.59%	0.58%	83.7%	134	0.964
k -means 256	0.10%	0.10%	77.0%	138	0.982
k -means 1,024	0.02%	0.02%	68.4%	146	0.991
<i>MTP-4B, four steps: 16,016 pairs, 160 prompts</i>					
Contiguous 64	0.08%	0.08%	98.5%	146	1.000
Contiguous 256	0.00%	0.00%	96.4%	233	1.000
Contiguous 1,024	0.00%	0.00%	92.9%	268	1.000
Random 256	0.00%	0.00%	94.7%	255	1.000
k -means 64	0.64%	0.64%	96.0%	146	0.998
k -means 256	0.11%	0.11%	92.3%	152	0.999
k -means 1,024	0.02%	0.02%	86.8%	162	1.000

Table 12. Transport on the pairs from whole held-out capture steps (DFlash-4B: 4,020 from 110 prompts; MTP-4B: 16,016 from 160), over every bound family and tiling tested (tiles of 64 to 1,024 entries). Single values are the best case; ranges span the tilings (by tiling: Table 11). “Skipped”: mean share of the vocabulary a greedy verifier skips; “read by a real verify batch”: the share in tiles that at least one position of the batch still needs; P_γ : mean probability, over proposals at $T = 1$, that the sampled acceptance test stays undecided. Measured [E23,E24].

	DFlash-4B	MTP-4B
Skipped: certified transport	0.59%	0.64%
Skipped: static screen (no draft)	0.58%	0.64%
Skipped: oracle radii	68.4–89.3%	86.8–98.5%
Read by a real verify batch	$\geq 99.48\%$	$\geq 99.44\%$
Sampled test undecided, P_γ	≥ 0.964	≥ 0.998

Table 13. Transport by bound family on k -means tiles of 64 entries: mean share of the vocabulary skipped for greedy verification (tile level), and the median width of the partition interval in nats at $T = 1$, for transport and for the static screen at the target state with the same geometry. Oracle radii are the realized per-tile deviations, a ceiling that no certified geometry reaches. Measured [E23,E24].

Bound family	DFlash-4B (4,020 pairs)				MTP-4B (16,016 pairs)			
	Skipped		$\log(Z^+/Z^-)$		Skipped		$\log(Z^+/Z^-)$	
	transport	static	transport	static	transport	static	transport	static
Scalar ℓ_∞ radius	0.52%	0.52%	820	890	0.53%	0.53%	901	930
Coordinate radii	0.59%	0.58%	287	312	0.64%	0.64%	315	324
Grouped ℓ_2 , groups of 32	0.53%	0.53%	158	173	0.53%	0.53%	169	176
Grouped ℓ_2 , groups of 128	0.53%	0.53%	144	158	0.53%	0.53%	152	159
Low rank, basis from W (64)	0.53%	0.53%	150	165	0.54%	0.53%	151	160
Low rank, basis from Δ (64)	0.54%	0.53%	135	146	0.55%	0.54%	147	157
Low rank, basis from Δ (256)	0.55%	0.54%	151	163	0.56%	0.55%	166	174
Minimum of all certified radii	0.59%	0.58%	134	146	0.64%	0.64%	146	156
Oracle radii (not certified)	83.70%	89.74%	13	17	96.02%	99.36%	12	23

mean is 0.92–0.96 on k -means 64-entry tiles, whether it is taken at the draft token or averaged over proposals $x \sim q$, at $T = 1$ or $T = 0.7$, and 0.95–1.00 on every other tiling; for MTP-4B it is at least 0.997 on every tiling. Retaining a small set R_g per tile tightens the transported mass bounds: recompute its target logits exactly and transport only the rest,

$$\sum_{i \in R_g} e^{z_i^t} + e^{a_g - \epsilon_g} Z_{g \setminus R_g}^d \leq Z_g^t \leq \sum_{i \in R_g} e^{z_i^t} + e^{a_g + \epsilon_g} Z_{g \setminus R_g}^d, \quad (7)$$

which is never looser than transporting the whole tile, at the cost of projecting $|R_g|$ more entries exactly. Computing the exact logits of the 64 entries of the whole vocabulary with the largest draft logits at each position (R_g being those in tile g) and transporting only the rest (Equation (7)) narrows the median partition interval to 110–125 nats on DFlash-4B and lowers P_γ at the draft token to 0.58 on average, with a median still of one (0.81 on MTP-4B). Even oracle radii leave the mean P_γ over proposals at 0.68–0.78 for DFlash-4B and 0.79–0.92 for MTP-4B across tilings, because their partition interval is still 12–18 nats wide at the median.

A skipped tile saves weight traffic only if no position of the batch needs it. If each position of a batch of B independently needed a tile with probability f , the batch would need it with probability

$$1 - (1 - f)^B, \quad (8)$$

about 87.3% at $f = 1/16$ and $B = 32$, so a per-position skip of 93.75% can coexist with an almost full weight stream. That is a model of independent masks; the measured unions follow. Over the capture’s real verify batches, the union of tiles left unresolved covers 99.48–100% of the vocabulary for every certified family and tiling on DFlash-4B (batches of 9 to 128 paired positions) and 99.4–100% on MTP-4B (1 to 32 positions). Oracle radii fare little better once positions are grouped: with k -means 256-entry tiles, random groups of 4, 16 and 64 DFlash-4B pairs need 63%, 97% and 99.9% of the vocabulary, and groups of 16 and 64 MTP-4B pairs need 62% and 92%. At the batch sizes the verifier serves, the union therefore removes most of the weight traffic even oracle radii would save.

C.8 The drift-scaling extrapolation

The replay estimates, with a model rather than a drafter, how far a drafter would have to move before transport became useful: it replaces h^d by $h^t - s\Delta$, which scales ρ , the logit drift and every radius term by s , and re-evaluates the best certified family on k -means 256-entry tiles. Table 14 gives the result. On DFlash-4B, greedy transport skips half the vocabulary on average only at a tenth of the measured drift, and the sampled acceptance test decides a third of proposals only at a hundredth. The MTP layer needs a somewhat smaller reduction for greedy transport (80% skipped on average at $s = 0.1$, against 49%) and the same for sampled acceptance.

Table 14. Drift-scaling extrapolation, a model and not a measured drafter: the draft input is replaced by $h^t - s\Delta$ and the best certified family on k -means 256-entry tiles is re-evaluated. Shares skipped are for greedy verification; the partition width is a median in nats and $\mathbb{E}_q P_\gamma$ a mean, both at $T = 1$. Derived from measured pairs [E23,E24].

Scale s	DFlash-4B				MTP-4B			
	skip, mean	skip, median	$\log(Z^+/Z^-)$	$\mathbb{E}_q P_\gamma$	skip, mean	skip, median	$\log(Z^+/Z^-)$	$\mathbb{E}_q P_\gamma$
1 (measured)	0.10%	0.09%	137.9	0.98	0.11%	0.11%	152.3	1.00
0.3	1.55%	0.27%	48.7	1.00	1.90%	0.38%	57.5	1.00
0.1	49.22%	49.38%	22.5	1.00	80.08%	96.23%	24.8	1.00
0.03	76.94%	96.49%	7.0	0.96	96.17%	99.77%	7.5	0.97
0.01	83.68%	99.10%	2.3	0.66	97.88%	99.85%	2.5	0.70

The extrapolation says nothing about whether any drafter can reach such a drift. A drafter trained with an auxiliary loss on $\|h^t - h^d\|_2^2 / \|h^t\|_2^2$ is the obvious test and is untried (research notes, Section 5.4).

D EXACTNESS CONTRACTS AND THE STOCK ENGINE

Section 2 names the stock head kernel at the same batch shape as the reference for the certified head. This appendix places that choice among the contracts a decoder can promise, lists what each consumer of the head needs, and gives the evidence behind the same-shape rule: where stock configurations that ought to agree already disagree. How the engine keeps its recurrent state under speculation, and the tests of that path, are in Section 6.2 of the research notes.

D.1 Five contracts and the witnesses that separate them

An exactness claim needs a reference and a sense of equality. The reference is an executable specification, not an idealized network: the weights, the kernels, the decoding policy, the random-number protocol and the execution shape, together with whether it is stock plain decoding, stock speculative verification or a recorded execution. Mathematical equivalence over the reals does not imply that two GPU reductions choose the same token, because a different batch shape or reduction order can move logits that are close (NVIDIA, 2026; PyTorch contributors, 2026); a certificate must therefore enclose the value the reference computes, casts included. Table 15 lists the contracts a change can claim against such a reference and the results of this paper that rest on each. They are ordered roughly from strongest to weakest, but none implies the next in general.

Table 15. Exactness contracts and the results that claim them. “Measured on replayed positions” means captured engine states replayed offline, not a served run.

Contract	What must be equal	Results that claim it	Status
Identical internal state	Named logical tensors (recurrent state, attention cache) at named boundaries, bitwise; not layouts or scratch buffers	None in this paper: the certified head reads the hidden state and writes no state	Not claimed here
Identical greedy outputs	Decisions, ties and stopping, with a valid state from which to continue	Certified head under R-stock (Theorem 2.3, Proposition 2.4)	Proved under an error model; measured on replayed positions, through the kernel on 60,000 of them [E15]; in SGLang at the same batch shape on every certified greedy path [E20]
Identical seeded samples	The reference’s mapping from seed to output; with an inverse-CDF reference, certify with CDF bounds against the same uniform and fall back without redrawing	Certified race with the stock seeded field; fixed-noise verification against the seeded sampler on the verify pass’s logits (Section 3.3)	Proved; on 60,000 replayed positions at $T = 0.7$ and 1.0 , without top- k or top- p , the certified sampler with its fallback equals the stock seeded sampler on every position [E19]; in SGLang, fixed-noise verification returned SGLang’s seeded token for the verify pass’s logits on all 21,016 verification positions of the check [E20]; acceptance against the stock rejection-sampling verifier not measured
Exact sampling distribution	A named law: the ideal model’s, the categorical law of the reference’s scores, or the finite random-number implementation’s; speculative correction targets the verifier’s own p	Races (Theorem B.2); fixed-noise verification in law; residual races (Appendix B.4)	Proved
Empirical quality equivalence	Task scores and distances within declared margins, with confidence intervals	None in this paper	Not claimed here

Small exact witnesses separate the contracts [E31]. Reassociating an FP32 sum flips a greedy winner: $1 + 2^{-24} + 2^{-24}$ summed left to right returns 1 and ties a competing logit of 1, so the first-index rule picks the competitor, while the regrouped sum returns $1 + 2^{-23}$ and wins, as it does in real arithmetic. Distinct FP32 values, 1 and $1 + 2^{-10}$, fall in one BF16 rounding cell, so a BF16-output head ties them and the index rule decides. Logits $(1, 0)$ and $(2, 0)$ have the same greedy decision but different laws: $(\frac{2}{3}, \frac{1}{3})$ against $(\frac{4}{5}, \frac{1}{5})$ in the base-2 mass model, and 0.731 against 0.881 on the first token at temperature one. Two inverse-CDF samplers that list the tokens of $p = (\frac{1}{2}, \frac{1}{2})$ in different orders have the same law but map the uniform

$\frac{1}{4}$ to different tokens. Evaluating the gated-delta recurrence $S_t = S_{t-1}A_t + B_t$ in the split form $S_t = S_0P_t + W_t$, which is equal in exact arithmetic, changed 1,481 of 1,600 FP32 outputs over 50 steps at width 32, and 2 of them after rounding to BF16. A state difference the current query cannot see becomes visible after one recurrent step: in the gated-delta update with $\alpha = \beta = \frac{1}{2}$, key $k = (\frac{3}{5}, \frac{4}{5})$ and query $q = e_1$, a difference $\Delta S = ue_2^\top$ gives $\Delta S q = 0$ now and $-\frac{3}{25}u$ one step later [E32], so equal outputs now do not imply equal state. Two greedy decoders that each answer two of four problems correctly have equal accuracy, yet their answers can differ on every problem, so empirical quality equivalence implies none of the stronger contracts. Equal token sequences, finally, say nothing about latency or about whether the cache that produced them is owned correctly.

A last witness, in the exact floating-point reference, separates R-bf16 from R-stock: with $w_0 = (16, 0, 0)$, $w_1 = (16, \frac{1}{16}, 2^{-30})$ and $h = (1, 1, 1)$, the real logits are 16 and $16.0625 + 2^{-30}$, so R-bf16 rounds them to 16 and 16.125 and picks token 1, while a sequential FP32 accumulation loses the 2^{-30} , rounds 16.0625 to even and returns 16 for both entries, so the stock argmax picks token 0; the gap condition refuses to certify this position [E28]. The floating-point witnesses use IEEE binary32 with round to nearest and every reduction written as an explicit sequential loop; a kernel that contracts, reorders or accumulates differently can round differently. They show that the contracts are distinct in general and say nothing about how often the differences occur in the engine.

D.2 The commit rule in general

Every certificate in this paper instantiates one rule. Partial computation yields information I about the final logits: an approximate pass with error bounds, a transported summary, or an approximate hidden state with a radius. From I we keep a *sound set* $Z(I) \subseteq \mathbb{R}^V$ that contains the reference implementation’s logits. A decision rule $D(z, R)$ maps logits and the randomness fixed by the contract (a noise field, a uniform draw) to a token or an accept bit.

Definition D.1 (Commit rule). Commit the decision d when $\{D(z, R) : z \in Z(I)\} = \{d\}$. Otherwise acquire more information, which must shrink or replace $Z(I)$ by another sound set, and test again.

If the reference logits z^* lie in $Z(I)$, a committed d equals $D(z^*, R)$; that is the whole soundness argument, and it holds whatever produced $Z(I)$. For a box $Z(I) = \prod_j [l_j, u_j]$ the greedy rule commits a when $l_a > \max_{j \neq a} u_j$, and a Gumbel race with a fixed field g commits a when $l_a/T + g_a > \max_{j \neq a} (u_j/T + g_j)$, which needs no partition function as long as the field stays fixed through every refinement (Maddison et al., 2014). The rule names the reference; it is the gap condition of Theorem 2.3 that turns a sound set for the real logits into one for the stock kernel’s decision.

D.3 What each consumer of the head needs

Table 16 lists the information each consumer of the head requires. A fast path is admitted by these contracts, not by a request being labelled “decode”. Grammar masks, repetition penalties and temperature transforms are part of the scores; they cannot be applied after candidates have been discarded.

D.4 Where equivalent stock configurations disagree

The contract compares a certified head with the stock head only at the same batch shape and in the same serving configuration, and a speculative configuration with stock speculative verification (Section 4.1). This subsection and the next three give the evidence for that rule: stock configurations that ought to agree already emit different greedy tokens, namely plain decoding at different batch sizes, plain decoding against MTP speculation, and speculation at different batch sizes. Both greedy paths reduce to one rule, `torch.argmax` (first maximal index) over the BF16 head output widened exactly to FP32, so two runs that feed identical logits to it cannot choose different tokens. Every divergence therefore means that the computed logits differed, and the question is where. “A tie” is an observation about where the divergence became visible, not an explanation of why it happened. Batch-invariant kernels make greedy outputs reproducible across batch compositions (He & Thinking Machines Lab, 2025), and floating-point nondeterminism in inference has been characterized and partly mitigated (Yuan et al., 2025); what follows locates the kernels responsible in this engine. The target’s decoder layers mix full attention with Gated DeltaNet (GDN) layers, a form of linear attention with a recurrent state (Yang et al., 2025).

Table 16. Consumers of the head and what each needs. The last rows are where a narrow fast path usually loses.

Consumer	Required information	Certificate in this paper
Greedy token	Winning index under the reference kernel and tie rule	Ladder plus stock contract (Theorems A.6 and 2.3)
Exact top- k	Winning indices in order, and their scores; no partition function	Ladder on membership and on the order within the set (Appendix B.1)
Seeded or fixed-noise sample, full vocabulary	Argmax of perturbed scores under the token-keyed field	Certified race (Theorem B.2)
Seeded top- k or top- p sample (stock)	Each candidate’s rank in the sorted probabilities, which keys its noise, and the sort’s order on ties; for top- p , the mass above each token	Membership and complete order within the set, then the race (Appendix B.1; analysis)
Acceptance bit	$\exp(z_X)$ against Uq_XZ ; an interval can suffice	Guards (Appendix B.3; analysis). The engine path uses fixed-noise verification instead
Residual sample	The $(p - q)_+$ law, including omitted mass	Residual races (Appendix B.4; analysis)
Bonus sample	A sample from the target law under the random-number contract	Certified race
Top- p sample	Mass above each token relative to Z	Partition bounds, or the ordinary path
Arbitrary log-probabilities	Requested logits and the full partition function	None; the ordinary path

In brief:

- neither the tie rule nor the head GEMM caused a divergence [E33];
- plain decoding and MTP speculation part inside layer 0’s recurrence, whose decode and verify kernels produce different bits from identical inputs and state [E36];
- plain decoding at two concurrencies parts at batch-dependent kernels [E33];
- with identical pools, MTP diverges from plain decoding at 3.5 to 4.0 per 1,000 compared tokens, always at an exact tie or within two BF16 spacings [E38];
- with the prefix cache on, a request’s output can depend on which earlier request computed its shared prefix and on what the cache holds when it is prefilled [E40,E36], two routes that Section 6.1 of the research notes traces.

Differential tests A fixed set of 320 prompts, the *divergence prompts*, from GSM8K, HumanEval, MT-Bench, AlpacaEval and CNN/DailyMail at pinned revisions, tokenized once so that every configuration receives identical token IDs (105 of them in the model’s thinking mode), is decoded greedily for up to 256 tokens with the top five log-probabilities at every position. The configurations differ only in settings that should leave greedy output unchanged: plain decoding against MTP speculation (one, three and five draft steps with top- k 1, and a small tree with top- k 2), the prefix cache and the overlap scheduler on or off, `--enable-deterministic-inference` and `--enable-fp32-lm-head`, each with one request in flight and with a client concurrency of 32. Two sets of runs were made, named by their cap on running requests. In the *cap-16 runs* every server was capped at 16 running requests and sized its memory pools from the memory free at start-up, so “concurrency 32” meant decode batches of up to 16 with the rest queued; the *cap-8 runs* below cap every server at 8 with identical, fixed pools. Two runs of a prompt are compared position by position; the first position where the tokens differ is a divergence, the positions before it are the exposure, and the rate per 1,000 tokens is divergences over exposure. Every run uses the engine at the pinned SGLang commit (Appendix F.3) with CUDA graphs, the overlap scheduler and the FlashInfer attention backend [E34].

D.5 Where the outputs first differ

The tap A diagnostic tap, an engine patch,¹ hashes the exact bits of every module output of the target, for each token, and two points inside the GDN backend (after the causal convolution and after the recurrent kernel), into static device buffers that the CUDA graphs capture and replay; for selected requests it also saves the head input and the logits fed to the argmax. For each divergence both configurations are replayed on the prompt and the committed positions (for MTP, only verification positions on the accepted path) are walked in order to the first position and, in execution order, the first module whose output

¹engine/sclang/patches/state/0001-state-tap.patch; Appendix F.3 lists the patch series.

bits differ. Before that point every hashed module output agrees. The caches (attention KV, the GDN convolution window and the recurrent state) are not module outputs, so a difference written into a cache without a differing output would first show in the module that reads it.

The cache-hashing tap A later version of the tap closes that gap: it also hashes the caches each forward reads, the keys and values of every attention layer at every cached position and the convolution window and recurrent state of every GDN layer [E36]. The comparison skips the caches of a prefill with no cached prefix, which reads no earlier state: its recurrent slot can still hold an earlier request’s state when the tap hashes it, because the engine clears the slot inside the forward. This version has been run at batch 1 for plain decoding against MTP, for the history pairs, for the prefix cache on against off and for one prompt’s same-server repeats (below); and for the concurrency pair with fixed pools (below) [E36].

Checks of the tap The tool was checked on itself: a one-ulp change injected into layer 9’s MLP down-projection output is named at that module, at the first prompt token, in 4 of 4 prompts, and one injected into layer 13’s GDN recurrence output is named at that operation in 4 of 4 when the search starts at the first decode position; searched from the prompt, the second is attributed to the enclosing GDN attention module, because in prefill the attention backends run eagerly between captured graph segments [E35]. Tapped runs match untapped ones in tokens and log-probabilities for 162 of 167 prompts with the tap version behind Figure 6 [E33], and for 35 of 40 with the cache-hashing version, the other five being the same five prompts [E36]. Those tapped and untapped servers had different pools, so the five are not attributed to the tap itself; Section 6.1 of the research notes follows them up [E37]. In the concurrency-pair rerun, with the same fixed pools and engine, the tapped concurrency-1 session matches an untapped pass on all 40 tapped prompts [E36]. Outside the cap-8 runs below and the fixed-pool rerun of the concurrency pair, every comparison in this section across two servers had different pools, while the concurrency comparison, the same-server repeats and the deterministic-mode checks compare passes on one server.

Findings that do not depend on an error model Two pairs have been analysed [E33]. First, neither the tie rule nor the head GEMM caused a single divergence: in every case the head input itself already differed. Second, the first differing module output is specific to each pair (Figure 6).

Plain decoding against MTP For plain decoding against MTP speculation with three draft steps, both at batch 1, it is layer 0’s GDN recurrence output at the first speculative cycle, in all 167 prompts analysed, while that layer’s causal-convolution output, the recurrence’s immediate input, is bitwise equal. At the pinned commit, decode and target verify call different Triton kernels (`fused_recurrent_gated_delta_rule_packed_decode` and `fused_sigmoid_gating_delta_rule_update`), which fuse the gating and the state update differently. Rerun with the cache-hashing tap on 40 of these prompts, every cache entering every forward is identical up to that output, and the first cache to differ is layer 0’s recurrent state, which the differing kernel writes [E36]; the two kernels therefore produce different bits from identical inputs and identical state. The two tapped servers had different pools, and MTP’s draft layer attends over its own cached prefix, so a different copy of a shared prefix could change the proposals, the accepted lengths and the later verify forwards without any target attention output differing first. The location of the first difference bounds the pools only for that difference, which is at output index 1, the first position of the first verify forward, whose layer-0 recurrence output depends on its token and on the state left by the prefill, not on the proposals or on cached attention values (reasoning from the row-wise structure of the kernels); whether the pools contributed to the divergences that follow is not known.

Plain decoding at two concurrencies For plain decoding at concurrency 1 against 32 (batches of up to 16; 40 tapped prompts), it is a GDN gated RMSNorm in decode in 29 prompts, a full-attention layer’s output in decode in 6, and the MLP down-projection of a prefill batched with other requests in 5; never the GDN recurrence and never a decode GEMM. These two tapped servers also had different pools. A different copy of a shared prefix’s attention values would first show at a full-attention output, so for the 6 prompts that first differ there the pools and the prefix cache’s history are possible causes besides the attention kernel’s dependence on the batch; for the other 34, every attention output before the first difference agrees. The gated norm chooses how many positions each program reduces from the batch’s size (one per program for a batch of 1, two for a batch of 16), which changes the reduction tile and so the FP32 sum order of a position; the same function returns a constant in batch-invariant mode. FlashInfer’s decode plan and cuBLAS’s GEMM choice also depend on the batch. Rerun with the cache-hashing tap and identical fixed pools (at most 16 running, 98,304 attention-cache tokens, 80 recurrent-state slots), with both sessions serving all 320 prompts in the same order, all 40 prompts diverge again, and in every one the caches entering every forward are identical up to the first differing module output: a gated norm in decode in 30, layer 3’s attention in decode in 5 and the batched-prefill down-projection in 5 [E36]. The 5 attention cases are 5 of the 6

above and enter with identical attention values, so FlashInfer’s decode attention itself gives different bits in a batch of 15 or 16 than alone; the sixth, `mt_bench-0056`, first differs at a gated norm at output index 29 once both sessions share one request history. Classified at the divergence, none comes from the tie rule or the head GEMM; under the conservative model 3 are rounding flips, 16 order flips and 21 ambiguous, and under the Hopper model 14, 17 and 9.

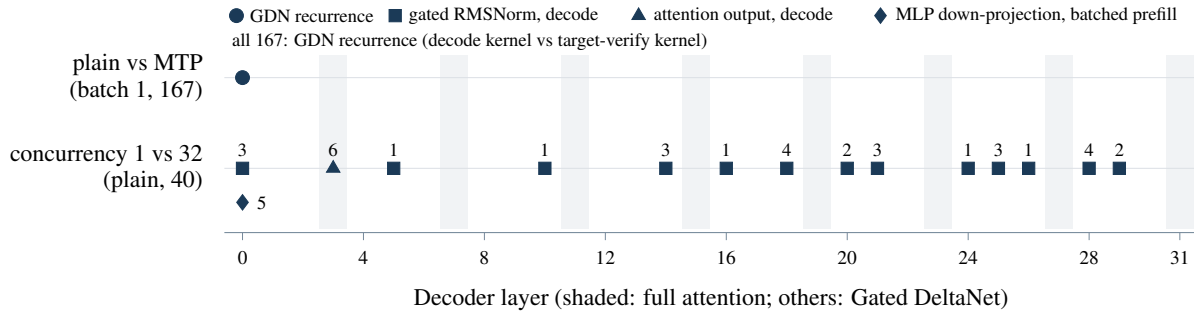


Figure 6. Where equivalent stock configurations first produce different module outputs, by decoder layer; each number counts the prompts at its mark. Plain decoding and MTP speculation part at layer 0’s GDN recurrence in every prompt, because decode and target verify use different recurrent kernels; plain decoding at concurrency 1 and 32 (batches of up to 16) part at batch-dependent kernels across the depth. The plotted runs’ two tapped servers had different pools in each pair. Reruns of 40 prompts that also hash the caches find them identical on entry: for plain decoding against MTP, every cache entering the differing recurrence; for the concurrency pair, with identical fixed pools, every cache entering every forward up to the first differing module output (gated norm 30, layer 3’s attention 5, batched-prefill down-projection 5, against 29, 6 and 5 plotted). Measured with a bit-exact tap [E33,E36].

D.6 How often they disagree

Table 17 gives how often configurations that ought to agree diverge in tokens [E34,E38]. In the cap-16 runs, plain decoding at concurrency 1 against 32, two passes on one server with identical pools, diverges in 167 of 320 prompts within 256 tokens, 3.42 divergences per 1,000 compared tokens, with a largest margin at the divergence of 0.375 nats; this is the batch-shape floor. Repeating either configuration on the same server reproduced every token of all 320 prompts, and every log-probability except those of one 128-token prompt, which differ from its prefill onward in both repeats. A fresh server at batch 1 reproduced the first pass bitwise on all 320 prompts, that one included, so its difference on the same server comes from what the prefix cache already held, and the cache-hashing tap places it in the recurrent prefill (research notes, Section 6.1). A fresh server at concurrency 32 reproduced 296 of the 320 outputs; its session differed from the first in request arrival, and with it batch composition, and also in its pools [E37]. On these prompts plain decoding at batch 1 has an exact BF16 tie between its top two logits at 11.3 of every 1,000 positions and a nonzero gap of at most 0.125 at another 22.2, and no run committed a token that was not its own top-1 [E34]; the 1.0% of Section 1 is on the capture prompts.

Deterministic mode SGLang’s deterministic mode replaces the default matrix multiplication with a batch-invariant one (DeepGEMM’s deterministic BF16 GEMM with the pinned commit’s defaults on this GPU, a persistent Triton kernel otherwise) (The SGLang Team, 2025) and, with the FlashInfer backend, also switches sampling to PyTorch and disables the prefix cache. It made plain decoding batch-invariant in an 8-prompt, 128-token check (concurrency 1 against 8: identical tokens and bitwise-identical log-probabilities) but not MTP speculation (2 of 8 prompts diverged). Passing the deterministic KV split to FlashInfer’s target-verify plan, as an engine patch² behind an environment switch that is off by default and was on for this run, left 28 of 96 MTP prompts diverging between concurrency 1 and 32, 1.55 per 1,000 compared tokens, all at exact ties [E34]. A plausible reason, not tested, is that the draft is not batch-invariant, so accepted lengths, and with them a position’s offset inside its verify block, differ between batch sizes.

With fixed pools The cap-8 runs fix the pools: every server is capped at 8 running requests with 49,152 attention-cache tokens and 40 recurrent-state slots, each was checked to have allocated exactly those, and every pair compares identical pools [E38,E37]. These runs replace the cap-16 comparisons for every pair they cover. With decode batches of at most 8 instead of 16, their floor of 0.63 per 1,000 is not comparable with the 3.42 of the cap-16 runs, and a same-server repeat no longer reproduces every token, so with the prefix cache on it is not an equality reference; Section 6.1 of the research notes

²engine/sclang/patches/state/0002-verify-kv-split-deterministic.patch.

Table 17. How often stock configurations that ought to agree diverge in tokens on the 320 divergence prompts within 256 tokens: prompts whose tokens diverge, divergences per 1,000 compared tokens, and the class of the divergences. Cap-16 runs: servers capped at 16 running requests, pools sized at start-up; cap-8 runs: capped at 8 with identical, fixed pools. “One spacing”: both runs’ top-two margins within one BF16 spacing. Measured [E34,E38].

Pair	Diverged	Per 1,000	At the divergence
<i>Cap-16 runs</i>			
Plain, concurrency 1 vs 32, one server (the floor)	167	3.42	margin at most 0.375 nats
Plain, same-server repeat at either concurrency	0	0	one prompt’s log-probabilities differ
Plain, fresh-server repeat at batch 1	0	0	bitwise identical
Plain, fresh-server repeat at concurrency 32	24	0.35	exact ties
<i>Cap-8 runs</i>			
Plain, fresh-server repeat at concurrency 1	0	0	bitwise identical
Plain, fresh-server repeat at concurrency 32	4	0.06	exact ties
Plain, concurrency 1 vs 32, one server (the floor)	41	0.63	39 exact ties, 2 within one spacing
Plain, same-server repeat at concurrency 1	17	0.25	exact ties or within one spacing
Plain, same-server repeat at concurrency 32	4	0.06	exact ties
MTP (one, three or five steps, or a tree) vs plain, concurrency 1 or 32	164–181	3.5–4.0	exact ties, or within one or two spacings (text)
MTP, concurrency 1 vs 32	152–165	3.1–3.5	exact ties, or within one or two spacings

gives the explanation. Every MTP configuration, with one, three and five draft steps at top- k 1 and the tree with three steps at top- k 2, diverges from plain decoding at both concurrencies about as often as it diverges from itself between concurrency 1 and 32. At concurrency 1 no MTP output is bitwise identical to plain decoding’s: the log-probabilities of 313 or 314 of the 320 prompts first differ at output index 1, the first verify forward, and the rest at index 2 or 3, which is consistent with the decode and verify recurrent kernels parting there, although only the tapped prompts locate the kernel. In every cap-8 pair each divergence is at an exact tie in one run or has both margins within one BF16 spacing, or within 0.25 nats, two spacings, for 11 events in 5 prompts; none falls outside these classes, no run committed a token that was not its own top-1, and log-probability drift without a token change is at most 0.62 nats [E38].

Divergence given perturbation A perturbation analysis of the same runs separates how many prompts a change perturbs at all, meaning that their two outputs are not bitwise identical in tokens and top-five log-probabilities, from how often a perturbed output then diverges in tokens [E39]. Plain decoding’s batch shape perturbs 75 of the 320 prompts at a cap of 8, the other 245 being bitwise identical between concurrency 1 and 32, and all 320 at a cap of 16, while speculation perturbs all 320 against plain decoding, at concurrency 1 from the first verify forward. Nearly all perturbed prompts first differ within the first 32 output tokens. Among those, the share whose tokens diverge is 35 of 62 (0.56, 95% interval 0.44 to 0.68) for plain decoding at a cap of 8, 28 of 50 (0.56, 0.42 to 0.69) for plain decoding at a cap of 8 without the prefix cache, 165 of 318 (0.52, 0.46 to 0.57) at a cap of 16, and 0.51 to 0.57 for every MTP configuration against plain decoding at concurrency 1 and 32; from the first difference on, divergences occur at 3.5 to 4.1 per 1,000 tokens in all of them.

The gap between the cap-8 floor (0.63 per 1,000 compared tokens) and the speculative rate (3.5 to 4.0) is therefore mainly the number of prompts perturbed, and on this measure no excess of speculation over any of the three floors is detected. With 62 prompts in that onset range at a cap of 8 the interval is wide, so a modest difference is not excluded, and the intervals assume independent prompts. The conditional rate is not the same for every source of perturbation: over all pairs with at least 30 perturbed prompts the share ranges from 0.29 to 0.60, lowest for MTP under deterministic inference with the verify KV-split patch (28 of 96, 0.29, 95% interval 0.21 to 0.39, unexplained) and for the FP32 head (130 of 320, which has no exact BF16 ties to flip).

D.7 Consequences for the contract

A certified head is compared with the stock head at the same batch shape and in the same serving configuration (Section 4.1). And the reference for a speculative arm is stock speculative verification at the same shape, not plain decoding: greedy speculative verification is exact in real arithmetic, but its verify pass runs different kernels from plain decoding, and on every prompt analysed with MTP their outputs already differ at layer 0’s recurrence in the first speculative cycle. The DFlash capture is consistent with the same effect at the level of whole outputs: only 39% of its prompts produced output identical to plain decoding (Appendix C.4), though its divergences were not traced to a kernel. The noise floor applies to fixed-noise

sampling in the same way (Section 3.3).

D.8 Engine code paths at the pinned commit

The statements about the stock engine in this paper come from reading SGLang at the pinned commit (Appendix F.3), a fixed commit rather than the latest revision. Table 18 lists the code paths that decide them and where a certified head would attach; the integration itself is not part of this paper’s evidence.

Table 18. Code paths of the stock engine behind the paper’s statements about it, read in the source at the pinned SGLang commit; observations, not measurements.

Path under python/sglang/	Observation
srt/layers/logits_processor.py	<code>_compute_lm_head</code> is the single hook through which plain decoding, the MTP drafter and the verifier compute logits: <code>torch.matmul</code> with BF16 output by default, <code>torch.mm(..., out_dtype=float32)</code> under <code>--enable-fp32-lm-head</code> . A certified head would replace this call for admitted consumers only; log-probability requests and unsupported transforms keep the ordinary path.
srt/models/qwen3_5_text.py	The Qwen3.5 head is tied to <code>embed_tokens</code> , which is not a <code>ParallelLMHead</code> , so SGLang’s quantized-head paths do not apply; a low-precision copy must be separate, and the BF16 head stays resident for re-scoring and fallback.
srt/layers/sampler.py	Unseeded sampling is an exponential race; seeded sampling over the full vocabulary is a Gumbel race keyed by (seed, position, token) in FP64 (<code>multinomial_with_seed</code>), and the certified race uses the same field. The seeded top- k and top- p path keys the same hash by position in the sorted probabilities instead, and the sampler refuses min- p with a seed (SGLang contributors, 2026).
srt/models/dflash.py	DFlash projects draft hidden states through the target head for its candidates; this is where transport’s draft-side summaries would be produced.

E WHERE THE TIME GOES, AND THE SERVING BASELINE

This appendix measures the stock engine, the baseline a certified head has to improve, and then the engine changes timed against it. A component that occupies a fraction f of a step can speed the step up by at most $1/(1 - f)$, so the head’s share of the time bounds what any head certificate can deliver. Appendices E.1 and E.2 measure where the time of a plain decode step and of a speculative cycle goes, and Appendix E.3 says how the stock serving frontier of Section 4.5 is measured. Appendix E.4 composes three exact engine changes on DFlash, the certified head among them, and Appendix E.5 serves the certified head alone on four tuned arms. The serving protocol in full, what the other layers of the stack could deliver, the first lever measurements and the proposals for writing the recurrent state less often or sharing it across requests are in the research notes (the protocol in their Section 4, the rest in Section 3).

E.1 Bytes per step

Decode on this GPU streams bytes, so the byte counts of Table 19 give a first picture before any trace. A plain decode step streams every decoder weight once plus the whole head, while the embedding lookup reads one entry per request, so the head is 15.1% of the weight bytes of a plain step. An MTP draft step runs one small layer followed by the same head, so there the head is 84.1% of the weight bytes. The recurrent state is per request: the configuration stores it in FP32 (`mamba_ssm_dtype`), so each request holds 50.3 MB of GDN state, and a decode kernel that reads and writes it once per step moves 100.7 MB per request per step, which equals the weight bytes of a step at a batch of 84 requests.

E.2 Measured attribution

Method The attribution uses the production configuration (CUDA graphs for prefill and decode, the overlap scheduler, FlashInfer attention, greedy decoding; Appendix F.3) with plain decoding at batch sizes 1, 8, 32 and 128 and MTP speculation (three steps, top- k 1, four draft tokens, otherwise default flags, not tuned) at 1, 8 and 32. For each batch size B the client holds B streaming requests with chat-templated essay prompts of 26–36 tokens and a budget large enough that none finishes, and a two-second window is profiled once decode has settled, so the batch is exactly B and no prefill falls inside the window; mean contexts in the plain windows were 987, 862, 688 and 418 tokens. Nsight Systems (NVIDIA, 2025) traces each window with node-level CUDA-graph tracing. Each step is reassembled from the correlation identifiers of

Table 19. Byte counts for Qwen3.5-4B at revision 851bf6e806ef, derived from the checkpoint’s safetensors headers and config.json (Qwen, 2026b); calculations, not measurements.

Component	Bytes	Derivation
Tied head (and input embedding)	1.271 GB	$248,320 \times 2,560 \times 2$ (BF16)
Decoder weights other than the embedding	7.140 GB	sum of tensor sizes in the checkpoint
Weights streamed per plain decode step	8.411 GB	decoder weights plus the head; head share 15.1%
MTP layer	0.241 GB	sum of tensor sizes; a draft step streams it and the head (head share 84.1%)
Final decoder layer’s MLP	0.142 GB	$3 \times 2,560 \times 9,216 \times 2$
Vision tower (loaded, unused)	0.667 GB	sum of tensor sizes
GDN recurrent state per request	50.3 MB	$24 \times 32 \times 128 \times 128 \times 4$ (FP32)
Attention KV per context token	32 KiB	$8 \times 4 \times 256 \times 2 \times 2$ (BF16 K and V)

its graph replays: a plain step runs from one target-graph replay to the next, and a speculative cycle from one draft-graph replay to the next. Kernels are grouped by name, GEMMs are labelled by their neighbours in the replay, and the labels were checked structurally against the per-step counts the model implies (32 MLP gate/up and down, 48 GDN input and 24 output projections, 8 attention projections of each kind and one head per replay). Where kernels on two streams overlap, each instant is split equally among them, so the categories and the idle time sum exactly to the step [E42]. Every configuration was also measured with no profiler attached, three windows each, which gives the throughput numbers: plain decoding produced 289, 2,048, 6,298 and 14,388 output tokens per second at $B = 1, 8, 32$ and 128 (3.46, 3.91, 5.08 and 8.90 ms per step), with a standard deviation of 0.1–0.4% of the mean across windows. The collected plain steps are within 2.3% of the uncollected ones, so their kernel shares are reported directly. The bandwidth reference is measured on this GPU: the best of nine launch configurations of a read kernel sustains 3.83 TB/s over 4 GiB (the nine span 3.59–3.83 TB/s), the same configuration reads exactly the head’s 1.27 GB at 3.79 TB/s, and the attribution uses the latter as the peak [E44].

Table 20. Where a plain decode step goes, from Nsight Systems windows of 227–572 steps at the stated batch sizes, CUDA graphs and the overlap scheduler on, greedy decoding. Measured [E42].

Share of a plain decode step	$B = 1$	$B = 8$	$B = 32$	$B = 128$
Step under the profiler (μ s)	3,541	3,982	5,128	8,886
Head chain (head GEMM, FP32 copy of its logits, argmax)	10.3%	9.4%	7.8%	6.1%
Weight GEMMs (GDN, attention and MLP projections)	68.0%	64.5%	54.0%	34.5%
GDN recurrent kernel	3.5%	6.8%	18.0%	41.6%
Other GDN kernels (fused convolution, gated norm, state tracking)	2.6%	2.7%	3.3%	3.8%
Attention kernels, QK norm and RoPE, gate, KV store	4.8%	5.5%	7.2%	8.5%
Norms, activation, small kernels and copies	6.5%	7.1%	6.7%	3.8%
GPU idle, inside and outside graph replays	4.3%	4.0%	3.0%	1.7%

Plain decoding Table 20 gives the attribution. The head chain takes 10.3% of a step at $B = 1$ and 6.1% at $B = 128$ [E43], below the head’s 15.1% share of the weight bytes, for two reasons visible in the table. The head GEMM is the most efficient kernel in the step: it streams the 1.27 GB weight at 3.3–3.6 TB/s, while the backbone’s weight GEMMs reach 1.6–3.2 TB/s, the split- K projections from 4,096 to 2,560 features being the slowest. And the step contains work that does not scale with the weights: the recurrent state, attention, 65 RMSNorm launches and idle time.

The stock head in isolation Replaying the exact calls SGLang makes with the real tied weight under CUDA graphs, the head GEMM takes 354.5 μ s at $M = 1$, 362.0 μ s at $M = 32$ and 569.2 μ s at $M = 256$, and the whole chain with its FP32 copy and argmax 369.0 μ s at $M = 1$ (the certified head’s microbenchmark, a separate harness, measures the same chain at 370.5 μ s [E17]); up to $M = 32$ it runs at 93–95% of the measured read bandwidth, and reading the BF16 weight once at that bandwidth takes 335 μ s. cuBLAS selects an `nvjet_sm90_tst` kernel whose tile depends on M . At $M \geq 64$ the FP32 copy and the argmax, each re-reading an $M \times 248,320$ tensor, add 22–50% on top of the GEMM (the whole chain takes 558.6 μ s at $M = 128$ and 850.6 μ s at $M = 256$). Cold L2 changes the GEMM by under 1 μ s up to $M = 128$ (5 μ s at $M = 256$), since the weight is 21 times the 60 MB L2 [E43]. This chain is the comparator a certified head must beat, and a kernel that reads the same BF16 bytes has at most a few percent to gain at small M . A pass that reads half the bytes therefore needs at least about 170 μ s at $M = 1$ (derived) [E44].

Table 21. Where a speculative cycle goes with the native MTP layer (untuned: three steps, top- k 1, four draft tokens). Shares are of the cycle under the profiler; the idle share is inflated by the profiler, which lengthens this host-bound cycle by 25%, 19% and 11% at $B = 1, 8$ and 32 (first two rows). Measured [E42,E43].

MTP speculation, three draft steps	$B = 1$	$B = 8$	$B = 32$
Cycle under the profiler (μ s)	8,407	9,833	13,886
Cycle without a profiler (ms, mean of 3 windows)	6.70	8.27	12.49
Output tokens/s without a profiler (plain decoding at the same B)	384 (289)	2,573 (2,048)	7,087 (6,298)
Accepted length, tokens per cycle (windows without a profiler)	2.57	2.66	2.77
Target head chain (verify GEMM over $4B$ positions, FP32 copy, argmax)	4.4%	4.1%	3.9%
Draft head chains (three draft GEMMs, FP32 copies, top-1, extend argmax)	12.9%	11.3%	8.5%
MTP layer, excluding its head	4.6%	4.4%	3.7%
Target weight GEMMs	29.6%	28.4%	22.2%
GDN verify kernel (writes four FP32 states per request)	1.4%	5.5%	18.3%
GDN state commit (scatter of the accepted state)	0.5%	3.2%	8.9%
All other kernels (remainder)	9.3%	10.4%	11.0%
GPU idle, inside and outside graph replays	37.3%	32.7%	23.5%

MTP speculation A cycle replays a draft graph (the MTP layer twice, each time followed by the full head, the FP32 copy and a split-vocabulary top-1), a target-verify graph over four tokens per request, and a draft-extend graph (the MTP layer over the verified tokens and the head once more), with verification and the recurrent-state commit running eagerly between them. Every cycle therefore streams the 1.27 GB head four times. Table 21 gives the attribution. The head takes 17.2%, 15.3% and 12.4% of a cycle at $B = 1, 8$ and 32, three quarters of it in the draft heads; because much of the cycle is idle, its share of the time the GPU is busy is larger, 27.5% at $B = 1$ [E43]. Most of the idle time is host work in the attention planning of the verify and draft passes [E45]; Section 3.6 of the research notes traces it and removes part of it. The profiler inflates this gap, chiefly because each graph launch costs about 350 μ s under node-level tracing; subtracting the traced GPU-busy time from the cycle measured without a profiler leaves about 1.4, 1.7 and 1.9 ms per cycle idle (21%, 20% and 15%; derived from two measurements). Without a profiler, MTP produced 1.33 $\times$, 1.26 $\times$ and 1.13 $\times$ the plain throughput at $B = 1, 8$ and 32 in these fixed-batch windows, which contain no prefill.

DFlash speculation The tuned DFlash arm with 16-token blocks, Triton attention for the target and the drafter (Section 4.5), spends its cycle differently [E46]. A cycle replays the target-verify graph (every layer over the block, then the head over all 16 block positions) and, after the eager acceptance and state commit, the draft graph (the drafter’s six layers, then its projection through the target head over the 15 draft positions), so it streams the head twice. At $B = 1$ the verify and draft heads take 4.8% and 4.7% of the traced cycle, each a 360 μ s GEMM that streams the weight once (Figure 1). Over the untraced cycle, itself an estimate, that is 9.3% (derived), so removing both heads could make the cycle at most 1.10 $\times$ faster, and a head that read half the weight bytes at most 1.05 $\times$ (both derived; at most 1.08–1.10 $\times$ at $B = 4$ to 64). The target’s verify forward takes 66% of the traced cycle at $B = 1$ and 73% at 64, and two parts of it change with the batch. SGLang’s Triton extend-attention kernel runs one thread block per query tile and head, 16 in the target and 32 in the drafter at $B = 1$ on a GPU with 132 streaming multiprocessors; with the drafter’s share it takes 30% of the traced cycle at $B = 1$ and 14% at 64. The GDN verify kernel, which writes one FP32 state per block position, takes 36% of the cycle at $B = 64$, and 40% with the state commit. This arm has no host gap: the host issues each graph launch 0.75 to 1.06 of a cycle before the GPU starts it, and the GPU is busy 96.6–98.8% of the traced cycle. The tuned arm with 8-token blocks, FlashInfer target attention and FA4 draft attention keeps one: its GPU idles 1.0–1.9 ms per traced cycle outside graph replays at $B = 1$ to 64, and at most about 0.66 ms per untraced cycle at $B = 1$ (derived, an overestimate). The profile’s essay prompts accept only 2.5 to 3.8 tokens per cycle, against 6.2 and 5.0 on the drafter’s panel at 16- and 8-token blocks, so the cycle’s composition largely carries over (only the state commit and the drafter’s KV projection grow with the accepted tokens) but its per-token rates are not the frontier’s.

What the attribution decides Other work finds the head a large share of drafting: FR-Spec profiles EAGLE-2 drafting on Llama-3-8B, with a 128k vocabulary, and finds the head projection at 49% of draft time and the softmax over its output at a further 13% (Zhao et al., 2025), a trimmed-vocabulary study finds the head at 64% of an EAGLE-3-style drafter’s FLOPs (Ben Shoham, 2026), and the compute-optimal vocabulary grows with the compute budget (Tao et al., 2024). On this model and GPU the head is a smaller share of the time than of the bytes: removing the head chain entirely is worth at most

1.11× for plain decoding at $B = 1$, 1.07× at $B = 128$, 1.21× for the MTP cycle at $B = 1$ and 1.10× for the tuned DFlash cycle at $B = 1$ (derived from an estimated untraced cycle), before any cost of the certificate. These ceilings are Amdahl bounds for removing a component under otherwise unchanged execution, not predicted gains. The other layers of the stack leave more headroom, and Section 3.2 of the research notes ranks them.

E.3 The serving baseline

The stock engine’s tuned serving configurations set the bar a certified verifier must clear. Section 4.5 reports their frontier (Figure 4) and the engine changes measured against them (Table 1); this subsection gives how the frontier is measured. None of these configurations uses the certified head, which Appendix E.5 serves against them.

What the frontier measures Each *arm* is one server configuration, swept over client concurrency in closed loop on a frozen workload of 1,857 public chat, code and mathematics prompts with every output fixed at 512 tokens [E48]. For request i of the N at a sweep point, with n_i generated tokens, sent at s_i and finished e_i later, the frontier plots

$$x = \frac{1}{N} \sum_{i=1}^N \frac{n_i}{e_i}, \quad y = \frac{\sum_i n_i}{\max_i (s_i + e_i) - \min_i s_i}, \quad (9)$$

the mean per-request output rate including the time to first token, and output tokens per second on the one GPU. The arms were chosen on a tuning split of the workload, one launch per configuration [E49], and are reported on a disjoint confirmation split of 1,152 prompts in three sessions, each arm launched afresh beside its matched plain baseline [E50]. Every arm is stock or exact up to rounding: each arm that changes the target’s arithmetic first diverged from a matched stock reference on the divergence prompts only at an exact tie, within one BF16 step or at a near tie with both margins within two steps, and no output differed in length [E51]. This class rule was fixed after the first comparisons had been seen, from divergence classes that predate them (Appendix D.4). The protocol in full (launch checks, capacity, the configuration search, the output comparisons, a GSM8K quality check and the measurement hazards) is in Section 4 of the research notes.

Figure 4 draws four of the nine arms. At concurrency 1 the DFlash arm with 16-token blocks gives 3.06 times the throughput and 3.45 times the per-request rate of its matched plain baseline; the best speculative arm, MTP, gives 0.98 times plain at concurrency 48 and 0.86 times at 128 with SGLang’s default admission. Delaying prefills and capping each prefill batch at 16 requests lifts MTP to 1.18–1.32 times the best non-speculative arm at concurrency 48 to 128 in three further sessions (Section 4.5) [E58]. Over the 52 points with three or four sessions, the coefficient of variation of throughput has a median of 0.35% and a maximum of 1.9% [E50].

E.4 The composed exact stack

Table 22 gives every arm of the composed stack. The stack runs on the tuned stock DFlash arm with 16-token blocks of Table 1 (Triton attention, at most 64 running requests) and composes three exact engine changes, each of which passed an equality step before any timed run: the snapshot-free GDN verify (F, SGLang’s fold-every-commit protocol), the backbone GEMM routing table (G) and the certified head on verification (H: per-position fallback, the conservative model, at most 64 positions per call). Other exact changes were left out for reasons fixed in advance; the host-gap patches, for one, change nothing on this arm. The composed engine is the pinned SGLang with 33 opt-in patches. S0 is stock SGLang, and B0 the composed engine with every switch off. The arms, their order, the expected ranges and the decision rule were declared before any timed run. Each of three sessions launched S0 and the full stack FGH at its start and end and every other arm once between them, in reversed order in the second session. An arm’s session ratio is over the mean of S0’s two launches, and an interval that excludes 1 decides a speedup or a slowdown [E56].

B0 is bitwise equal to S0 (token identifiers and top-five log-probabilities), and F to B0. G, FG and FGH are exact up to rounding against stock DFlash with 16-token blocks: every first divergence is an exact tie, one BF16 step or a near tie. H gave the same tokens as B0, FGH the same as FG, and in check mode none of 249,392 and 250,464 certified verify positions differed from the stock head. All of these were classified in the equality step, on 320 prompts of 256 tokens at $c = 1$ with a running limit of 4; the timed sessions, at c up to 8 with a running limit of 64, did not compare outputs.

Against S0 the full stack loses at low concurrency and gains at $c = 8$. Its per-request rate is 0.986 times S0’s at $c = 1$ and 0.972 times at $c = 2$, then 1.013 times at $c = 4$ and 1.079 times at $c = 8$, where throughput is 1.073 times; all are decided. It lies inside its declared range only at $c = 1$ and below it at $c = 2, 4$ and 8. F’s three sessions agree with its one-session

Table 22. The composed stack and its parts against tuned stock DFlash with 16-token blocks (S0): per-request rate x and throughput y at client concurrency c , each the geometric mean of three session ratios with its 95% t interval, rounded outward. Measured [E56]. H alone has no row: its server did not start (text). Decisions use the unrounded interval, so G’s y at $c = 2$ (upper end 0.9992) is a slowdown.

Arm		$c = 1$	$c = 2$	$c = 4$	$c = 8$
FGH: full stack	x	0.986 (0.982–0.989)	0.972 (0.965–0.979)	1.013 (1.006–1.021)	1.079 (1.063–1.095)
	y	0.990 (0.987–0.993)	0.980 (0.973–0.987)	1.013 (1.004–1.021)	1.073 (1.057–1.089)
FG	x	0.974 (0.966–0.983)	0.971 (0.958–0.985)	1.021 (1.007–1.034)	1.082 (1.058–1.107)
	y	0.981 (0.972–0.989)	0.979 (0.967–0.992)	1.020 (1.008–1.033)	1.077 (1.053–1.100)
F: fold verify	x	0.970 (0.967–0.974)	0.987 (0.983–0.990)	1.009 (1.001–1.016)	1.073 (1.064–1.082)
	y	0.971 (0.968–0.974)	0.987 (0.984–0.990)	1.007 (1.001–1.014)	1.068 (1.060–1.076)
G: GEMM routing	x	1.009 (1.003–1.014)	0.988 (0.985–0.992)	1.017 (1.011–1.022)	1.008 (0.995–1.021)
	y	1.014 (1.009–1.020)	0.996 (0.992–1.000)	1.017 (1.012–1.023)	1.007 (0.993–1.021)
B0: switches off	x	0.999 (0.996–1.003)	0.998 (0.994–1.003)	0.999 (0.989–1.010)	0.995 (0.983–1.007)
	y	0.999 (0.996–1.002)	0.998 (0.995–1.002)	0.999 (0.990–1.009)	0.995 (0.983–1.007)

measurement in Table 1 (throughput 0.971 and 1.068 times at $c = 1$ and 8, against 0.968 and 1.061). Part of G’s change is not GEMM time: G rounds differently, so the token trajectories change, and the tokens accepted per verify cycle rise by 0.7% at $c = 1$ and fall by 0.4% at $c = 2$, the same in every session. B0 shows no detectable cost of carrying the patches.

H alone never served a request. In each session its server ran out of memory during start-up. The certified head’s work buffers inside the verify CUDA graphs, which the engine’s memory sizing does not see, made those graphs take 6.13–6.25 GB against 2.91 GB for S0, and beside the stock verify’s 48.75 GB per-position state cache for 64 requests too little memory remained for DFlash’s sampling warm-up. F removes that cache, so FGH started, with 13.4–13.6 GB free after its graph captures. By the declared rule H’s own ratio is void, and the certified head’s effect on DFlash alone stays unmeasured here; Appendix E.5 measures it with the attention cache pinned. Within each session FGH against FG gives +1.1% in per-request rate at $c = 1$, +0.1% at $c = 2$, –0.7% at $c = 4$ (below 1 in every session) and –0.3% at $c = 8$. That is a reading from an unbalanced order, neither declared nor a test. By H’s 64-position limit, most verify calls at $c = 8$, of 128 positions, would run the stock head; the sessions recorded no head counters.

Derived bounds put these ratios in proportion. An engine running this drafter at the bandwidth floor, with no host idle and no per-position state, would have at most 1.81 times the tuned stock arm’s per-request decode rate at $c = 1$ and 3.15 times at $c = 8$, at the measured 5.7–5.8 tokens per verify cycle (derived); the composed changes give 0.986 and 1.079 times S0’s per-request rate there. That most of what remains is a drafting limit is an inference from these bounds, not a measurement. The stack ran on 16-token blocks only. At $c = 8$ the DFlash arm with the higher throughput uses 8-token blocks, where F alone gave 1.058 times in one session [E54]; no composed stack was timed there, and FGH’s throughput at $c = 8$, 3,749 tokens/s, exceeds that arm’s 3,651 only across sessions, without an interval.

E.5 The certified head, served

The served benchmark asks whether the certified head improves the tuned arms’ frontier without changing their outputs, under a design, metric and decision rule declared before its first timed run [E59]. Each family’s tuned arm runs against the same arm with the certified head, on one engine build and with identical flags apart from the head’s variables: plain decoding (the decode head), MTP with three steps and Triton attention (the verify, draft and draft-extend heads), and DFlash with 16- and 8-token blocks (the verify head and the draft projection). Each pair ran in three sessions, certified first in the second. The head runs in the microbenchmark’s best mode where it pays: the per-position fallback, the conservative model, and certification only for calls of at most 64 positions; a larger call replays the same CUDA graph with the head gated off. Both DFlash arms pin their attention cache at 60,000 tokens, because the certified graphs’ work buffers are allocated after SGLang sizes its pools (unpinned, the composed stack’s certified server ran out of memory, Appendix E.4). None of the 132 timed points is invalid. Table 23 gives every point.

Throughput By the declared rule the certified head raises throughput at $c = 1$ by 2.5% for plain decoding, 4.6% for MTP and 1.2% for 16-token DFlash, and lowers 8-token DFlash’s by 1.2% at $c = 4$, where the decision rests on Holm’s last step at the nominal 0.05 and the Bonferroni interval across the four primaries includes 1. The gains are confined to small batches and fall short of the prediction. Plain decoding gains about 2% up to $c = 16$ and 1.1% at 32, then nothing at 64. MTP gains

Table 23. The certified head served against the same tuned arm without it: throughput y with over without, geometric mean of three session ratios with its 95% t interval, and the ratio predicted before the runs from the head-only times. One primary point per family, Holm-adjusted across the four; the other points are descriptive. Gated off: no certified path runs at the point’s batch sizes, apart from the drain at its end [E59].

Arm	c	Role	y ratio (95% interval)	Predicted
Plain decoding	1	primary	1.025 (1.022–1.028)	1.035
	4	descriptive	1.021 (1.019–1.024)	1.031
	8	descriptive	1.021 (1.014–1.027)	1.030
	16	descriptive	1.020 (1.016–1.025)	1.029
	32	descriptive	1.011 (1.002–1.020)	1.016
	64	descriptive	0.999 (0.994–1.005)	1.005
	128	gated off	0.994 (0.993–0.996)	1.000
MTP, Triton attention	1	primary	1.046 (1.045–1.046)	1.086
	2	descriptive	1.035 (1.029–1.041)	1.077
	4	descriptive	1.030 (1.024–1.037)	1.071
	8	descriptive	1.017 (1.004–1.030)	1.057
	16	descriptive	1.000 (0.980–1.019)	1.041
	32	descriptive	0.979 (0.939–1.021)	1.017
	64	descriptive	0.967 (0.949–0.986)	1.004
DFlash, 16-token blocks	1	primary	1.012 (1.007–1.018)	1.045
	2	descriptive	0.990 (0.984–0.996)	1.026
	4	descriptive	0.970 (0.958–0.983)	1.009
	8	gated off	0.988 (0.977–1.000)	1.000
DFlash, 8-token blocks	4	primary	0.988 (0.977–0.999)	1.024
	8	descriptive	0.975 (0.967–0.984)	1.008
	16	gated off	0.988 (0.962–1.015)	1.000
	32	gated off	0.994 (0.945–1.046)	1.000

3.0–4.6% up to $c = 4$ and 1.7% at 8, shows no detectable change at 16 and 32, and loses 3.3% at 64, where its certified calls are draft calls of 64 positions. Both DFlash arms lose 1.0–3.0% at every tested point where the head is active above $c = 1$, where their certified calls reach 28–64 positions. On the served envelope, the confirmation frontier’s best arm at each concurrency (Figure 4), the ratio is 1.012 at $c = 1$, 0.990, 0.970 and 0.975 at $c = 2, 4$ and 8, and 0.994 at 128, with no detectable change at 16, 32 and 64: the served envelope rises only at $c = 1$ and falls at $c = 2, 4, 8$ and 128. Where every certified path is gated off, the captured certified graph still costs 0.6% (plain decoding at $c = 128$) to 1.2% (16-token DFlash at $c = 8$), and the certified graphs take 3.8–25.7 GB more memory at capture. The check launches show the share of head calls that need a fallback growing with the batch, from 2% at $c = 1$ to 61% at 64 for plain decoding and to 76–82% at $c = 4$ for 16-token DFlash. That is consistent with the losses, but the time per fallback inside the served graphs was not measured, so the mechanism is a hypothesis.

Exactness Check launches ran each certified arm at its tuned flags with the stock head also computed in every certified step, counting on the device the positions whose token differs: every certified call was counted, and 0 of 592,433 certified positions differ. At $c = 1$, where one request at a time gives both arms the same batches, every request’s token ids were identical between the arms in all three sessions (plain decoding, MTP and 16-token DFlash, 32 of 32 prompts each); 8-token DFlash has no $c = 1$ point and rests on check mode alone. A batch’s verify goes to the stock head if any of its requests asks for log-probabilities, penalties, a grammar or token mask, logit bias, a custom logit processor or logprob capture. The benchmark workload has none of these. In a deployment, a batch that holds such a request gets the stock verify: a condition on both the exactness scope and the speedup.

Above $c = 1$ closed-loop batches vary from run to run. Pooled over $c > 1$, first divergences between the certified and stock runs of a session occur at 1.21 times (95% interval 0.92–1.59) the rate between stock runs of different sessions for plain decoding, 1.11 times (1.0002–1.22) for MTP and 1.03 times (0.91–1.16) for 8-token DFlash; 16-token DFlash diverged 9 times against none between stock runs, on the same 3 prompts at $c = 8$ in every session, and a wave control points to the batch evolution as the cause. Re-scored on a stock server at batch 1, all but 20 of the 1,113 first-divergence contexts are exact ties or one BF16 step apart (19 within 0.5 nats, 1 large). MTP’s lower bound sits just above 1, and the excess fits the certified engine’s different step times changing the batch evolution: under identical batch evolution the arms are identical. In a seeded control at the most sensitive position known (prompt 579ae7ce, decoded with MTP from output position 400 in waves of 1,

8, 12 and 16 requests), the certified verify decided all 3,708 verify steps and every token of the 109 requests equalled stock's on both passes. With log-probabilities requested, which keep the verify on the stock head, the certified graphs, the certified draft and draft-extend heads and the gated-off verify left every token of the 109 requests equal to stock's, and the top-five and five tracked tokens' log-probabilities at position 439 identical to the bit. Waves with fixed batch evolution also gave identical outputs for MTP (64 of 64 prompts, and 512 of 512 in waves of 64 at the timed pools) and for 16-token DFlash at $c = 8$ with the head gated off (64 of 64).

The one large divergence Session 1's certified MTP run at $c = 64$ committed token 1756 at prompt 579ae7ce, output position 439, a single position in text the model wrote after its own end-of-text token; stock runs there committed 68189, 8078 or 5715. At that position an FP32 forward puts 68189 on top and 1756 8.9 nats below it, and FP32's two paths (one forward, and recurrent decoding) agree within 0.0001 nats and side with the stock BF16 prefills, while stock batch-1 plain decoding from position 400 puts 1756 on top. By the reading of these paths fixed before the comparison ran, the stock decode path carries the error there, independent of the certified head; the layer that errs was not located. A later comparison makes it an SGLang outlier at this position [E41]: Hugging Face transformers (5.12.1) in BF16 (two GDN implementations, BF16 or FP32 cached state, four paths each) keeps FP32's top token in all 16 readings, with 1756 at -10.11 to -7.60 against FP32's -9.79 (three readings move a tracked token by 1.15 to 3.08 nats, which the reading set before the run calls a partial BF16 sensitivity), and FP32 under relative noise of one BF16 rounding moves 1756 by at most 0.52 nats in 16 draws (a probe, not a bound); no single swap of an SGLang kernel family removes the miss. The position was found because SGLang erred there. On 15,360 positions per path that nobody selected, SGLang's mean error on FP32's top token is no larger than transformers', and misses above 2 nats are too rare (none or one per path) to compare, so whether such outliers are more frequent in SGLang is not known. Against FP32, closed-loop runs at $c = 64$ commit a token at least 2 nats below the top there in 4 of 19 stock MTP draws and in 8 of 34 uninstrumented draws of engines carrying the certified graphs, a grouping chosen after seeing the data; 1756 itself appears only in the latter (4 of 34 against 0 of 19; post hoc, one-sided Fisher p about 0.16), so the arms differ in which wrong token, not measurably in how often. No observation requires an error in the certified head: the certified graphs with the head never deciding committed 1756 too, the head matched the stock argmax at every check-mode position and at all 967,680 positions of a stress test (batch-1 inputs in head-only graphs), and stock batch-1 plain decoding itself puts 1756 on top. The certified head's contract is identity with the stock engine's output for the same batch, and the stock engine is not FP32-exact at positions like this one.

F EVIDENCE REGISTER AND REPRODUCTION

This appendix connects the paper's claims to the repository. Appendix F.1 lists every committed result the text cites, with the files that hold it and the program that produced them. Appendix F.2 gives the commands that rerun the CPU checks and says where the GPU commands are, Appendix F.3 lists the hardware, software and model versions, and Appendix F.4 states the authorship and the use of AI.

F.1 Evidence register

Every committed result the paper cites is listed below with the claims it supports, the files that hold it and the program that produced them. All paths are relative to the repository at <https://github.com/SamMausberg/verified-progress>. The exact command behind each evidence file is in the README.md of the file's directory, and, where the generator records them, the files give the commits of this repository and of the engine that produced them. The text cites an entry as [En]. Proved statements are not listed; their proofs are in the text and Appendix A.10 says which are machine-checked.

Table 24. Where the numbers come from, and which decision rule each applies. A replay re-evaluates recorded head inputs offline; the held-out replay draws whole decode steps at random from the held-out half until it has at least 6,000 positions (6,005 in 750 steps).

Source	What	Decision rule	Size
Held-out replay	FP64 replay of held-out plain-decoding positions	R-real to compare copies; gap and interval conditions	6,005 positions
Kernel replay	the GPU kernel on the first captured positions, both halves, at their batch shapes	interval condition; seeded race	60,000 positions
Engine check	the kernel inside SGLang, conservative model	interval condition; fixed-noise verification	64 prompts
Microbenchmark	the head alone, CUDA graphs, warm L2 cache	time only	$M = 1$ to 256
Speculative captures	aligned draft and target head inputs; verification positions	R-real, gap condition; drift	40,000 pairs per drafter
Served sweeps	the serving engine at full capacity, swept over client concurrency, with and without engine changes	time; output equality; for the certified head, the interval condition under the conservative model	1, 3 or 4 sessions

Entry	What it supports; files and the program that produced them
<i>The certified head on captured engine states</i>	
E1	Captured plain-decode head inputs reproduce the engine’s tokens; R-real and R-stock differ at about 0.5% of greedy positions (Section 4.1) evidence/head_geometry/alignment_plain4b.json; produced by experiments/head_geometry/validate_alignment.py.
E2	Candidate sets, error-bound checks, head-input norms, batch unions, FP32 re-scoring and cascades on 6,005 held-out plain-decode positions (Section 4.2, Appendix A.8) evidence/head_geometry/selfevidence_plain4b.json; produced by experiments/head_geometry/analyze_selfevidence.py.
E3	Candidate counts per cheap copy, error bound, accumulation model and decision evidence/head_geometry/selfevidence_plain4b.csv; produced by experiments/head_geometry/analyze_selfevidence.py.
E4	Share of positions needing at least k candidate tokens (Figure 3) evidence/head_geometry/selfevidence_plain4b_ccdf.csv; produced by experiments/head_geometry/export_candidate_ccdf.py.
E5	Weight-row norms, top-two margins, top-token mass, error-bound width against realized error evidence/head_geometry/stats_plain4b.json; produced by experiments/head_geometry/analyze_stats.py.
E6	Rows and batches left to the stock kernel by rule and error model; no certified token differed from the engine’s (Table 5) evidence/head_geometry/rstock_plain4b.json; produced by experiments/head_geometry/analyze_rstock.py.
E7	Candidate sets on DFlash-4B verification and draft positions evidence/head_geometry/selfevidence_dflash4b.json, selfevidence_dflash4b.csv, selfevidence_dflash4b_ccdf.csv; produced by experiments/head_geometry/analyze_selfevidence.py.
E8	Candidate sets and fallback shares on 4,010 MTP-4B verification positions and 4,024 draft positions (Appendix A.8) evidence/head_geometry/selfevidence_mtp4b.json, selfevidence_mtp4b.csv, selfevidence_mtp4b_ccdf.csv; produced by experiments/head_geometry/analyze_selfevidence.py, export_candidate_ccdf.py.

The certified head kernel

E9	The certified-head package and its tests: the W8A16, W8A8 and BF16 passes with the error-bound epilogue, the per-variant self-test and the runtime probes; 145 tests passed at commit 9f7f369, where all 148 kernel variants compiled for sm_90 and each default tile configuration had 0 misses in about a million checks; changes after that commit add host-side refusals with CPU tests (Section 3.4) src/certified_head/, tests/test_certified_head.py, test_certified_bounds.py, test_certified_head_inputs.py, evidence/certified_head/gpu_tests.log, x8s2_compile.log, stress_defaults.json, README.md; produced by experiments/certified_head/stress_defaults.py, compile_check.py.
----	---

Entry	What it supports; files and the program that produced them
E10	Measured violations of the modelled arithmetic by int8 TMA tiles of the W8A16 pass (Triton 3.7.1): an int8 tile loaded through a 64-byte TMA box gave wrong raw products against FP64 in every such configuration tested, at some or all of $M = 1, 16, 64, 128$ and 256, while pointer loads and 128-byte boxes were exact; three tiles with block_m 128 missed the error bound in some runs; both families are refused evidence/certified_head/tma_repro.json, tma_candidates.json, large_m_check.json, tma_m128_check.json, tma_m128_neighbourhood.json; produced by experiments/certified_head/tma_repro.py, tma_candidates.py, large_m_check.py, tma_m128_check.py.
E11	Which Triton and ptxas builds give the certified head's int8 TMA fault (64-byte inner box): of seven tested builds, the four without both Triton $\geq 3.8.0$ and ptxas ≥ 13.3 gave wrong products and the three with both did not, a version pattern that matches the closed issue triton-lang/triton#9433; correctness only evidence/triton_tma/int8_tma_summary.json, cases/, README.md; produced by experiments/triton_tma/int8_tma_check.py, run_versions.sh, summarize.py.
E12	The cuBLAS kernel of the stock head per batch size; turning off reduced-precision reduction changes neither the kernel nor any output bit; the stock FP32-output kernel's largest observed accumulation error, 2.07×10^{-6} of $\sum_j w_{ij}h_j$ over 2,048 real head inputs, 295 and 58 times inside the conservative model and the Hopper model; the BF16-output kernel's accumulator is not observable evidence/certified_head/stock_invariance.json; produced by experiments/certified_head/stock_invariance.py.
E13	Which tensor-core instruction the two stock head kernels whose SASS could be read issue (HGMMMA, warpgroup wgmma, at $M = 1$ and 32) and how the stock head kernels and Triton's wgmma accumulate on crafted inputs (16-product blocks, 25 fractional bits, truncation; BF16 epilogue to nearest even) evidence/profiles/wgmma_precision.json, head_tensor_instructions.json; produced by experiments/profiling/wgmma_precision.py, tensor_instructions.py.
E14	Per-position (row-wise) and column-subset bitwise invariance of the stock head GEMM at 20 batch sizes from 1 to 256, including the gathered-candidate shape of the column fallback; a measured property of this build evidence/certified_head/stock_invariance.json; produced by experiments/certified_head/stock_invariance.py.
E15	60,000 real decode positions replayed through the kernel at their engine batch shapes (at most 16 positions): positions and steps that need the fallback under both error models, and certified positions plus the fallback equal to the reference on every position, greedy and seeded evidence/certified_head/replay_decisions.json, fallback_vs_model.json; produced by experiments/certified_head/replay_decisions.py, fallback_vs_model.py.
E16	Cost of the per-position fallback, which completes an undecided position from its gathered candidates, against the whole-batch fallback and the stock head chain at $M = 1$ to 256, under both error models (the head alone) evidence/certified_head/micro_head.json, head_path_time.csv, head_path_table.md; produced by bench/micro_head.py, summarize_head.py.
E17	Head-path microbenchmark: the certified head with probes on and off, both fallback modes and both error models, the W8A8 and BF16 passes and seeded sampling, against the stock head chain at $M = 1$ to 256; the head alone, not decoding evidence/certified_head/micro_head.json, head_path_time.csv, head_path_table.md, micro.hostload.json; produced by bench/micro_head.py, summarize_head.py.
E18	Primitive costs (the stock chain, the W8A16 pass alone, tile GEMMs, re-scoring and the draft summary), Nsight Compute of the production W8A16 launches, and the fastest TMA and pointer-load configurations per pass evidence/certified_head/head_primitives.json, ncu_gemv_summary.json, pointer_vs_tma_w8a16.json, pointer_vs_tma_w8a8.json, pointer_vs_tma_bf16.json; produced by bench/head_primitives.py, tune_gemv.py, experiments/certified_head/ncu_summary.py.
E19	Seeded sampling without top-k or top-p: on the 60,000 replayed positions at $T = 0.7$ and 1.0, certified positions plus the fallback equal SGLang's seeded sampler on every position; the sampled path's timings; the small-probability guard's derivation and its tests evidence/certified_head/replay_decisions.json, micro_head.json, src/certified_head/head.py, tests/test_certified_head_inputs.py; produced by experiments/certified_head/replay_decisions.py, bench/micro_head.py.

Entry	What it supports; files and the program that produced them
E20	The certified head inside SGLang in check mode, with SGLang’s own head run beside it at the same batch shape: eleven arms on 64 public prompts (at most 256 new tokens, the conservative model, the whole-batch fallback and in one arm the per-position fallback) covering greedy decoding, MTP and DFlash greedy verification, MTP draft steps and draft extend, the DFlash draft projection and MTP fixed-noise sampled verification at $T = 0.7$ under deterministic inference; 0 of 196,734 certified positions differed from the stock token; certified and stock outputs identical on 64 of 64 prompts one request at a time, for plain decoding and MTP; the merged package’s 167 GPU and CPU tests; correctness only, nothing timed evidence/certified_head/engine_v2.json, engine_v2_arm_checks.log, engine_v2_gpu_tests.log, x9e2_commit.txt, README.md; produced by experiments/certified_head/engine_validate.sh.

Transport

E21	Weight-only constants and the crossover thresholds δ^* (Table 8) evidence/precision/head_constants.json; produced by experiments/precision_head_constants/head_constants.py.
E22	Drift on 40,000 DFlash-4B pairs, by outcome, position, domain and context, with the per-entry thresholds (Figure 5, Table 9) evidence/head_geometry/rho_dflash4b.json, rho_dflash4b_quantiles.csv, rho_dflash4b_pairs.csv; produced by experiments/head_geometry/analyze_rho.py.
E23	Transport on DFlash-4B by bound family and tiling: skips, partition widths, P_7 , batch unions, drift scaling (Tables 12, 13 and 14) evidence/head_geometry/transport_dflash4b.json, transport_dflash4b.csv; produced by experiments/head_geometry/analyze_transport.py.
E24	The same on 16,016 MTP-4B pairs, with their drift evidence/head_geometry/transport_mtp4b.json, transport_mtp4b.csv; produced by experiments/head_geometry/analyze_transport.py.
E25	Drift on 40,000 aligned MTP-4B pairs, with the per-entry thresholds (Figure 5) evidence/head_geometry/rho_mtp4b.json, rho_mtp4b_quantiles.csv, rho_mtp4b_pairs.csv; produced by experiments/head_geometry/analyze_rho.py.
E26	Draft and target head-input statistics; alignment of the DFlash-4B capture (Appendix C.4) evidence/head_geometry/stats_dflash4b.json, alignment_dflash4b.json; produced by experiments/head_geometry/analyze_stats.py, validate_alignment.py.
E27	Draft and target head-input statistics; alignment of the MTP-4B capture and its outputs against plain decoding (Appendix C.4) evidence/head_geometry/stats_mtp4b.json, alignment_mtp4b.json; produced by experiments/head_geometry/analyze_stats.py, validate_alignment.py.

Exact references and formal checks

E28	Exact floating-point reference: 21 test methods, 40,015 checks, the R-bf16, split- K and noise-stream witnesses (Appendix A.10) evidence/precision/precision_tests.json, precision_tests.log; produced by tests/test_precision.py.
E29	Lean 4.19.0 elaboration of the two formal files of the paper’s own lemmas (Appendix A.10) and of three files on drafting proposals (research notes, Section 5.8) formal/STATUS.md, evidence/precision/lean_check.log; produced by scripts/check_lean.sh.
E30	Exact rational reference for decisions and races, with the instances it checks and the replay witness for the recurrent state (research notes, Sections 8 and 6.2) evidence/decision_tests.json, race_tests.json and their logs; produced by tests/test_decisions.py, tests/test_races.py, via scripts/verify_artifact.py.
E31	Witnesses that separate the exactness contracts; the regrouped recurrence; the transition norm (Appendix D.1) evidence/contracts/contract_witnesses.json, contract_witnesses.log; produced by tests/test_contracts.py.
E32	Witnesses for the structure of the recurrent state, among them a state difference the current query cannot see (Appendix D.1) evidence/state_structure/state_structure_tests.json, state_structure_tests.log; produced by tests/test_state_structure.py.

Entry	What it supports; files and the program that produced them
<i>The stock engine</i>	
E33	First differing module outputs and divergence classes under both error models (Figure 6) evidence/state_safety/mechanism_plain_c1_vs_mtp_s3_c1.json, mechanism_plain_c1_vs_c32.json, first_difference_by_module.csv, divergences.csv; produced by experiments/state_safety/mechanism.py, analyze_all.sh, compare.py.
E34	Divergence rates, BF16 top-two ties, same-server and fresh-server repeats and the deterministic-mode checks; the server settings of every run evidence/state_safety/noise_floor.csv, noise_floor.json, run_meta.json; produced by experiments/state_safety/run_all.sh, compare.py.
E35	Positive controls of the bit-exact tap; the method; the tap-changed prompts and their module signature in two tapped sessions evidence/state_safety/tap_control_down9.json, tap_control_core13_decode.json, tap_control_core13.json, tap_check.json, tap_signature.json, README.md; produced by experiments/state_safety/mechanism.py, analyze_all.sh, tap_signature.py.
E36	Cache-level checks with the cache-hashing tap: at batch 1, plain decoding against MTP with three steps (40 prompts), the two history pairs at the level of the KV, the prefix cache on against off (40 prompts), one prompt’s same-server repeats with and without a flush; plain decoding at concurrency 1 against 32 with fixed pools (40 prompts); and the tapped runs against untapped ones (Appendix D.4) evidence/state_safety/cachecheck_v4_plain_c1_vs_mtp_s3_c1.json, cachecheck_v4_plain_c1_radix_vs_noradix.json, history_v4_mt_bench-0056.json, history_v4_humaneval-0008.json, repeats_v4_h44_flush.json, repeats_v4_h44_noflush.json, cachecheck_v4_plain_c1_vs_c32.json, README.md; produced by experiments/state_safety/mechanism.py, from the tapped sessions of run_tap_v4.sh and run_tap_v4_batch.sh (tap_runs.py).
E37	The pools each server allocated for every comparison of Appendix D.4, with whether the two servers were the same and the pools identical evidence/state_safety/pools.json, run_meta.json, noise_floor.csv; produced by experiments/state_safety/pools.py, analyze_all.sh.
E38	The cap-8 runs with fixed pools: divergence rates and classes for plain and MTP pairs with identical pools, self-consistency, the pools and token counts of every run, divergences per fragile position by the previous cycle’s commit length with the first cycle after prefill, the pinned against unpinned batch-1 comparison, MTP with three steps and plain decoding under the prefix cache off, the overlap scheduler off, deterministic inference and the FP32 head, a second MTP session, and the cap-8 runs against the bench’s runs with pools sized at start-up, without the prefix cache (Appendices D.4 and E.3, and Section 6.1 of the research notes) evidence/state_safety/noise_floor_pinned.json, noise_floor_pinned.csv, divergences_pinned.csv, run_meta_pinned.json, cycles_mtp_s1_pinned.json, cycles_mtp_s3_pinned.json, cycles_mtp_s5_pinned.json, cycles_mtp_tree_pinned.json, cross_regime.json, cross_bench.json, divergences_cross_bench.csv, README.md; produced by experiments/state_safety/analyze_all.sh, compare.py, cycles.py.
E39	Divergence given perturbation: prompts perturbed per pair, the share of perturbed prompts that diverge in tokens overall and by onset with Wilson intervals, the median onset and post-onset divergence rates, for the cap-16 and the cap-8 runs (Appendix D.4) evidence/state_safety/perturbation.json, perturbation_pinned.json, README.md; produced by experiments/state_safety/perturbation.py, analyze_all.sh.
E40	Targeted state tests: truncation and stop tokens inside MTP verify cycles and for the tree and plain decoding, warm against cold continuation, history dependence through the prefix cache with and without it, and checkpoint reuse (Appendix D.4 and Sections 6.1 and 6.2 of the research notes) evidence/state_safety/targeted.json; produced by experiments/state_safety/run_targeted.sh, targeted.py, summarize_targeted.py.
E41	BF16 against FP32 at the two served positions where stock SGLang’s BF16 paths miss FP32 by 9 to 15 nats: transformers in BF16 in four configurations, SGLang at its pin and in four variants that each swap one kernel family, FP32 under relative noise of BF16 rounding size, and the error rates of SGLang and transformers against FP32 on 15,360 unselected positions per path (Appendix E.5) evidence/bf16_paths/README.md, positions.json, rates.json, rates_eot.json; produced by experiments/bf16_paths/hold.sh, summary.py, rates.py, hf_paths.py, perturb.py, sglang_variants.py.

Entry	What it supports; files and the program that produced them
<i>Where the time goes</i>	
E42	Attribution of the plain step and the MTP cycle by component; unprofiled throughput; the state-slot reservation (Tables 20 and 21) evidence/profiles/tables.md, breakdown.csv, step_share.csv, label_structure_check.json, attribution/, windows/; produced by experiments/profiling/run_profiles.py, attribute.py, summarize.py.
E43	The head chain’s share of a plain step and of an MTP cycle; the head microbenchmark and the kernels cuBLAS selects evidence/profiles/head_microbench.json, head_microbench_kernels.json, attribution/; produced by experiments/profiling/attribute.py, head_microbench.py.
E44	Read bandwidth: 3.83 TB/s over 4 GiB for the best of nine launch configurations, 3.79 TB/s over the head’s footprint with that configuration evidence/profiles/hbm_bandwidth.json; produced by experiments/profiling/hbm_bandwidth.py.
E45	Host functions running while the GPU idles in the MTP cycle evidence/profiles/diagnostics/; produced by experiments/profiling/host_gaps.py, pypspy_summary.py.
E46	The DFlash cycle by phase on the two tuned DFlash arms at batch 1 to 64, traced and untraced: the head chains’ share and its ceiling, the extend-attention and GDN verify costs, the host gap, and the profile’s acceptance (Figure 1, Appendix E.2) evidence/profiles/README.md, dflash_cycle.json, dflash_cycle.csv, dflash_breakdown.csv, step_share.csv, attribution/, windows/; produced by experiments/profiling/run_profiles.py, collect_run.py, dflash_cycle.py, attribute.py, summarize.py.
<i>Drafting and serving</i>	
E47	Divergences of DFlash and MTP outputs from plain decoding, with the plain run’s margins evidence/drafter/panel_v2/equality_zlab_b16.json, equality_mtp3.json; produced by experiments/drafter/run_equality.sh, compare_outputs.py.
E48	The frozen serving workload: sources, revisions, splits and token statistics bench/workloads/mixed-v2/manifest.json; produced by bench/build_workload.py.
E49	Configuration search on the tuning split, one launch per configuration, and the chosen arms (Appendix E.3); derived snapshot traffic evidence/bench/tuning/points.csv, frontier.csv, launches.csv, evidence/bench/README.md, bench/arms.toml; produced by bench/campaigns/tuning_depth.sh, tuning_knobs_dflash.sh, tuning_backend_dflash.sh, bench/pareto.py.
E50	Confirmed serving frontier: nine tuned arms on the confirmation split, three sessions, paired ratios, run-to-run variance and the envelope over points with at least three sessions evidence/bench/confirm/frontier.csv, envelope.csv, envelope-all.dat, pairs.csv, points.csv, launches.csv, evidence/bench/README.md; produced by bench/campaigns/confirm.sh, confirm_supplementary.sh, confirm_evidence.sh.
E51	Greedy outputs of every arm that changes the target’s arithmetic, and of stock MTP and DFlash, against matched stock references without the prefix cache: first-divergence classes, rates and ratios to the batch-shape floor, each server’s pools, and the first classification against the prefix-cache-on plain run (Appendix E.3) evidence/bench/equality/classes.json, report.json, divergences.csv, table.csv, pools.csv, evidence/bench/README.md; produced by bench/campaigns/equality_tuned.sh, bench/divergence.py, experiments/state_safety/run_matrix.py, compare.py.
E52	GSM8K test accuracy of six runs (two of plain decoding, buffered MTP, MTP with snapshot verification, DFlash at block 8, replayed plain decoding) with paired exact McNemar tests (Section 4.5; research notes, Section 4) evidence/bench/quality/comparisons.json, evidence/bench/quality/; produced by bench/campaigns/quality.sh, quality_compare.py.
E53	Host-side idle in the MTP and DFlash cycles and the five patches that compute FlashInfer’s attention plans without blocking reads: stock host traces, the derived untraced idle, plan and attention-output equivalence, in-engine validation, greedy output equality with stock, untraced cycle times before and after, and interleaved served A/Bs on tuned MTP and on untuned DFlash with FlashInfer draft attention (Table 1; research notes, Section 3.6) evidence/hostgap/README.md, stock_traces_hold1.json, cycle_profiles.json, plan_equivalence.json, equality/, hold4/, hold5/, engine/sglang/patches/hostgap/; produced by experiments/hostgap/profile_arms.sh, gap_analysis.py, hold_equality_profiles.sh, plan_equality.py, equality.py, summarize.py, hold_mtp.sh, hold_dflash.sh, ab_sweep.sh, ab_summary.py.

Entry	What it supports; files and the program that produced them
E54	SGLang’s fold-every-commit GDN verification enabled for DFlash and MTP: kernel parity with the stock verify; served greedy outputs against stock with matched pools and fixed batch composition; the per-phase split at batch 8 and 16; one session of paired serving on the two tuned DFlash arms (Table 1; research notes, Section 5.9) evidence/drafter/buffered_verify/gdn_verify_parity.json, exactness.csv, localize/, evidence/drafter/phase_timing/summary.csv, evidence/drafter/fold_timing/summary.json, evidence/drafter/README.md, engine/sclang/patches/drafter/; produced by experiments/drafter/gdn_verify_parity.py, run_replay_check.sh, run_fold_localize.sh, run_phase_timing.sh, run_fold_timing.sh, ab_timing_summary.py.
E55	Backbone weight GEMMs in isolation and served: faster kernels reading the same weights and their bitwise agreement with cuBLAS; each switch’s greedy outputs against stock plain decoding; one session each of paired serving on tuned plain decoding and on MTP with FlashInfer and with Triton attention; an Nsight trace of the GEMM kernels a served decode step runs; the GPU tests (Table 1; research notes, Section 3.6) evidence/backbone/gemm_microbench.json, merge_microbench.json, norm_microbench.json, chain_microbench.json, chain_fold_variants_microbench.json, gpu_tests_hold2.log, tables.md, README.md, hostload/, served/, engine/sclang/patches/backbone/; produced by experiments/backbone/gemm_bench.py, summarize.py, make_table.py, bitwise_runs.py, experiments/state_safety/compare.py, bench/pareto.py, experiments/backbone/insitu_gemm.py, stream_text_identity.py, experiments/profiling/run_profiles.py.
E56	The exact engine changes that apply to DFlash with 16-token blocks (fold verify, backbone routing and the certified head on verification), composed and timed against the tuned stock arm in three pre-registered sessions at concurrency 1 to 8: each arm’s session ratios with 95% t intervals, the equality step, the start-up memory of every server, the certified head’s reading inside the stack, tokens per verify cycle, and the derived ceilings (Appendix E.4) evidence/stack/README.md, composition.json, ratios.csv, sessions.csv, startup_memory.csv, last_lever.csv, accept.csv, gap_by_concurrency.csv, ceiling.json, evidence/stack/equality/gate.json, evidence/stack/equality/table.csv; produced by experiments/stack/hold_equality.sh, hold_session.sh, analyze_campaign.sh, analyze.py, startup_memory.py, diagnostics.py, figures.py, ceiling.py.
E57	The levers on the low-concurrency envelope’s DFlash arms (fold with narrow verify tiles, FA4 drafter attention, FA4 target attention with the head-dimension-256 loader fix) alone and composed, against stock: the equality gate on 320 prompts and three pre-registered sessions with 95% t intervals (Table 1; research notes, Section 3.6) evidence/speed_lowc/confirm/README.md, ratios.json, points.csv, accept.json, launches.csv, equality/; produced by experiments/speed_lowc/confirm_analyze.py, confirm_accept.py, confirm_gate.py, hold_confirm_equality.sh, hold_confirm_session.sh.
E58	Admission at high concurrency: probes of SGLang’s prefill delay flags, three sessions of MTP with the prefill delayer and a 16-request prefill cap against the non-speculative arms at concurrency 1 to 128 with times to first token, the outputs’ class at concurrency 64 and 128, and the cost of one small prefill (Table 1; research notes, Section 3.6) evidence/admission/README.md, confirm_analysis.csv, confirm_points.csv, confirm_arms.csv, logprob_classes.json, prefill_requests.json, prefill_trace.json; produced by experiments/admission/analyze_confirm.py, summarize_probe.py, classify_logprob.sh, collect_records.sh.
E59	The certified head served on the four tuned arms against the same arms without it: three pre-registered sessions, paired throughput ratios with the four primaries Holm-adjusted, the served envelope, check-mode counters, token identity at concurrency 1, closed-loop divergences and their classes, the wave and seeded controls, BF16 and FP32 reference paths at the large divergences, and capture memory (Table 1, Appendix E.5) evidence/certified_head/served/README.md, summary.json, ratios.csv, pairs.csv, check.csv, equality.csv, classes.csv, control_waves_mtp.json, control_waves_mtp64.json, control_waves_dflash16.json, control_seeded_mtp.json, control_seeded_stats.json, control_seeded_tokens.json, reference_paths.json, drain_579ae7ce.csv, stress_summary.json, capture_memory.csv, experiments/benchcert/README.md; produced by experiments/benchcert/analyze.py, run_session.py, rescore.py, control_waves.py, paths.py, drain.py, fallback_stress.py.
E60	The two declared approximations, FP16 recurrent state and the int4 target with its int4 DFlash drafter, against the tuned non-lossy arms: three pre-registered sessions, GSM8K and logit-probe quality runs and the verdicts against the lossy budget; the exploratory W4A16 GEMM microbenchmark (Section 4.5; research notes, Section 3.6) evidence/lossy/decisions.json, ratios.csv, README.md, sessions/, quality/, probes/, gemm_w4a16.json; produced by experiments/lossy/analyze.py, run_hold.py, gsm8k_partial.py, gemm_w4a16_bench.py.
<i>Literature</i>	
E61	Literature review and the citation audit of every reference sources/literature_review.md, citation_audit.md, source_manifest.json.

A file name listed without a directory is in the directory of the entry’s first file. Raw captures, traces and per-position arrays are not in the repository; each evidence README gives the command that regenerates them on the same hardware.

F.2 Reproduction

The CPU evidence (the exact references, the contract and state-structure witnesses, the synthetic experiment and the Lean check) is reproduced from the repository root with Python 3.13, NumPy and, for the formal files, Lean 4.19.0:

```
python -m venv .venv && . .venv/bin/activate
python -m pip install -r requirements-cpu.txt -r requirements-dev.txt
python scripts/verify_artifact.py      # rational references, synthetic drift, Lean check
python tests/test_precision.py         # exact floating-point reference
python tests/test_contracts.py         # contract witnesses
python tests/test_state_structure.py   # recurrent-state witnesses
python -m pytest tests/                # every CPU test; GPU tests skip without CUDA
```

The GPU experiments need the hardware and engine of Appendix F.3: one GH200 and the engine at the pinned SGLang commit with the patches under engine/sglang/patches/; engine/sglang/README.md gives the commands that apply each series. Every evidence directory’s README.md lists the exact command behind each file, and RUNBOOK.md collects the procedures. Timed runs were taken with exclusive use of the GPU and with the CPU load of other processes recorded for every point. The paper builds with latexmk -pdf paper.tex in paper/ (pdfLaTeX and BibTeX, no shell escape); the build fetches the MLSys style file from the official kit and checks its SHA-256 digest (paper/template/README.md), and a figure whose data file is absent builds as a marked placeholder.

F.3 Hardware, software and model pins

Every measurement in the paper was taken with the pins below; the precise command, commits and resolved server settings of each run are recorded next to its evidence.

Item	Pin
GPU	One NVIDIA GH200 480GB: 96 GB HBM3, compute capability 9.0 (sm_90), 132 SMs, 60 MB L2; clocks 1,980 MHz (SM) and 2,619 MHz (memory) during the profiles
Host	64-core Grace (aarch64); driver 570.195.03 with CUDA 13.0 forward-compatibility libraries
Engine	SGLang bd66ce343e4f6e2f2b75d7e820fe4d0718a8d824 (0.5.21.dev861), PyTorch 2.13.0+cu130, FlashInfer 0.6.18, Triton 3.7.1, FlashAttention 4 (flash-attn-4 4.0.0b19; the DFlash drafter’s attention in one tuned serving configuration); FlashAttention 3 is unavailable on this aarch64 host
Target model	Qwen/Qwen3.5-4B at revision 851bf6e806efd8d0a36b00ddf55e13ccb7b8cd0a, BF16; its MTP layer loads from the same checkpoint
Block drafter	z-lab/Qwen3.5-4B-DFlash at revision 9a1996ccf887b79ab3af4fcbf8c1d1f4b5658bcf (weights SHA-256 1eb221d3. . . ; byte-identical to modal-labs/Qwen3.5-4B-DFlash)
Engine patches	Opt-in patch series under engine/sglang/patches/, one directory per experiment: head-input capture, the divergence tap and verify KV split, the stack levers of the research notes (Section 3.6), an acceptance trace hook, repair probes and a timing-only verify switch, backbone GEMM routing and folded norms, and sync-free attention planning. engine/sglang/README.md names each series’ directory; each patch is off unless its flag or environment variable is set
Serving client	AIPerf 0.13.0, closed-loop concurrency, streaming chat completions
Profilers	Nsight Systems 2025.3.2, Nsight Compute 2025.3.1
CPU references	Python 3.13 (3.13.5 for the rational reference, 3.13.15 for the floating-point reference and the witnesses; each evidence file records its version), NumPy 2.3.5; Lean 4.19.0
Prompts	320 capture prompts (evidence/head_geometry/prompt_manifest.csv), 320 divergence prompts (evidence/state_safety/prompt_manifest.json), the serving workload [E48], each with its sources and SHA-256 digests

F.4 Attribution and disclosure

Samuel Mausberg is the author of this paper, directed the work and is responsible for its claims. AI (Claude Opus 5.5, GPT-6 Astra Pro) was used in developing the paper and the code. The confidential assignment that motivated the work is not reproduced or quoted, and no model weights are redistributed.

G GLOSSARY

This table defines the terms the paper relies on, in two groups: those the main text uses, then, in alphabetical order, those that appear only in the appendices, with the place where each is introduced.

Term	Definition
<i>Terms of the main text</i>	
Vocabulary entry i	A token of the vocabulary and its row w_i of the head matrix W ; the rows of W are always called entries, never positions.
Position	One head input h and the token decided from it: a decode step’s request, or one drafted token under verification.
Head input h	The BF16 vector the engine feeds to the output head (the final norm’s output) at one position.
Batch shape M	The number of positions in one head call; cuBLAS chooses its kernel by it.
Real logit z_i	$\langle w_i, h \rangle$ in exact arithmetic over the stored BF16 operands.
Accumulator $\hat{y}_i$	The FP32 sum a kernel forms for a logit before rounding it to BF16.
Stock head	SGLang’s head at the pinned commit: a cuBLAS BF16 GEMM with FP32 accumulation and BF16 output, then <code>torch.argmax</code> (first maximal index). Its GEMM is the <i>stock kernel</i> .
R-real, R-bf16, R-stock	The three references for one greedy decision (Definition 2.1).
Error model $\gamma_{\text{stock}}, G_i$	An assumed bound $G_i = \gamma_{\text{stock}} \sum_j w_{ij} h_j + \eta$ on the stock kernel’s accumulation error, with γ_{stock} from the conservative model or the Hopper model (Section 2.2, Table 3).
Gap condition; interval condition	The two sufficient conditions under which intervals around the stock kernel’s accumulators fix its token: a margin between the real logits (Theorem 2.3), and the rounded intervals themselves (Proposition 2.4). The certified head screens with the first and decides with the second.
Error bound β_i	A proved bound on the error of the int8 copy’s score for entry i (Theorem 3.1).
Interval $[lo_i, hi_i]$	An interval that contains the value it bounds, given the stated assumptions, computed with outward rounding; the appendices call it an enclosure.
Candidate	A vocabulary entry the screen cannot exclude; it is recomputed from its BF16 row.
Certified position	A position whose R-stock token the intervals fix for every accumulator value the error model admits.
Fallback	The stock head at the same batch shape, for a position the intervals cannot decide: on the whole batch, or per position over its gathered candidates, the per-position fallback. The evidence files call the per-position fallback the column fallback.
Capture, replay	Head inputs recorded bit for bit from the running engine; their offline re-evaluation (Section 4.1).
Held-out	From prompts that no choice of head, bound or threshold was fitted on.
Held-out replay, kernel replay, engine check	Three of the six data sources of Section 4: an FP64 replay of held-out plain-decoding positions, the GPU kernel on captured positions at their batch shapes, and the kernel inside SGLang (Table 24).
Transport; tile summary	Bounds carried from a drafter’s per-tile summaries to the target’s head input; the largest score and the sum of exponentiated scores of a tile of vocabulary entries at the drafter’s head input.
GDN layer	A Gated DeltaNet layer: linear attention with a recurrent state per request. Three of every four of Qwen3.5-4B’s decoder layers are GDN layers, the rest full attention (Section 4.1).
Arm, session	A serving configuration; one round in which the compared arms are launched afresh and swept once (Section 4.5).
Exact up to rounding	An arm whose first divergence from a matched stock reference is, on every prompt, an exact tie, one BF16 step or a near tie within two steps, with no change in output length; a class rule fixed after the first comparisons had been seen (Section 4.5).
Check mode	The certified head running inside SGLang with the stock head also computed at every certified step, counting the positions whose token differs (Section 4.3).
Gated off	A head call larger than the 64 positions the served head certifies: the same CUDA graph runs the stock head (Section 4.5).
Host-gap patches, backbone GEMM routing	Engine changes of Table 1: FlashInfer attention planning made cheaper and free of blocking GPU reads; a table that sends each backbone GEMM to the kernel a microbenchmark found fastest for its shape.
Replayed GDN decoding, fold verify	SGLang’s buffered GDN decode, which keeps recent inputs in a ring and writes the full recurrent state every 16 tokens; DFlash verification without per-position state snapshots (SGLang’s fold-every-commit protocol) (Table 1).
$\rho; \delta_i^*$	Drift $\ h^t - h^d\ _2 / \ h^t\ _2$ of a draft pair; the ρ below which transport’s bound is narrower than the int8 copy’s on entry i (Proposition 5.2).
Batch size B , concurrency c	B is the number of requests in one decode step; c is the number of requests the client keeps in flight, which the server may cap below c .
Bound family, tiling	The geometry of a transport bound (scalar, coordinate, grouped, low-rank radii, or their minimum) and the partition of the vocabulary into tiles (contiguous, random or k -means; 64 to 1,024 entries) (Appendix C.1).

Term	Definition
Fixed-noise verification	Sampled verification that accepts a drafted token exactly when it wins the target’s race under the keyed noise field (Theorem B.3).
Guards, P_γ	Interval tests that decide the standard acceptance rule from enclosures of the numerator and the partition function; P_γ is the probability that neither fires (Theorem B.5).
Keyed noise field	Gumbel noise $g_{t,i}$ that is a pure function of (seed, token position t in the sequence, token i), as in the engine’s seeded sampler.
Oracle radii	The realized per-tile deviations used as transport radii; a ceiling that no certified geometry reaches (Appendix C.6).
Static screen	Tile bounds $\langle \mu_g, h \rangle + r_g \ h\ $ evaluated at the target state without any draft (Proposition C.4).
Verify pass, draft step, cycle	The target’s forward pass over a drafted block; one drafter forward; a draft, its verification and the state commit (Appendix E.2).
<i>Terms used only in the appendices</i>	
Accepted length	Committed tokens per verify cycle, the bonus token included (Table 21).
Batch union	The set of vocabulary tiles or entries that at least one position of a batch still needs; a kernel that shares weight reads across the batch must read the whole union (Appendix C.7).
Commit rule, sound set	Commit a decision when every logit vector in a set known to contain the reference logits gives the same decision (Definition D.1).
Divergence, exposure, noise floor	The first position at which two runs choose different tokens; the positions compared before it; divergences per 1,000 exposed tokens between stock configurations that ought to agree (Appendix D.4).
Drift scaling s	A model extrapolation that replaces the draft input by $h^t - s\Delta$, scaling every drift term by s ; not a measured drafter (Appendix C.8).
First differing module output	In a bit-exact tap of two runs, the first position and, in execution order, the first module whose output bits differ; cache state is checked separately (Appendix D.4).
Frontier coordinates x, y	Mean per-request output rate including the time to first token, and output tokens per second on the GPU (Equation (9)).
Head chain	The stock head GEMM, the FP32 copy of its logits and the argmax or top-1 that follows it (Appendix E.2).
Integer pass	A certified pass that quantizes the head input as well and accumulates exactly in int32 (Theorem A.3).
Partition interval $\log(Z^+/Z^-)$	The width, in nats, of an enclosure of the partition function; it decides how often the acceptance guards fire (Appendix C.7).
Point-mass proposal	A deterministic draft token, SGLang’s default for sampled verification; its residual is a race over the untried tokens (Appendix B.4).
Residual race	An exact sample from the residual law $(p - q)_+$ by a certified race under a fixed priority field (Theorem B.6).
S-race	The reference for one seeded decision: the winner of the race with a fixed noise vector (Definition B.1).
Tuning and confirmation splits	Disjoint prompt sets of the serving workload: configurations are chosen on the first and reported on the second (Appendix E.3).